OWNER’S EDITION

Your complete RTR Digital owner’s manual.

Minute steps, from your first local sign-in to publishing, payments and a live store.

LEARN
Local training

→

PREPARE
Content + payments

→

LAUNCH
HTTPS + backups

Local setup · Every administrator workspace · Books and other digital originals · Subscriptions · Payment and bank settings · File editing · Hosting · Launch · Backups and recovery

24 OCTOBER 2026 / INDIA → WORLDWIDE / USD

Prepared for the delivered RTR Digital application. RTR Digital is the sole seller and licensor. Read the setup checks before using real customer data or live merchant credentials.

Find the step you need.

Click any entry or use the PDF bookmarks. Page numbers refer to this complete PDF, including the cover and contents.

01 Start here: your store, your files and your first decisions 6

How to use this manual without getting lost 6

What you should have received 7

Understand the pieces before changing them 8

Find your way around the application 8

Decide how a digital product will be delivered 9

Keep three environments separate 10

Know what this manual has and has not verified 11

02 Run RTR Digital on your own computer 13

1. Install the three tools you need 13

2. Download, extract and identify the correct project folder 14

3. Install the project's dependencies 15

4. Start the simplest training version 16

5. Understand where your work is saved 17

6. Create a local configuration file correctly 17

7. Stop, restart and edit without losing your place 19

8. Build and run the compiled application locally 19

9. Resolve the common first-run problems 20

10. Your local setup is complete when these checks pass 21

03 Editing, branding and maintaining the website 23

1. Know which kind of change you are making 23

2. Open the correct folder in an editor . . . 24
3. Use this source-file map . . . 25
4. Change homepage words and sections . . . 26
5. Change colors, spacing and fonts . . . 26
6. Replace the logo, favicon and hero image . . . 27
7. Change product prices and membership prices correctly . . . 29
8. Correct the current Explore $150 ceiling before selling higher-priced items . . . 30
9. Add or rename categories carefully . . . 30
10. Understand save, restart, build and deployment . . . 31
11. Keep source history private and useful . . . 32

04 Admin panel and publishing . . . 35

Entering the administrator workspaces . . . 35
Preparing a book before uploading . . . 36
Uploading a book with the standard form . . . 37
Creating a private product in Publishing studio . . . 38
Editing metadata and deciding access . . . 41
Public preview samples . . . 43
Private activation-key inventory . . . 43
Bulk changes to existing catalog records . . . 44
Customer feedback, reports, and support . . . 45
Administrator boundaries and final checks . . . 46

05 Customers, accounts and memberships . . . 52

Finding, saving and comparing products . . . 52
Accounts and sign-in . . . 54
Security, profile and privacy . . . 55
Shopping bag, checkout and order records . . . 58

Opening purchased products and learning content . . . 59
Choosing and managing a Digital Pass . . . 61
Studio workspaces and invitations . . . 62
Asking for help . . . 63

06 Payments, bank accounts and finance . . . 68

Prepare the business before enabling payment . . . 69
Open and understand Payment connections . . . 70
Set up Stripe with your deployer . . . 71
Connect PayPal . . . 73
Connect Razorpay . . . 74
Publish and change direct bank instructions . . . 75
Understand the customer payment flow . . . 77
Reconcile bank transfers . . . 78
Create and manage promotions . . . 79
Handle refunds carefully . . . 80
Seller details, receipts, reporting, and records . . . 82
Preserve keys and verify operation before launch . . . 84
Official provider reading and source authority . . . 85

07 Hosting and launch . . . 90

1. Understand what you are hosting . . . 90
2. Prepare your computer and project ZIP . . . 92
3. Buy the domain and create the VPS . . . 93
4. Prepare Ubuntu and install Docker . . . 95
5. Upload the complete source . . . 96
6. Create the production configuration . . . 98

7. Point the domain at the server . . . 101
8. Start the website and complete the first login . . . 102
9. Turn a reachable website into a launch-ready store . . . 104
10. Managed alternative: a persistent Render web service . . . 106
11. Final go-live checklist . . . 108
12. Official references . . . 108

08 Operations, backups and recovery . . . 110

1. Establish your operating notebook . . . 110
2. Daily, weekly and monthly work . . . 111
3. Read useful server status without exposing secrets . . . 112
4. What a complete backup must contain . . . 112
5. Make a consistent cold backup, step by step . . . 113
6. Restore a backup into a separate volume first . . . 115
7. Use a restored volume for an actual recovery . . . 116
8. Update code and containers without losing business data . . . 117
9. Respond to common incidents . . . 118
10. Your launch and recovery acceptance record . . . 120

09 Reference and troubleshooting . . . 121

1. Address and role reference . . . 121
2. Environment-variable reference . . . 122
3. Working limits and format behavior . . . 124
4. Diagnose a local setup problem . . . 125
5. Diagnose access, content and administration . . . 126
6. Diagnose payments and hosting . . . 127
7. Boundaries and support evidence . . . 128

CHAPTER 01

01 Start here: your store, your files and your first decisions

This is the operating manual for the delivered RTR Digital application, not a general ebook about starting an online business. It explains the actual screens and files in this project. Read the numbered steps in order the first time. When a step says to replace a placeholder, do that before running its command. Keep this PDF beside the browser and the code editor while you work.

The manual describes the redesigned application delivered on 24 October 2026. Its business model is one seller and licensor, **RTR Digital**, operating from India and selling digital products globally in **USD**. Customers can buy individual products or obtain access to eligible products through memberships. There is no marketplace seller onboarding. The Creator membership name describes a customer plan, not permission to become another seller.

How to use this manual without getting lost

The contents pages contain clickable chapter and section links. The PDF viewer's bookmarks provide another way to jump between tasks. Use Ctrl+F on Windows/Linux or Command+F on macOS to search for a button label, an error, or a variable such as `PUBLIC_URL`. Zoom screenshots when useful; the accompanying numbered instructions contain the actions you need to take.

Choose the reading path that matches your immediate task:

Your immediate goal	Read in this order	Successful result
See the site on your own computer	Start here, Local setup, then the first admin procedures	The homepage opens locally and you can enter the training administrator workspace.
Upload and sell your first book	Local setup, Admin and products, Payments, then Hosting	A genuine original, a cover, a price, and a tested purchase-to-download flow exist.
Change the appearance or wording	Local setup, Editing and customization, then the update procedure in Operations	Your source change appears locally, builds successfully, and can be deployed consistently.
Launch publicly for paying customers	Local setup, Payments, Hosting, Operations, Reference	The live HTTPS store has your real admin account, persistent data, appropriate merchant configuration, and a tested backup.
Maintain an already live store	Operations, the relevant admin chapter, then Reference	You can perform the task without replacing customer data or losing encryption keys.

Do not attempt to learn all controls by experimenting on live customer orders. First use a separate local training installation. Next use a separate HTTPS staging installation and provider sandbox accounts. Finally configure production and run the launch checks in your own live accounts. Training actions, sandbox payments, and real payments are different kinds of evidence.

What you should have received

You need the complete project ZIP and this manual. Earlier delivery also included a short deployment PDF and a short payment/bank PDF. This manual supplies the detailed procedures; keep the short guides as quick references rather than trying to combine contradictory settings from several recipes.

The existing complete project archive can be downloaded from this immutable release link:

Download the complete RTR Digital project ZIP.

1. Download the archive to a folder you can find again, such as your normal Downloads folder.
2. Keep that original ZIP untouched. Make a separate working extraction for editing.
3. Extract the ZIP. A ZIP is a compressed package; opening its preview is not the same as extracting its files.
4. Open the extracted directory and locate `package.json`. This is the **project root** for the application commands in this manual.
5. Confirm that the same directory contains `package-lock.json`, `src`, `server`, `shared`, `public`, `Dockerfile`, and `.env.example`. File extensions may be hidden in your operating system; enable them before editing configuration.
6. If you see only `LICENSE`, you have the repository's original branch rather than the application extraction. If you see a ZIP and guide PDFs only, you have the delivery branch rather than the unpacked application. Extract the complete project ZIP instead of creating substitute files.
7. Keep a private working copy somewhere sensible. Avoid a cloud-synchronized folder for an actively running SQLite data directory. A simple folder such as `C:\Projects\RTR-Digital` or `~/Projects/RTR-Digital` is easier to troubleshoot.

The ZIP contains application code and a compiled frontend, not your future live customer database, merchant account, domain registration, or bank relationship. Those are created or connected during setup. A fresh extraction is not a restore of an operating store.

Understand the pieces before changing them

Term	What it means here	What you normally do with it
Frontend	The React screens customers and administrators see in the browser	Edit text, layout, components and CSS under `src`; rebuild for production.
Backend or API	The Node server that authenticates people, validates requests and controls data	Run it alongside the frontend locally; run it continuously on the host.
Database	Private SQLite business records such as users, orders and product metadata	Change normal records through the UI; back it up consistently.
Original	The private digital file a paying or entitled customer receives	Upload it through an administrator publishing workflow.
Cover	A public image that describes a product	Supply a real PNG, JPEG or WebP in the initial standard-upload form.
Public preview	A deliberately public sample of the product	Upload a separate limited sample; never confuse it with the full paid original.
`dist`	The generated production frontend	Regenerate with the build command; do not use it as your primary editing folder.
`data` / `DATA_DIR`	The private database, uploaded originals and encryption-key storage	Keep it durable and outside public directories; protect and back up the whole set.
Environment variable	A server setting provided at startup	Set it in your local environment or host configuration, then restart/recreate the server.
Domain and DNS	The human-readable address and its routing records	Buy/control the domain and point it to the host you actually operate.
HTTPS	Encrypted browser access with a trusted certificate	Use it for the production site, sign-in cookies and payment callbacks.

The browser does not itself hold the entire store. A page can look correct while its backend is unavailable. Seeing the homepage is only the first check. Sign-in, an admin change, an actual entitled download, and a durable restart check are stronger checks of the installation.

Find your way around the application

The same application serves public pages, customer accounts and administrator workspaces. Administrator screens require an administrator account; typing their URL does not grant permissions.

Address after your domain	Who uses it	Main purpose
`/`	Everyone	Home, editorial product discovery and the animated hero.
`/explore`	Everyone	Search, filter and sort the product catalog.
`/collections`	Everyone	Curated entry points into the digital catalog.
`/compare`	Everyone	Compare selected products.
`/products/PRODUCT_ID`	Everyone, with protected actions for entitled users	Product information, sample, purchase, eligible full viewing and release tools.
`/membership`	Everyone	Compare Explorer, Creator and Studio plans and billing intervals.
`/library`	Signed-in customer or administrator	Owned/eligible content, orders and membership controls.
`/members`	Signed-in eligible account	Trial and Studio team/workspace tools.
`/account`	Signed-in account	Profile, password/security, sessions, preferences and account tools.
`/billing`	Signed-in account	Payment activity, receipts and refund requests.
`/admin`	Administrator	Overview, standard product upload and operational summaries.
`/publisher`	Administrator	Drafts, metadata, releases, large/resumable originals and publishing controls.
`/operations`	Administrator	Seller information, payment connections, promotions, refunds and finance tools.
`/support`, `/about`, `/policies`	Everyone	Help form, brand information and policy content.

For a local development installation, prepend `http://localhost:5173`, for example `http://localhost:5173/publisher`. For the production-build local check, the default backend address is `http://localhost:3001`. For your live site, prepend your actual HTTPS domain. Do not copy `localhost` into a customer email; it means the recipient's own computer.

Decide how a digital product will be delivered

Before uploading a book or any other product, create a private preparation folder on your computer. Give it a descriptive name, such as `Quiet-Garden-Book-Edition-1`. Put only the intended delivery files there. A publishing workflow is easier when the business decision and file preparation are already complete.

1. Confirm that RTR Digital has the right to distribute the content under the license you intend to offer. Changing the displayed seller name does not obtain someone else's copyright permission.
2. Choose a clear title and edition. Record which source document produced the final exported file.
3. Export the deliverable in a customer-usable format. For a book, PDF is straightforward for the browser reader; an EPUB remains a downloadable file rather than an in-browser EPUB reader in this application.
4. If customers should receive both PDF and EPUB, place the two deliverables and a short reading instruction file inside a ZIP. Upload that ZIP as the product's one current original. ZIP contents are downloaded; they are not individually streamed by the product viewer.
5. Open the actual deliverable yourself before upload. Confirm pages, fonts, audio playback or installation instructions as appropriate. The application's integrity check detects changed bytes; it does not proofread the book or establish that software is safe.
6. Create a cover image in PNG, JPEG or WebP. Keep it within the documented 10 MiB limit, and use the standard upload form when you need to supply that cover through the UI.
7. Create a separate public sample if wanted. A sample must contain only material you deliberately intend to expose without purchase.
8. Write the product description, formats, compatibility requirements, support contact, license explanation and intended USD price. Describe software's supported operating systems honestly.
9. Decide whether the product belongs in the member collection. Membership access is limited to eligible marked products; a membership is not a promise that every item is included.
10. Follow the Admin and products chapter for the chosen upload route, then check the result from a separate customer session.

The product's published description should tell a buyer what they receive. The original filename, browser viewer, activation key inventory and membership badge each solve different parts of delivery. They do not create vendor-specific DRM, license activation servers or an automated malware-scanning service.

Keep three environments separate

A **local training environment** is for learning the UI and modifying code. Development demo accounts are convenient there. Treat them as shared demonstration identities, not your real administrator account. Demo library access and demo order records do not prove that live payments work.

A **staging environment** is a separate HTTPS deployment with its own database, keys, upload storage, test users and sandbox payment credentials. Keep it away from search engines and do not present sandbox checkout as a real customer purchase. Use a different subdomain and data volume from production.

The **production environment** serves real customers. Production disables shared demo accounts. It needs an individual owner account, working HTTPS, durable private data, real merchant approval, tested mail delivery where enabled, and a backup/restore procedure. Never point staging at production's SQLite file or mounted upload volume.

You can inspect the code and train locally before deciding on a host. You cannot receive live funds merely by entering a bank name into a demonstration panel. The Payments chapter explains which receiving details belong in this application and which payout settings belong in a provider's merchant account.

Know what this manual has and has not verified

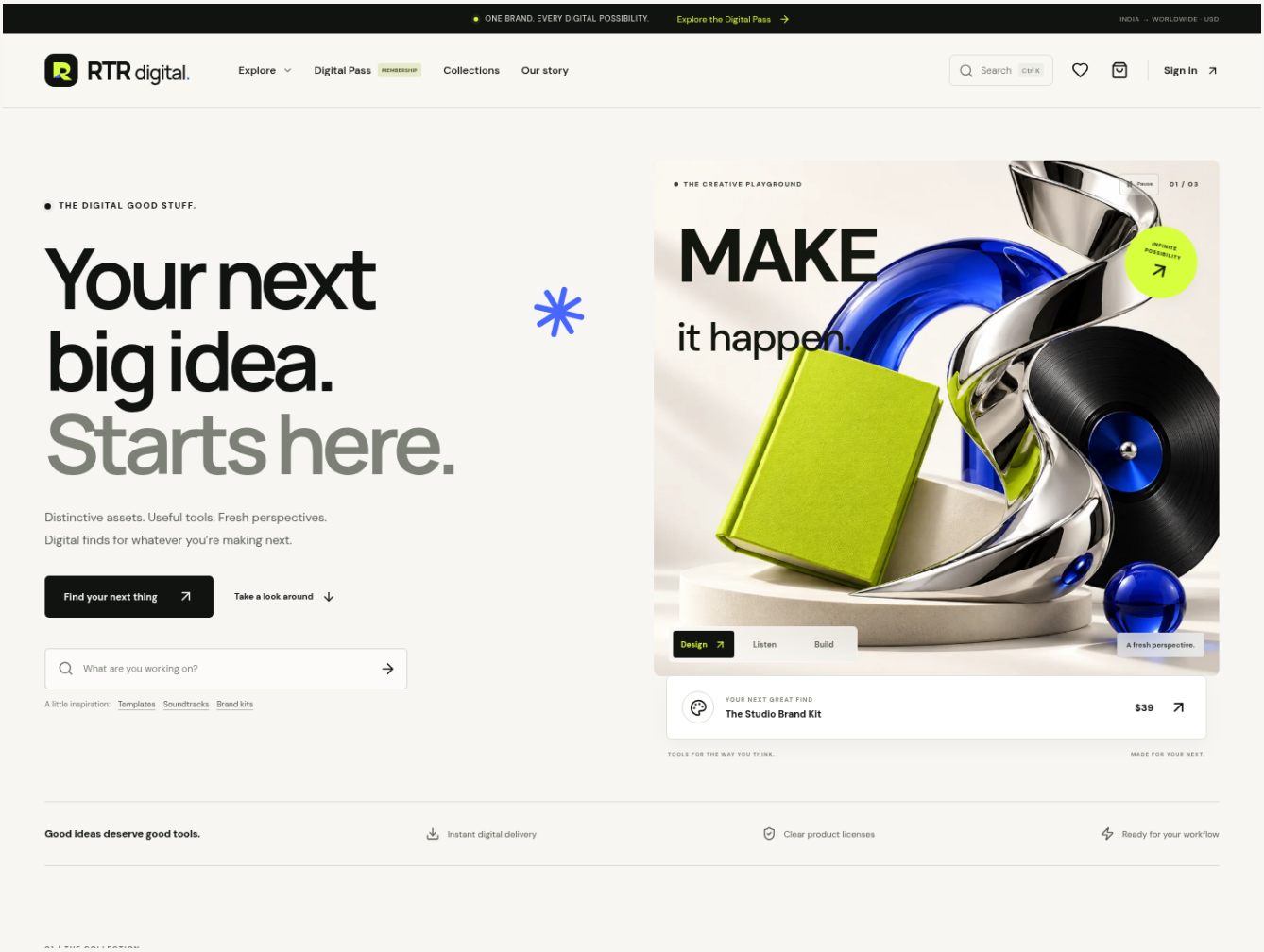
The delivered application previously passed its production build, 99 backend tests and 20 browser journeys. The large-upload workflow was also checked with a roughly 101 MiB file, including interruption and resumption. Those results describe application checks, not a tested five-gigabyte transfer, a load-capacity certificate or approval for every country/payment method.

This manual is checked against the delivered source and administrator interface. It does not claim that your domain, bank account, production server, email domain, tax registrations or real payment flow have already been configured. Host/provider dashboard labels can change, so the instructions identify the technical values you must achieve as well as the expected sequence.

When you complete a procedure, record the date, the environment, the account used and the observed result in your own private operating notebook. If something fails, save the exact error and follow the relevant troubleshooting entry before making another unrelated configuration change. This makes the difference between a repeatable setup and a collection of unexplained adjustments.

SCREEN REFERENCE

01.1 / Homepage



The storefront: Explore opens the catalog, Digital Pass opens membership plans, and Sign in opens account access.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 02

02 Run RTR Digital on your own computer

This chapter starts with an extracted project and ends with a working storefront, administrator workspace and original book on your computer. Local development is the place to learn the controls and change the design before deploying. A browser address beginning with `localhost` refers to the computer running the application. It is not your public shop address, and customers elsewhere cannot use your localhost link.

Follow one operating-system path at a time. Commands belong in a terminal, not in the browser address bar, an email, or the JavaScript console. Lines beginning with # in a configuration file are comments. Where this manual says to replace a placeholder, supply your own value; do not include angle brackets around it.

1. Install the three tools you need

Windows 10 or Windows 11

1. Open your browser and go to nodejs.org. Find the latest patched **Node.js 24.x LTS** release. This application requires at least 24.2 because it uses Node's built-in SQLite support. An older Node installation used for another project is not sufficient.
2. Download the Windows installer matching your computer, normally x64; use ARM64 only for a Windows ARM computer. Run the installer, accept the license, and keep npm and the PATH option selected. The project does not normally require the optional native-build tool bundle.
3. Close every existing terminal after installation. Open a fresh PowerShell or Windows Terminal window from the Start menu. Existing windows can retain the old PATH.
4. Enter `node --version` and press Enter. The result must begin with `v24.` and be at least `v24.2.0`. Enter `npm.cmd --version`; a numeric version confirms npm is available. The `.cmd` spelling works around PowerShell's restriction on running `npm.ps1` without changing your computer's execution policy.
5. Download and install Visual Studio Code. This is the editor used in the examples, although another plain-text code editor is acceptable. Microsoft Word is unsuitable for editing JavaScript or environment files.
6. Use Windows File Explorer for extracting the ZIP. Git is optional for this first run; install it later if you want version control.

macOS

1. Visit nodejs.org and download the latest patched Node 24.x LTS installer for macOS. Choose the appropriate Apple Silicon or Intel option if offered. Use a trusted Node version manager instead if you already maintain several Node versions and understand that workflow.
2. Run the installer. Open Applications → Utilities → Terminal after installation. Run `node --version` and `npm --version`. Confirm Node 24.2 or later within the 24.x family.

3. Install Visual Studio Code from code.visualstudio.com, move it into Applications, and open it. Approve the normal macOS first-open prompt if shown.
4. Use Finder to extract the project archive. Keep your editable copy in a normal local folder rather than editing directly inside a ZIP viewer.

Linux desktop

1. Install a current patched Node 24.x LTS runtime using the official Node download instructions or a trusted version manager. Your distribution's default `apt install nodejs` package may be older; check the actual version rather than assuming it meets the requirement.
2. Open your terminal and run `node --version` followed by `npm --version`. Resolve an old version before installing this project's dependencies. With a version manager, select Node 24 in this terminal and set the same default for future terminals.
3. Install a plain-text editor such as Visual Studio Code using its official instructions. Install your distribution's ZIP extraction tool if the graphical file manager cannot open archives.
4. Keep the project under a directory owned by your normal user. Do not use `sudo npm ci` to compensate for an incorrectly owned project folder. Correct the folder location or ownership first.

A Node message describing SQLite as experimental may appear on some compatible releases. Read the whole output: a warning alone differs from a fatal error followed by process exit. This project deliberately requires the current runtime feature; installing an old third-party SQLite package does not replace that requirement.

2. Download, extract and identify the correct project folder

1. Download the delivered **Complete Project ZIP**. Keep an untouched copy as your reference release. Downloading the PDF manual does not download the application.
2. On Windows, right-click the ZIP → **Extract All** → choose a folder such as `C:\RTRDigital`. On macOS, double-click the ZIP in Finder and move the extracted folder to your working location. On Linux, use **Extract Here** or your archive manager.
3. Open the extracted directory. Continue into a nested project directory if necessary until you see `package.json`, `package-lock.json`, `src`, `server`, `public` and `shared` together. This is the project root. Your terminal must work in this directory.
4. If you see only `LICENSE`, you have not extracted the application delivery. The public repository's main branch at the time of this manual may contain only the license; the dedicated download branch contains release artifacts. Do not run setup commands in a license-only clone.
5. Prefer a simple local path without cloud synchronization for the development database. For example, avoid putting an actively written SQLite database inside an automatically synchronized OneDrive/iCloud/Dropbox folder. You can store a completed backup there later under your own controls.
6. In VS Code, choose **File → Open Folder**, select the project root, and open it. If the editor asks whether you trust the authors, make that decision after confirming this is the project you intended to download. The Explorer sidebar should now display the source folders.

7. Choose **Terminal → New Terminal** inside VS Code. Verify the working location before running commands. On Windows, `Get-Location` and `Get-ChildItem package.json` show the location and file. On macOS/Linux, use `pwd` and `ls package.json`.

Example Windows navigation, if your extracted project is at this exact location:

```
cd "C:\RTRDigital"
Get-ChildItem package.json
node --version
npm.cmd --version
```

Example macOS/Linux navigation; replace the path with your actual folder:

```
cd "$HOME/Projects/RTRDigital"
ls package.json
node --version
npm --version
```

Quoting a path keeps spaces together. Do not copy a sample path literally when your files are somewhere else. The environment variable `$HOME` in the second example refers to your existing home directory; it is not a value to change.

3. Install the project's dependencies

1. Keep the terminal at the project root. Ensure an internet connection is available for the first dependency installation.
2. Run the command for your operating system below. `npm ci` installs the versions recorded in `package-lock.json`. It creates `node_modules`; you do not need to create that directory yourself.
3. Wait until the command finishes and the normal terminal prompt returns. The first run can take several minutes. Do not open a second installation over the same project while the first is running.
4. A successful command normally ends with an installation summary and exit status zero. Funding or audit notices do not by themselves mean installation failed. If an error appears, preserve its wording and solve it before continuing.

Windows PowerShell:

```
npm.cmd ci
```

macOS/Linux:

```
npm ci
```

Do not delete `package-lock.json` or run a forced audit fix as your first troubleshooting step. Those actions can change the dependency set and create a different project. If your package cache is unwritable, choose a cache folder owned by your user rather than granting global administrator permissions. For example, `npm.cmd ci --cache .npm-cache` works in a writable Windows project folder, and `npm ci --cache .npm-cache` is the equivalent elsewhere. Treat that extra cache folder as local tooling output when creating source archives.

4. Start the simplest training version

This first route uses the development defaults and needs no payment credentials or environment file. It runs two processes: the API on port 3001, and Vite's frontend on port 5173. Keep the terminal open while using the site.

1. In Windows PowerShell, run `npm.cmd run dev`. On macOS/Linux, run `npm run dev`.
2. Watch for the frontend URL and the API startup message. The API identifies development demo mode and says no real charges are taken. If only one process starts, read the other process's error.
3. Open a browser and enter `http://localhost:5173`. Use this exact origin for the first session. Do not alternate between `localhost` and `127.0.0.1` while debugging sign-in because browsers treat them as separate cookie origins.
4. Confirm the RTR Digital storefront appears. Try the Design/Listen/Build hero controls, product discovery, search and membership page. A visible shell alone does not prove the backend works; the next sign-in step checks that connection.
5. Select **Sign in**. In the **JUST EXPLORING? TRY THE DEMO** area, select **Admin workspace**. These shared demo controls are available only when development demo mode is enabled.
6. Open `/admin`, then use the account navigation to open **Publishing studio** at `/publisher` and **Commerce office** at `/operations`. A permission message means you are not signed in as an administrator; typing the URL does not give a customer admin rights.
7. For the customer view, sign out, open **Sign in**, and choose **Customer experience**. Use sample information only. A shared demo identity is unsuitable for your private business or for testing an individual's account security.
8. Register a separate fictional customer account when practicing verification, recovery, authenticator setup or team invitations. Those workflows depend on the individual account and should not reuse the shared administrator identity.

The development server binds to all interfaces by the included package script. Use it on your trusted computer/network and do not expose it through a router or public tunnel as your production shop. The hosting chapter uses a different production arrangement with HTTPS and demo mode disabled.

What the local addresses mean

Address	Purpose	What to expect
`http://localhost:5173`	Development frontend	Use this for normal local browsing while `npm run dev` runs.
`http://localhost:3001`	API/backend process	Supplies API responses in development; after a build it also serves the compiled website.
`http://localhost:4173`	Optional Vite preview	Static preview only; it does not start the application backend.

Vite forwards `/api` requests from port 5173 to port 3001. Changing only the API's `PORT` without changing Vite's proxy breaks that connection. Avoid changing ports during your first setup.

5. Understand where your work is saved

The default private data directory is `data` under the project root. Starting the server creates the SQLite database and supporting directories. Uploaded originals, archived releases and encryption-key files belong to this operating data. They are not stored inside `src`, and they are not replaced by editing a React component.

1. Create a training product with an obvious test title before uploading important originals. Follow the publishing chapter for the complete book workflow.
2. Stop the server with Ctrl+C, restart it, sign in and confirm that training product remains. That demonstrates persistence in this particular data directory.
3. Keep the entire data directory private. Do not put it under `public`, `dist`, a public download link, or Git. Hidden files in it include encryption keys needed to read saved sensitive records.
4. If you later select another `DATA_DIR`, the site can look empty because you are opening another database. That does not mean the old database was deleted. Record the absolute path used by each training, staging and live installation.
5. A build creates `dist`; it does not back up your books or accounts. A new project ZIP restores code, not your operating data. Use the operations chapter before moving or restoring a real installation.

Do not delete `data` to solve a forgotten password or a publication problem. That would remove much more than the visible screen you are troubleshooting.

6. Create a local configuration file correctly

Use this route when you need a personal administrator, a separate training data directory, email configuration or another backend setting. **Node does not automatically load `.env` files.** Copying the file alone does not activate it. This manual uses explicit `--env-file` startup commands below.

1. Stop the combined development command with Ctrl+C. Wait until its processes exit and the prompt returns. Starting another API on port 3001 while the old one runs causes a port conflict or can leave you browsing the wrong configuration.
2. In VS Code, copy `.env.example` to a new file named exactly `.env.local` in the project root. On Windows, enable **View → Show → File name extensions** in Explorer if you use Explorer for this task. A file named `.env.local.txt` is not the same file.
3. Keep `.env.example` unchanged as the documented template. Add your values only to `.env.local` or the hosting service's private environment controls.
4. Start with the following configuration. These are illustrative placeholders; replace the administrator email with an unused email you control and the password with a strong unique secret of at least 12 characters. Do not retain the example password.

```
PORT=3001
DEMO_MODE=true
ADMIN_EMAIL=owner@your-real-domain.example
ADMIN_PASSWORD="REPLACE-WITH-YOUR-OWN-LONG-UNIQUE-SECRET"
```

5. Save the file with Ctrl+S on Windows/Linux or Command+S on macOS. A leading # disables a setting, so remove that comment character only from settings you intend to enable.

6. Do not add `NODE_ENV=production` to this local training example. Production changes authentication cookies, catalog visibility and payment rules. A production cookie requires HTTPS; a production server opened over ordinary localhost HTTP is not the correct full production sign-in test.
7. Optionally set `DATA_DIR` to a different absolute private folder before first startup. A Windows example is `DATA_DIR=C:/RTRDigital-private/training-data`; forward slashes avoid escaping confusion. A macOS/Linux example is `DATA_DIR=/home/your-user/rtr-private/training-data`. Substitute your real path and ensure your user can write it. The folder must not be inside `public` or `dist`.
8. If a password or another value contains spaces or `#`, enclose its complete value in matching quotes. Keep one setting per line. Never paste a real secret into a support screenshot or chat to diagnose syntax.

Start the configured backend and frontend in two terminals

Use two terminal tabs so it is obvious which process failed. Both terminals must start in the project root. These commands explicitly load the backend environment file; Vite does not need merchant secrets.

Terminal A, on all three operating systems:

```
node --env-file=.env.local --watch server/index.mjs
```

Terminal B, Windows PowerShell:

```
npx.cmd vite --host 127.0.0.1
```

Terminal B, macOS/Linux:

```
npx vite --host 127.0.0.1
```

1. Start Terminal A first and confirm it is listening on port 3001.
2. Start Terminal B and confirm Vite is listening on port 5173. Since dependencies were installed locally, `npx` should use the project's Vite. Do not approve installation of an unrelated latest package if the command unexpectedly offers one; check your working directory and completed `npm ci` first.
3. Browse to `http://localhost:5173`. If already signed in as the shared demo, sign out.
4. Choose normal **Sign in**, enter the administrator email and password from your file, and select **Welcome back**. Do not select the demo button for this personal administrator.
5. Open `/admin` and `/publisher` to confirm the account has the administrator role.
6. After confirming the initial account exists, remove `ADMIN_PASSWORD` from `.env.local` and restart Terminal A. The hashed database account persists; the plaintext bootstrap password no longer needs to remain in that file. Keep the password in your password manager.

The bootstrap only creates an administrator when that email does not already exist in the selected database. It does not promote an existing customer account, and it does not change an existing administrator's password. If you registered the intended address as a customer first, choose another unused administrator address for bootstrap instead of repeatedly changing the same environment value. Use normal account password controls for later password changes.

Do not substitute `node --env-file=.env.local --run dev` for these commands. In the Node version tested for this manual, the environment-file values were not inherited by the subprocess launched through

`--run`. The explicit server command above avoids relying on that behavior. Likewise, ordinary `npm run dev` does not itself load `.env.local` into the backend.

7. Stop, restart and edit without losing your place

1. To stop a process, focus its terminal and press Ctrl+C. On macOS the terminal interrupt is still Control+C, not Command+C. In a two-terminal setup, stop both processes when shutting down the whole local site.
2. To restart with the same settings, use the same command from the same project root. Terminal history's Up Arrow can retrieve it, but confirm it is the intended command before pressing Enter.
3. For a React component or CSS change under `src`, save the file and look at the browser. Vite normally updates the view automatically. Fix a red compilation overlay before assessing the design.
4. For a backend source change, Node's `--watch` process normally restarts the backend. Watch Terminal A for errors. A frontend page can remain visible while its backend is temporarily unavailable.
5. For a change to `.env.local`, stop and restart Terminal A explicitly. Do not assume watching server source reloads environment-file changes.
6. For a dependency or package-lock change, stop the processes, run `npm ci`, then restart them. Do not update packages casually in the middle of a production incident.
7. Refresh the browser once after a backend restart if its session or open dialog needs to reload. Avoid repeatedly clicking a payment or publication action while a request is pending.

8. Build and run the compiled application locally

The production-style build checks TypeScript and generates optimized frontend files. It is a useful check before deployment, but the actual production HTTPS/payment procedure belongs in the hosting and payments chapters.

1. Stop the development processes. Keep your private data backed up before making changes to an installation containing important content.
2. Run the appropriate build command. Wait for a successful exit before starting the server.

Windows:

```
npm.cmd run build
```

macOS/Linux:

```
npm run build
```

3. With no environment-file settings needed, run `npm.cmd start` on Windows or `npm start` elsewhere. If you need `.env.local`, use the following direct command instead:

```
node --env-file=.env.local server/index.mjs
```

4. Open `http://localhost:3001`. The Express server now serves the compiled frontend and API together. Port 5173 is unnecessary for this mode.

5. Confirm sign-in, catalog loading and a protected file download. Your latest source edits appear only after a new successful build; saving a source file does not update an already built `dist` automatically.
6. Stop with Ctrl+C when finished. To return to live editing, run the development workflow again and browse port 5173.

`npm run preview` is not a replacement for this procedure. It starts Vite's static preview on port 4173 and does not start accounts, publishing, checkout or downloads. A site that looks correct in static preview can still lack a working backend.

9. Resolve the common first-run problems

“node is not recognized” or “command not found”

Close and reopen the terminal after installing Node. Run the version command again. If multiple Node installations exist, Windows `where.exe node` or macOS/Linux `which node` identifies the executable being selected. Correct the installation/PATH or version-manager selection, then retry in a fresh terminal. Do not copy a random executable into the project.

PowerShell says running scripts is disabled

Use `npm.cmd` and `npx.cmd` in the commands shown. This avoids the PowerShell script wrapper. You do not need a global unrestricted execution policy to run RTR Digital.

npm cannot find package.json

Check your current directory. Open the extracted folder containing `package.json`, not Downloads, the ZIP itself, its parent folder, or a license-only repository clone. Then run `npm ci` again.

Port 3001 or 5173 is already in use

Find your previous terminal and stop its process with Ctrl+C. Start one copy only. If Vite silently selects 5174, use its displayed URL for inspection but resolve the conflict before testing authenticated API requests, whose development origins are configured for the expected ports. Do not kill an unrelated service merely because it uses the same port. Restarting your own local training processes is usually enough.

The design appears, but sign-in says the server is unavailable

Check Terminal A/API output, Node version and port 3001. Confirm Vite still points to the default API port. A browser opened at 4173 is static preview; switch to the appropriate 5173 development or 3001 built-server mode. Read the actual error response rather than reinstalling the frontend first.

The personal administrator password does not work

Confirm normal Sign in, the exact email, and that you started the backend with `--env-file`. Check the selected `DATA_DIR`. Remember bootstrap does not change an existing account's password or promote a customer. Do not display the environment file in a shared screenshot. Use password recovery when configured, or create a separate unused bootstrap administrator under your controlled recovery procedure; do not erase the database.

A production local test will not stay signed in

Production sets secure cookies. Test the true production mode on an HTTPS staging origin with correct `PUBLIC_URL`, or use development mode for localhost training. Do not weaken production cookies to make plain HTTP behave like the public website.

A sample download differs from the book you expected

Seeded catalog items are demonstrations with labeled sample downloads. Upload your own original to a new draft, complete publication, and test the entitled account's library. Local demonstration checkout is not evidence of a real paid transaction.

A new environment value appears to do nothing

Confirm the setting is uncommented, the filename is correct, and Terminal A was restarted using `--env-file=.env.local`. Existing shell environment variables can take precedence over values in that file. Inspect the presence of a setting privately rather than printing all secrets. If your training session inherited production variables, open a clean shell or remove only the specific unintended variable before restarting.

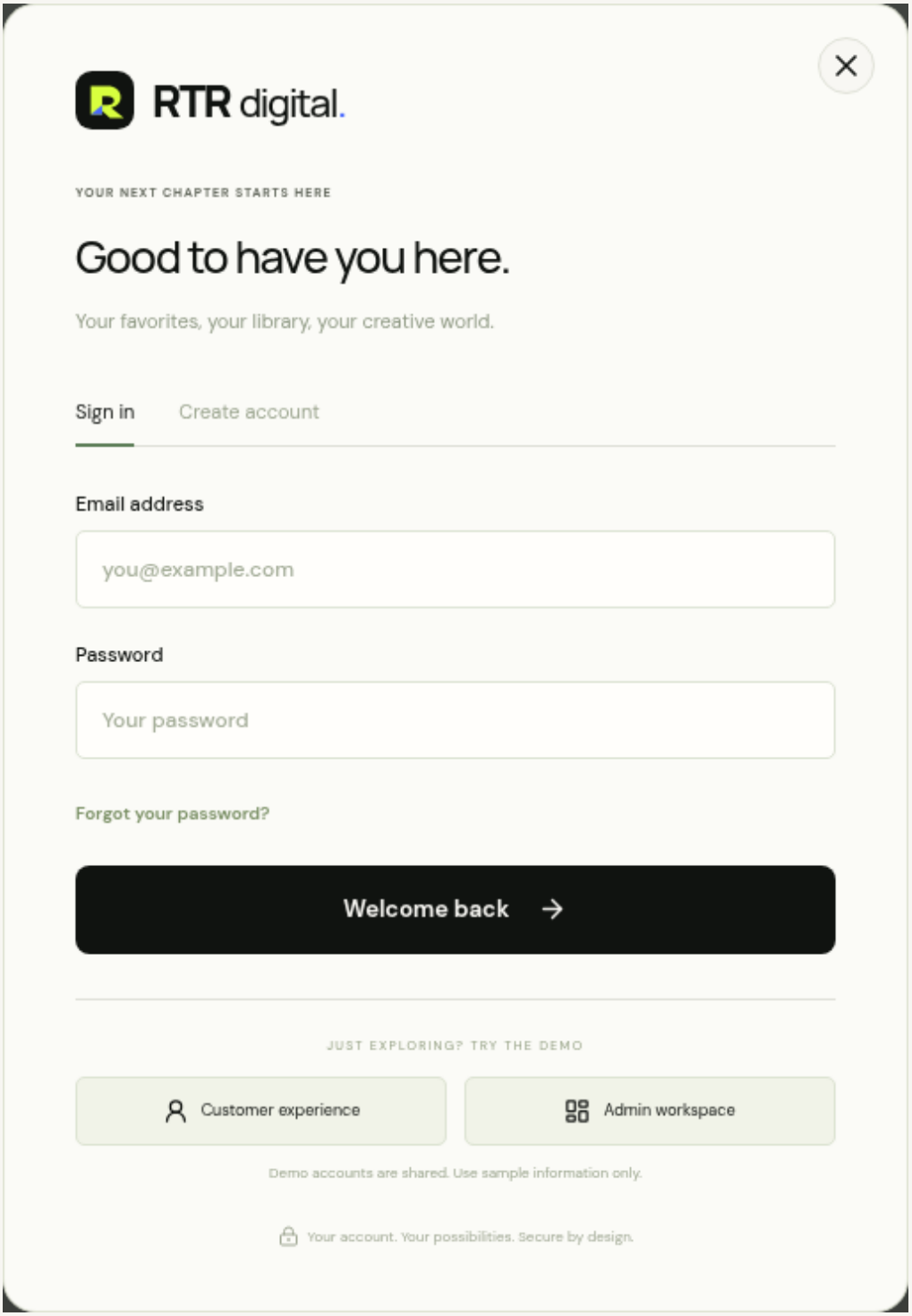
10. Your local setup is complete when these checks pass

1. Node reports a supported version and dependency installation completes.
2. Both API and frontend start without a fatal error.
3. The storefront opens, search works, and the administrator can access all three admin routes.
4. A draft containing your own test original can be published and retrieved by an entitled test customer.
5. Restarting preserves the product and account in the expected private data directory.
6. The build succeeds and the built backend serves the site at port 3001.
7. You know which data directory belongs to training and have not exposed it publicly.

These checks establish a local working installation. Live merchant eligibility, real email delivery, public DNS, HTTPS, bank clearance and tax invoicing require the separate setup and verification described later in this manual.

SCREEN REFERENCE

02.1 / Sign in dialog



Development sign-in includes Customer experience and Admin workspace demonstrations. These buttons are unavailable in production.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 03

03 Editing, branding and maintaining the website

This chapter explains how to change the delivered application, preview the result on your computer, and release that change to hosting. You do not need to edit code to add an ordinary book or change a published product's price: those are administrator tasks covered in the publishing chapter. Code editing is for the website's design, navigation, wording, supported rules and integration behavior.

Examples below describe the application delivered with this manual. A later developer may reorganize filenames. Search for the exact visible wording when a line number has moved. Keep one tested release available before making changes, and perform experiments on a local training copy rather than the only live installation.

1. Know which kind of change you are making

What you want to change	Where to start	How the change becomes visible
A book title, description, price, original file or publication status	Admin workspace / Publishing studio	Save through the authenticated administrator screen; refresh the storefront when necessary.
Bank-transfer instructions, supported PayPal/Razorpay credentials, seller contact information or promotions	Commerce office	Save the relevant settings; inspect the actual customer checkout afterward.
Homepage wording, section order, hero presentation, header or footer	React source in `src/`	Save locally for preview, build a release, then update the hosted application.
Colors, spacing, typography or responsive layout	CSS files in `src/`	Save locally, inspect several screen sizes, then build and deploy.
Stripe secret, email sender, public domain, trial toggle or upload capacity	Process environment / hosting configuration	Update private configuration and restart or recreate the application process.
Existing customer records, orders, original files or encryption keys	Private persistent data directory	Use supported administrator workflows and backup/recovery procedures; do not edit records with a text editor.

The website has no general-purpose visual page builder and no admin control that changes every piece of branding. Saving a product changes the database. Saving a `.tsx` file changes source code. Replacing `dist/` changes a compiled frontend. These are separate operations, and a production deployment must preserve the database and original files while updating the application.

2. Open the correct folder in an editor

1. Install Visual Studio Code from `https://code.visualstudio.com/` or use an equivalent code editor. An ordinary word processor is unsuitable because it can insert formatting into source files.
2. Download and extract the complete project ZIP. Keep the downloaded archive unchanged as a reference release. Open the extracted folder and confirm that it contains `package.json`, `package-lock.json`, `src`, `server`, `shared` and `public`.
3. Open the editor and choose **File → Open Folder**. Select the folder containing `package.json`, then choose **Open**. Do not select only `src`, the parent Downloads folder, or a folder containing only the ZIP.
4. If the editor asks whether you trust the folder, first confirm it is the project you intended to open. Granting editor trust enables project tooling; it is not necessary to trust unrelated downloads.
5. Choose **Terminal → New Terminal**. The prompt should show the project directory. Run `node --version`, then `npm --version`. On Windows PowerShell, use `npm.cmd --version` if script execution blocks `npm.ps1`.
6. Complete the local setup chapter, including `npm ci`. Start the local development server. Keep the terminal visible while editing so that a compilation error is immediately obvious.
7. Open `http://localhost:5173` in a browser. Make one small wording change first, save, and confirm the browser updates. This establishes that the editor and browser refer to the same project copy.
8. Open the editor's search panel with **Ctrl+Shift+F** on Windows/Linux or **Command+Shift+F** on macOS. Search within the project for a distinctive phrase visible on the page. Open the relevant match before making a change.
9. Save with **Ctrl+S** or **Command+S**. If an error appears, read the first useful error and its filename; undo the last change if necessary. Avoid editing several unrelated components while the build is already broken.

Vite normally updates the local browser after a frontend source change. If the editor shows a colored dot on the tab, the file may still be unsaved. If nothing changes, check the address bar, running terminal and open folder before trying repeated edits.

3. Use this source-file map

File or folder	Purpose
`src/main.tsx`	Browser entry point; imports fonts and global design layers and registers the service worker.
`src/App.tsx`	Main navigation and page selection, header/footer, authentication, product dialogs, bag, support and policy presentation.
`src/components/StudioHome.tsx`	Redesigned homepage, three hero scenes, homepage search, discovery sections and homepage plan cards.
`src/components/Storefront.tsx`	Product cards, Explore filters/sorting and the main membership page.
`src/components/StudioEditorial.tsx`	Editorial presentation used by collection/about experiences. Search visible wording to find a section.
`src/components/Brand.tsx`	React logo mark and product artwork presentation, including uploaded cover display.
`src/components/AccountPanels.tsx`	Library, administrator overview and standard upload form, alongside other account panels.
`src/features/PublisherStudio.tsx`	Advanced publishing interface, releases, metadata and associated publishing actions.
`src/features/CommerceOffice.tsx`	Administrator commerce operations and customer billing views.
`src/features/PaymentSettings.tsx`, `PaymentCheckout.tsx`	Payment connection controls and checkout presentation.
`src/features/AccountSettings.tsx`, `MembershipHub.tsx`, `ProductExperience.tsx`	Identity/security, memberships/teams and entitled product experiences.
`src/styles.css`, `src/account.css`, feature CSS	Base and account/feature styling still required by the application.
`src/studio-home.css`, `studio-catalog.css`, `studio-editorial.css`, `studio-shell.css`	Homepage, catalog/pass, editorial and shared shell design layers.
`src/lib/types.ts`, `api.ts`, `brand.ts`	Frontend types, API helpers and fixed seller-name constant.
`server/index.mjs`, `server/modules/`	Authentication, database, checkout, publishing, payments and other server rules.
`server/site.mjs`, `server/brand.mjs`	Production public metadata/sitemap handling and server seller branding.
`shared/catalog.json`	Demonstration catalog seed; does not overwrite existing product records.

File or folder	Purpose
`public/`	Publicly downloadable static assets copied into the frontend build. Never store paid originals, exports or secrets here.
`Dockerfile`, `.dockerignore`	Container build recipe and exclusions.
`.env.example`	Documented environment template, containing no real credentials.

The current CSS import order in `src/main.tsx` is base `styles.css`, then homepage, catalog, editorial and shell styles. Later rules may override earlier rules when their selector priority permits it. Account and feature components also have relevant style files. Deleting the base stylesheet because the new design exists will break screens that still rely on it.

4. Change homepage words and sections

1. Open `src/components/StudioHome.tsx`.
2. Search for the exact headline or supporting sentence you want to change. The hero heading is within the element whose id is `studio-hero-title`. Supporting copy uses the `studio-hero-description` class.
3. Edit only the text between the JSX tags. Keep surrounding braces, quotation marks and closing tags intact. A deliberate `<br />` creates a line break; remove or move it only after checking desktop and mobile layouts.
4. Save and inspect the full heading at desktop width and a narrow mobile width. A short phrase can fit well while a longer replacement collides with the decorative mark or forces an awkward extra line.
5. Select the homepage's primary and secondary buttons. Confirm that their wording still describes the destination. Changing button text does not change the existing `onClick` navigation behavior.
6. Search the same file for the section's heading before moving it. Move the complete JSX section, including its opening/closing tags and any related accessibility ids. Moving half a section causes syntax or layout errors.
7. Search `src/App.tsx` when changing header or footer wording. Check desktop navigation and the smaller-screen menu separately because the same destination may appear in more than one place.
8. Build after completing the change. Do not interpret an attractive local homepage as proof that the library, admin workspace and checkout still work.

To add a completely new page, a developer must coordinate the `Page` type in `src/lib/types.ts`, the `pathFor` mapping and rendering/navigation logic in `src/App.tsx`, the new component, and relevant navigation links. Public metadata/sitemap handling may also need updating in `server/site.mjs`. Adding only a menu label does not create a working route. A page requiring authentication also needs server authorization for its data; hiding a menu item is insufficient.

5. Change colors, spacing and fonts

The design currently uses ivory `#f7f6f2`, ink `#101310`, chartreuse `#d8ff3e` and cobalt `#4767ff`. These values occur both as CSS variables and as literal colors, including translucent variants and SVG fills. They are not controlled by one universal admin color setting.

1. Open `src/studio-home.css` and find the `.studio-home` variable block near the beginning. The variables include `--studio-paper`, `--studio-ink`, `--studio-acid` and `--studio-blue`.
2. On a training copy, change one accent variable, save and inspect the homepage. Keep a note of the original value.
3. Open `src/studio-catalog.css`. Its palette uses `--studio-lime` rather than `--studio-acid`. Coordinate this value with the homepage if you intend one shared accent color.
4. Search the entire `src` folder for the old hex value. Review each match rather than blindly replacing every similar number. A color such as `#4767ff13` includes an alpha channel; preserve the intended transparency when replacing its RGB portion.
5. Inspect `studio-editorial.css`, `studio-shell.css`, base/account CSS and SVG fills where needed. A successful homepage variable change may leave the header or buttons using their original literal colors.
6. Check foreground/background contrast, particularly light buttons, subdued labels, focus outlines, inputs and disabled states. Test keyboard navigation with Tab. Keep a visible focus indicator rather than removing outlines to simplify the appearance.
7. For spacing changes, find the relevant section class. The homepage uses responsive widths and media queries. Edit the intended rule and inspect all matching later rules before adding an override.
8. Test around mobile, tablet and desktop widths. Use browser developer tools' device toolbar or resize the window. Ensure no unexpected horizontal scroll and that important controls remain reachable without hovering.

The delivered fonts are self-hosted through `@fontsource-variable/dm-sans` and `@fontsource-variable/manrope`, imported in `src/main.tsx`. If changing fonts, obtain a license appropriate for website embedding, install or provide the new font, update imports and relevant `font-family` declarations, and retest headings, buttons and form labels. A different font changes character widths even at the same font size. Merely changing the CSS font name does not download a font that was never supplied.

6. Replace the logo, favicon and hero image

6.1 Keep the three logo locations consistent

1. Open `src/components/Brand.tsx`. Its `Brand` component contains the inline SVG mark and the `RTR digital.` wordmark used by the React interface.
2. Keep the brand name **RTR Digital** consistent with the intended single seller and licensor. Cosmetic logo work should not alter legal seller records or imply another marketplace seller exists.
3. If replacing the mark with your own SVG, preserve a sensible `viewBox`, responsive dimensions and the surrounding accessible brand link/text behavior. Do not paste unreviewed scripts or external tracking into an SVG.
4. Update `public/rtr-digital-logo.svg`, the standalone logo asset, to match the approved design.
5. Update `public/favicon.svg`, the browser-tab icon. Test the mark at a very small size; a detailed illustration becomes unreadable as a favicon.

6. Check `index.html` for the favicon reference and theme color, and check `public/manifest.webmanifest` for the application name/icon references. Do not claim an icon exists until the referenced file actually exists in `public/`.
7. Reload the browser. If the old favicon remains, use a private browser window or developer tools to inspect service-worker/cache behavior. `public/sw.js` currently caches `/favicon.svg` and `/offline.html` under a versioned cache name.
8. For an intentional public cache update, change that cache version in `public/sw.js`, build and deploy the new asset and worker together. The existing activation logic removes older matching public offline caches. Avoid replacing this with caching of private APIs, libraries or original files.

Changing `src/lib/brand.ts` alone does not rename the whole business. The server has its own fixed brand constant in `server/brand.mjs`, and headings/metadata include wording elsewhere. Historical payment evidence must retain its original seller snapshot. For this project, retain RTR Digital as the seller unless a separate business migration is deliberately implemented.

6.2 Replace the main hero visual without breaking layout

1. Locate `public/images/rtr-digital-signal-hero.webp`. The delivered image is 1200 × 1000 pixels, approximately 117 KB, in WebP format.
2. Make an original/licensed replacement with a similar 6:5 aspect ratio. Export an optimized WebP, keeping enough resolution for a large desktop screen without uploading an unnecessarily large source photograph.
3. Save a differently named file in `public/images/`, such as `rtr-digital-hero-edition-two.webp`. A new name helps avoid stale asset confusion.
4. Open `src/components/StudioHome.tsx`, find the image path and replace it with `/images/rtr-digital-hero-edition-two.webp`. Do not include the filesystem prefix `public` in the browser URL.
5. Update the image's `alt` text to describe the new image. The original text describes a chrome ribbon, cobalt arch, chartreuse book, vinyl record and ivory plinths; retaining that text for a different picture would be misleading.
6. Keep the image's dimensions and surrounding layout behavior coherent. Confirm that important subject matter is not clipped at a mobile width.
7. Select **Design**, **Listen** and **Build** in the hero. Only the Design scene uses this image; the other scenes include interface/CSS artwork. Replacing the image does not redesign every scene.
8. Check Play/Pause controls and keyboard operation. The delivered interaction respects reduced-motion preferences and pauses automatic progression in relevant interaction states. Preserve those controls when modifying animation.
9. Build, deploy and inspect the published asset in a private browser session. An editor preview of the image alone does not show whether the page crops it correctly.

Product cover changes are separate from hero artwork. Use the standard admin upload form's cover picker when creating an appropriately illustrated product. Publishing studio currently does not offer an editable cover-replacement control for an existing product. Do not upload paid books or software into `public/images` to make them easier to reference; that would make them public assets rather than protected originals.

7. Change product prices and membership prices correctly

7.1 Existing products belong in administrator workflows

For a published book, open Publishing studio, select the correct product and edit its price/metadata through the supported form. Confirm its public detail page and checkout afterward. Read the product-management chapter for precise controls and validation.

`shared/catalog.json` supplies demonstration records when a database is initialized. The server inserts these with `INSERT OR IGNORE` by product id. Consequently, editing a seed price does not replace an existing product with the same id in your existing database. Deleting the live database to force a new seed would also discard customers, orders, entitlements and operational history. Do not use that as a pricing procedure.

7.2 Membership pricing requires coordinated development

The current plans are Explorer at $9 monthly or $84 annually, Creator at $19 or $180, and Studio at $39 or $372. Internal ids are `starter`, `creator` and `studio`; Explorer's internal id remains `starter`.

1. Decide the new monthly and full annual prices in USD before editing. Record the intended rights, seats and renewal policy as well as the number displayed.
2. Update the authoritative plan amounts in the exported `plans` object in `server/index.mjs`. The server's `yearly` values are full annual charges.
3. Update `plans` in `src/components/StudioHome.tsx`. This homepage also stores full annual totals.
4. Update `planData` in `src/components/Storefront.tsx`. In this particular component, `yearly` is the displayed average monthly equivalent, multiplied by 12 for the billed-yearly label. For example, the current Explorer value `7` means `$84 billed yearly`, not a $7 annual subscription.
5. If the new annual total cannot be represented by the existing average-times-12 presentation without rounding errors, adjust that presentation to store/display explicit annual totals. Do not silently advertise one total and charge another.
6. Search the project for the old plan amounts, savings wording and plan descriptions. Update applicable explanatory text and meaningful tests, while avoiding unrelated numbers such as layout dimensions.
7. In your own Stripe account, configure the intended price records and portal behavior. Put the corresponding known price IDs into `STRIPE_PRICE_STARTER_MONTHLY`, `STRIPE_PRICE_STARTER_YEARLY`, and the equivalent CREATOR/STUDIO variables. These map authoritative portal plan changes to application rights; setting arbitrary IDs does not create Stripe products or prices.
8. Restart the backend and rebuild the frontend. Test a new checkout, its final charged amount, the authenticated entitlement, cancellation/resumption and any enabled portal plan change in the appropriate provider test environment.
9. Treat existing recurring Stripe subscriptions as existing contracts. Editing this application does not automatically migrate a customer's already-created Stripe subscription price. Plan any migration explicitly through the provider, applicable consent/notice rules and corresponding application verification.
10. Check PayPal, Razorpay and wire prepaid terms separately. These methods cover one prepaid period and do not become automatically renewing subscriptions because the displayed plan price changed.

Keep USD as the application's checkout currency unless a developer implements a complete currency change. Replacing the `$` symbol or a heading is not multi-currency support: amount calculations, providers, refunds, receipts and reporting must agree.

8. Correct the current Explore $150 ceiling before selling higher-priced items

The delivered Explore page initializes its maximum-price filter to $150, and its slider also ends at $150. An otherwise valid published product costing more than $150 is therefore filtered out of Explore. Selecting Reset does not solve this, because Reset returns the same ceiling. The server's product-price validation is separate and accepts prices up to $100,000. Other views may display the product, so a successful detail page does not prove Explore visibility.

For an owner whose immediate approved catalog will not exceed $1,000, the following is a concrete bounded adjustment for a developer to make on a local copy:

1. Open `src/components/Storefront.tsx` and find `export function ExplorePage`.
2. Inside that function, before the `maxPrice` state declaration, add `const priceCeiling = 1000;`.
3. Change `useState(150)` for `maxPrice` to `useState(priceCeiling)`.
4. Change the reset operation `setMaxPrice(150)` to `setMaxPrice(priceCeiling)`.
5. Within this Explore function, replace each comparison `maxPrice < 150` with `maxPrice < priceCeiling`. These comparisons control active-filter indicators and removal controls as well as the basic filter state.
6. Find the maximum-price input's `max="150"` and change it to `max={priceCeiling}`.
7. Find the price-filter removal button's `setMaxPrice(150)` and replace that with `setMaxPrice(priceCeiling)` too. Search the whole Explore function again to ensure no old default or comparison remains.
8. Save, open Explore and reset its filters. Confirm the slider now reaches $1,000 and that an authorized training product priced at $250 appears.
9. Move the slider below $250 and confirm that product disappears. Reset again and confirm it returns. Repeat search, category selection, membership-only filtering, sort and wishlist behavior.
10. Run the frontend build and relevant browser checks, then deploy using the release procedure. This is a documented code change you make; this manual does not assert the delivered application already includes it.

For a catalog with an open-ended price range, implement a dynamic ceiling derived from the loaded catalog instead. That requires coordinating state when catalog data arrives or changes, reset behavior, slider maximum, comparison labels and intentional customer selections. Simply changing the slider's `max` while leaving the initial/reset state at 150 still hides higher-priced items by default. Have the developer test asynchronous loading and later price updates as well as the first page load.

9. Add or rename categories carefully

The delivered product categories are Design assets, E-books, Audio & music, Courses, Software, Templates and Video. **All products** is a browsing choice, not a category to assign to an actual product.

Category names are referenced in several places: `server/index.mjs`, `server/modules/publishing.mjs`, `src/components/Storefront.tsx`, `StudioHome.tsx`, `AccountPanels.tsx` and `src/features/PublisherStudio.tsx`. Some category names also determine presentation artwork. The homepage's hero scenes point to existing categories.

1. Decide whether a new category is necessary or whether an existing category and descriptive product tags are enough.
2. Search the complete source for the exact old category or the relevant category array. Review both server validation and frontend options.
3. Add the new allowed value consistently, preserving exact spelling and capitalization. Update browsing icons, publishing dropdowns, homepage shortcuts and any artwork mapping that depends on it.
4. For a rename, plan how existing database products will be updated through an intentional supported edit/migration. Editing the allowed array does not rename existing records.
5. Test draft creation, standard upload, metadata editing, publication, category browsing, search and product detail display. A frontend dropdown option that the backend rejects is incomplete.
6. Build and deploy only after these paths agree. Keep existing product links and customer entitlements intact.

10. Understand save, restart, build and deployment

Change	During local development	In production
`.tsx`, frontend `.ts` or CSS	Vite normally updates after save; inspect errors.	Build the frontend and deploy the new release.
`public/` asset	Browser refresh may be needed; inspect asset path/cache.	Rebuild/redeploy and verify cache behavior.
Server `.mjs`	The documented watch process normally restarts after save.	Restart/recreate the backend using the new code.
Environment file	Stop/start with explicit environment loading.	Update hosting environment; recreate/restart as required.
Dependency or lockfile	Install the intended dependencies and run checks.	Rebuild using the matching lockfile.
Saved admin product/settings	Usually no code build; re-fetch/refresh the relevant view.	Database-backed change is already on that installation.

1. In the project terminal, run `npm run build` after finishing a logical frontend change. On affected PowerShell systems use `npm.cmd run build`.
2. Resolve TypeScript/build errors before release. A development page can appear while a production build still fails type checking.
3. Run `npm test` for relevant backend changes. The project also includes `npm run test:e2e`; its browser prerequisites and environment must be available. A missing browser executable is a setup failure, not proof the commerce flow passed.

4. Inspect the built application through the actual Node backend as described in the local chapter. `npm run preview` serves a frontend preview and is not a replacement for the complete API server.
5. Commit or privately archive the exact tested source and lockfile. Record the release identifier and changes.
6. Back up the production data using the recovery chapter before deploying a change that can affect storage or schema.
7. For the Docker hosting arrangement, update source inside the configured `app` directory and rebuild/recreate the application service as described in the hosting chapter. Merely copying a changed `.tsx` file onto the host does not modify an already-running container image.
8. Preserve the existing named data volume and stable encryption keys. Do not remove volumes, point to an empty directory or overwrite live data with your training database during an ordinary code update.
9. Check HTTPS, sign-in, Explore, one protected original, payment-method presentation and administrator access after deployment. Test the specific changed function too.

11. Keep source history private and useful

The original repository's `main` branch may contain only the license, while a separate download branch contains delivery artifacts. If you want Git-based maintenance or a managed host that imports from Git, initialize/use a repository containing the extracted real source. Do not assume cloning the original `main` will reproduce the complete application.

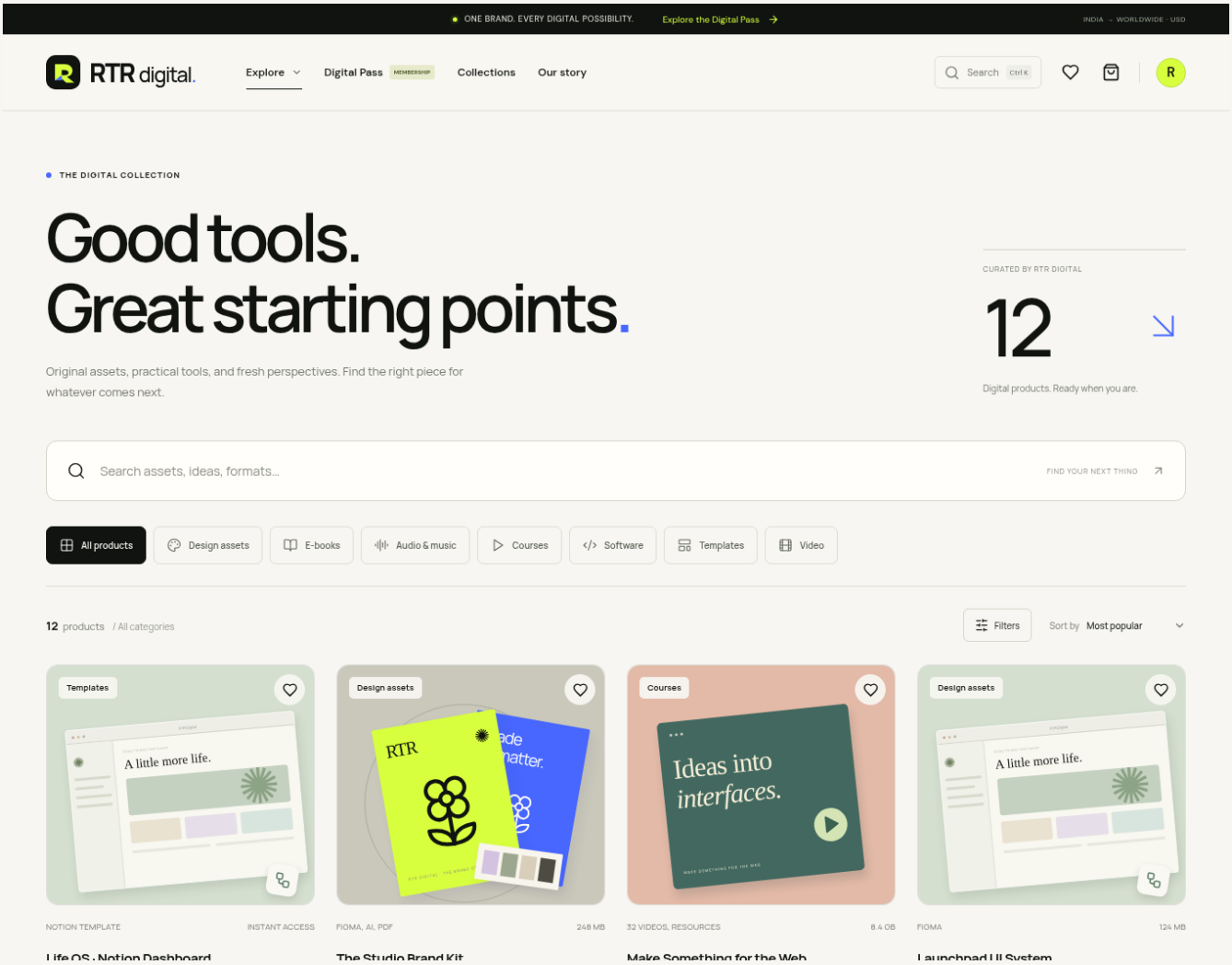
1. Prefer a private repository for business source maintenance. Create the repository in your own account and keep access limited to people you authorize.
2. Review `.gitignore` before adding files. The delivered exclusions include `node_modules/`, `dist/`, `data/`, `.env`, `.env.*` except `.env.example`, test output and deliverables.
3. If using an external `DATA_DIR`, keep that directory outside the source repository too. A custom folder name is not automatically protected by a rule that only excludes `data/`.
4. Run `git status --short` and inspect every staged file before committing. Review `git diff --cached` when files are staged. Never publish merchant secrets, bootstrap passwords, database files, customer exports, private original products or encryption keys.
5. Remember that `.gitignore` does not unpublish a file already tracked in history. If a real credential was exposed, rotate it through the provider and follow an appropriate history-cleanup procedure; deleting the current file alone is insufficient.
6. Use a descriptive commit message such as `Update homepage copy and hero artwork`. Keep unrelated payment-rule changes out of a visual-only edit so you can review and revert clearly.
7. Push to the intended private repository and branch after checking the remote. Avoid force-pushing over shared history or replacing `main` merely to obtain a downloadable build.
8. Store original paid media and business backups in appropriate private storage, not in a public source or downloads branch. A project ZIP is a release of software; a production data backup contains a different class of information.

Before handing changes to a developer, provide the intended behavior, screenshot or exact error, affected route, steps to reproduce and release identifier. Share a redacted configuration example when

needed. Keep real customer records, full bank details, passwords and keys out of general screenshots and bug reports.

SCREEN REFERENCE

03.1 / Explore filters



Catalog filters and sorting. The delivered Explore price filter has a USD 150 ceiling; follow this chapter before listing more expensive products.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 04

04 Admin panel and publishing

This chapter describes the actual administrator screens in RTR Digital. Use it when adding a book, replacing a download, preparing a release, changing a price, deciding whether members receive a product, or handling a customer question. All names and files in the examples are fictional. Substitute your own original work and authorized files.

The two main workspaces have different jobs. **Admin workspace** at `/admin` is the overview, the standard upload form, the product visibility table, recent orders, and the support inbox. **Publishing studio** at `/publisher` is the detailed editor for product metadata, versioned originals, resumable uploads, public samples, activation-key inventory, and review moderation. An upload is not a guarantee that a book is well formatted, a file is safe, or a license is legally sufficient. Open and check your own deliverables before offering them to customers.

Entering the administrator workspaces

Sign in with an administrator account

1. Open your store in a browser and select **Sign in** in the header. If you opened `/admin` or `/publisher` while signed out, select **Sign in as administrator** on the access screen instead. The sign-in form opens.
2. Enter the email address and password for the administrator account established during your installation. Select **Welcome back**. If that account uses an authenticator, complete the extra code step and select **Verify & sign in**. Successful sign-in replaces the header sign-in button with the account initial.
3. On a development or demonstration installation, the form can show **JUST EXPLORING? TRY THE DEMO**, with **Customer experience** and **Admin workspace**. **Admin workspace** opens the shared demonstration administrator account when demo access is enabled. Use fictional information there. That shortcut is not the production administrator setup procedure.
4. On a live installation, use the real administrator created by the installation/bootstrap process described earlier in this manual. **Create your account** registers a customer; it does not grant administrator privileges. If the real administrator account has not been provisioned, complete that setup through the authorized platform operator.
5. Select the circular account initial, whose accessible label is **Open account menu**. Select **Admin workspace**. The browser opens `/admin`, headed **The control room.**
6. To move to the detailed editor, open **Open account menu** again and select **Publishing studio**. The browser opens `/publisher`, headed **The publishing studio.** The signed-in workspace navigation also offers **Overview**, **Publishing studio**, and **Commerce office** for administrators.
7. Confirm you see the expected administrator controls before continuing. A customer account receives the administrator access screen and cannot publish products. A forgotten password is handled through **Forgot your password?**, not by creating a second supposed administrator account.

Read the control room before making changes

1. Open **Admin workspace** at `/admin`. Read **Total revenue**, **Orders**, **Subscribers**, and **Digital products**. Revenue separates live receipts from demonstration purchases; a demonstration amount is explicitly described as not charged.
2. Read the smaller counts for customer accounts, newsletter subscribers, and open support requests. These are counts, not an editable customer directory or a mailing tool.
3. Select **Product catalog** to inspect the product table. Each row shows **Product**, **Category**, **Price**, **Membership**, and **Status**. **Included** means the product is marked for membership access; **Purchase only** means that it requires its own purchase unless another applicable entitlement exists.
4. Use **Search products**, whose accessible label is **Search product catalog**, to narrow rows by title or category. Clear the text to see the complete table again.
5. Select **Recent orders** to read the order summary table. Use `/billing` for your own customer receipts and `/operations` for the detailed commerce administration described elsewhere in this manual. The overview table itself does not issue refunds.
6. Select the circular-arrow control labeled **Refresh dashboard** when you need a new snapshot. Wait for the loading indicator to stop. If loading fails, read the error and select **Try again** rather than assuming a missing row was deleted.

Preparing a book before uploading

Prepare the paid original and its supporting files

1. Decide what the customer is buying. For a single-format book, prepare an actual PDF or EPUB file, for example `quiet-garden-notebook-v1.pdf`. For a book supplied in several formats, prepare a ZIP such as `quiet-garden-notebook-v1.zip` containing the PDF, EPUB, any authorized worksheets, and a readable instruction/license file. Each product release accepts one original file; selecting several separate originals in the upload control is not the delivery model.
2. Open the final PDF or EPUB on your own computer. Check the cover page, contents, navigation, images, page breaks, chapter order, links, spelling, and the last page. For an EPUB, check it in a suitable EPUB reader. Merely renaming a file to `.epub` or `.pdf` does not convert its contents.
3. If supplying a ZIP, extract a copy into a new folder and open the extracted files. Confirm the archive contains the intended customer files and no private drafts, passwords, customer data, editing histories, or unrelated material. The storefront does not provide a customer-facing ZIP extractor or EPUB reader; customers download these formats and use suitable software.
4. Give the paid file a clear filename and keep a local master copy. For a resumable upload you must later choose the exact same original bytes, filename, and size. Editing the file while an upload is paused creates a different file and prevents a valid resume.
5. Prepare an optional cover as a real JPEG, PNG, or WebP image, no larger than 10 MiB. Use `.jpg`/`.jpeg`, `.png`, or `.webp` as appropriate. The standard upload checks both the extension and image signature. Changing only a filename extension does not make an image valid.
6. Prepare a separate public sample if useful: a short excerpt PDF, a small UTF-8 text sample, a short audio clip, or a short video. Do not use the complete paid book as the sample. Public previews can be accessed without a purchase on a published product.

7. Write the product description before entering the form. A useful fictional example is: “A practical 60-page notebook for planning a small balcony garden, supplied as a PDF with printable seasonal worksheets. No physical product is shipped.” State the actual formats, intended audience, contents, and any software requirements. Do not describe a PDF-only product as an EPUB bundle.
8. Decide the USD price, category, initial visibility, and whether the product belongs in the member collection. These are separate decisions: **Draft** keeps a product off the ordinary storefront, while the membership checkbox determines eligibility after publication. A price of zero is allowed, but do not use it as a substitute for checking your intended access and checkout experience.

Choose the correct upload route

1. Use **Upload a product** at `/admin` for a complete original up to 100 MiB when you want to supply a cover image in the same form. This route can save the new product as a draft or publish it immediately.
2. Use **New product** at `/publisher` for a private-first workflow, a versioned first release, or an original larger than 100 MiB. Create the metadata draft first and then upload its original under **Releases**. The configured default maximum original size is 5 GiB; the screen displays the server's current maximum.
3. Understand the current cover limitation before choosing. The browser's cover picker exists in the standard `/admin` creation form. **Publishing studio** has no cover-upload, cover-replace, or cover-remove control. A studio-created product can use the application's generated artwork. Adding an editable cover workflow requires development work; there is no hidden **Change cover** button to find.
4. For an ordinary first book, keep **Visibility** as a draft until you have checked the product and download. This allows you to correct text and access settings before customers find it.

Uploading a book with the standard form

Create a product and optional cover at `/admin`

1. Open **Admin workspace** at `/admin`, then select **Upload a product**. A dialog titled **Publish a possibility.** opens.
2. In **Product name**, enter the customer-facing name, for example `The Quiet Garden Notebook`. Use 3–120 characters. The browser input allows a longer name, but the server rejects names above 120 characters; staying within 120 avoids a failed submission.
3. In **Description**, enter a clear description of at least 20 characters. This standard dialog limits typing to 4,000 characters. The detailed studio editor later allows up to 5,000. Whitespace alone does not satisfy the minimum useful description.
4. Set **Category** to **E-books** for a book. Available categories are **Design assets**, **E-books**, **Audio & music**, **Courses**, **Software**, **Templates**, and **Video**. The category affects discovery and the appropriate artwork/access presentation; it does not convert the uploaded file.
5. In **Price (USD)**, type the amount without a currency symbol, for example `12.00`. This dialog accepts amounts from `0` through `99999` US dollars, in increments of `0.01` dollar. Use a period as the decimal separator when required by your browser. Customers see USD prices; entering `1200` means twelve hundred dollars, not twelve dollars.

6. In **Visibility**, leave **Save as draft** for a private review. Choose **Publish to storefront** only when you intend the product to become visible immediately after the upload succeeds.
7. Under **Digital product file**, select **Drop your file here, or browse**, or drag the intended original onto that area. Choose one paid PDF, EPUB, ZIP, or other deliverable. The selected filename and size replace the placeholder; read them before continuing.
8. Confirm the selected original is at most 100 MiB. If it is larger, cancel this form and use the studio draft/release procedure below. The standard form is not resumable; a failed ordinary upload must be attempted again.
9. If you need a different original before submission, click the selected file area and choose the replacement. A dropped collection of files is not assembled into a package: the form uses the first file. Build the ZIP yourself when several customer files belong together.
10. Under **Cover image (optional)**, choose the prepared JPEG, PNG, or WebP, no larger than 10 MiB. Leave the field empty if generated artwork is acceptable. The cover is display artwork; it does not replace the paid original and is not a second downloadable book.
11. Review **Include in membership**. It is checked by default. Keep it checked if eligible members should receive this product after publication. Clear it if this item should be individually purchased. This checkbox does not select a membership plan or charge the member separately.
12. Recheck the product name, category, description, price, visibility, chosen original, chosen cover, and membership checkbox. Then select **Add to catalog** once. The button changes to **Uploading…** while the request is in progress.
13. Wait for the dialog to close and the message **Your digital product has been added to the catalog.** If an error appears, read it in the dialog. Correct the specific problem and submit again. A server validation failure does not mean you should create a second unrelated product.
14. Find the new product in **Product catalog**. Confirm the price, membership state, and **draft** or **published** status. Select its title to open the product view and inspect the visible description, artwork, format, and size.
15. For detailed corrections, open **Publishing studio** at `/publisher`, search for this title, select it, and use **Metadata**. Your original is already attached. A later versioned release preserves the initial uploaded original in the version archive when it is replaced.

Creating a private product in Publishing studio

Create the metadata draft

1. Open **Publishing studio** at `/publisher` and select **New product**. The **Start with the story.** form appears above the product list.
2. Enter **Product title** using 3–120 characters, for example `The Quiet Garden Notebook`. Use a specific title that helps you distinguish this edition from another product.
3. Read **Seller & licensor**. It is fixed to **RTR Digital** and is read-only. There is no editable author/seller account field here. Put relevant authorship information in the description or book if appropriate; changing the storefront seller brand requires a code/configuration decision.
4. Choose **Category**. The initial choice is **E-books**. Select the category that describes the actual product rather than the delivery container alone: a ZIP of ebook formats can still belong in **E-books**.

5. Enter **Price (USD)**, between `0` and `100000`, with up to two decimal places. The default is `12`. Enter your intended amount rather than accidentally retaining the example default.
6. Complete **Description** with 20–5,000 characters. Describe what a customer receives and any limitations necessary to make a sound purchasing decision.
7. Review **Include in the member collection**, which starts checked. Clear it for a purchase-only product. The form explains that members can access it after publication.
8. Select **Create draft and upload**. Wait for **Draft created. Upload its first original release, then publish it from Product details.** The new product is selected and **Releases** opens.
9. Note that this action created a product record, not a completed book upload. The new record starts as **draft**, with an original still pending. If you leave now, return to the same record rather than creating another copy of the product.
10. Record its product ID from the editor heading in your private catalog notes. The generated ID is useful for bulk updates and support investigation. It is separate from the title and need not be edited when the title changes.

Upload the first original or an updated original

1. In `/publisher`, choose the correct product from the left list. Use **Find a product**, accessible as **Find a product to edit**, to search its title, category, or ID. Confirm both the editor title and ID before uploading.
2. Select **Releases**. For a new record, the history is empty. For an existing record, previous versions appear under **Every chapter, kept.**
3. Enter **Release version**, for example `1.0.0` for the first release or `1.1.0` for a corrected edition. Use a unique label for this product, at most 40 characters in the screen. Begin with a letter or number and use letters, numbers, periods, plus signs, underscores, or hyphens; avoid spaces and slashes. A version label already used for that product is rejected.
4. Under **Original product file**, choose the actual paid file. Its accessible label is **New release file** for a new upload. Choose one PDF, EPUB, ZIP, audio/video file, installer, or other intended deliverable. If replacing a book bundle, select the complete revised bundle, not only a changed chapter unless that is precisely what you sell.
5. In **What’s new?**, enter 5–4,000 characters. For a first release, `First complete edition with printable worksheets.` is sufficient. For an update, explain the customer-relevant changes, such as corrected worksheets or a new chapter. Do not put secrets or activation keys in a public-facing changelog.
6. Read the maximum file size displayed under the form. The default is 5 GiB, displayed approximately as `5.0 GB`. Hosting configuration can lower or raise it. The file must be nonempty; a folder or an empty placeholder is not a usable original.
7. Select **Publish this release**. At or below 100 MiB the form uses a regular upload. Above 100 MiB it checks the original and uses the resumable process described next. Avoid selecting the button repeatedly while it is busy.
8. Wait for **New release published. Existing customers can access the latest file.** Then check the new version, timestamp, file size, and changelog in the history.
9. Understand the two meanings of “publish.” **Publish this release** stores the original/version; it does not automatically change a draft product's visibility. For a new book, select **Metadata**, set **Visibility**

to **Published**, and select **Save product details** when ready. An already published product continues to expose the new latest release to entitled customers.

10. Open **SHA-256 integrity checksum** on the release entry to see the fingerprint. This identifies the uploaded bytes; it is not a malware-scan result, a proof of copyright, or a guarantee that the ebook opens correctly. Retain the original in your own archive and test a customer download through the authorized test workflow.

Pause, leave, resume, or cancel a large upload

1. Start a release above 100 MiB using the preceding procedure. Watch **Checking original file…** and **Checking the original file**. The browser calculates a SHA-256 fingerprint before creating the resumable session, so this stage can take time without network upload progress.
2. During **Saving your release**, watch the confirmed byte count and progress bar. The default chunk size is 8 MiB, although the platform can configure it. Confirmed chunks are kept privately on the server.
3. To stop temporarily during checking or uploading, select **Pause upload**. A saved session reports **Upload paused. Your confirmed progress is saved; resume whenever you are ready.** Pausing the initial file check before a session exists means the next attempt must check the file again.
4. Keep the original unchanged on your computer. The server saves confirmed bytes, not a reusable copy of your local file selection. Browsers require you to select the file again after returning to the page.
5. Return to `/publisher`, select the same product, and open **Releases**. Under **Pick up where you left off.**, find the saved filename and version. Sessions are retained for 48 hours without activity, not indefinitely.
6. If another session is selected, select **Resume** beside the desired session. The active row shows **Selected**. The saved version and changelog fill the form and are locked for that session.
7. Under **Original product file**, now accessible as **Reselect the original release file**, select the exact same file with the same filename, byte size, and SHA-256 fingerprint. A changed file is rejected even when you renamed it to match.
8. Select **Resume and publish release**. The browser checks the file again, obtains the server's confirmed offset, and sends the remaining chunks. An interrupted network request can be retried from confirmed progress; do not manufacture a smaller replacement file or alter the version to bypass a mismatch.
9. When **Verifying and publishing…** appears, allow the server to finish verifying the completed original. The screen says you can safely leave at this stage because finalization runs on the server. On return, use **Refresh status** for a finalizing session to reload its outcome and release history.
10. If you intentionally abandon a session, select **Cancel upload** beside it. The application removes that session's private temporary file. This is different from pausing: canceled progress cannot be resumed. Start a new upload if you need a different original, version, or changelog.
11. If the server reports an expired upload, start again. If it reports insufficient pending-upload capacity, cancel abandoned sessions or ask the operator to check capacity. The implementation limits an administrator to three active sessions and reserves pending storage; the default reservation budget is twice the configured maximum original size, or 10 GiB when the original limit is 5 GiB.

12. If a fingerprint fails, use the unchanged master copy or cancel and start a fresh session. If finalization fails, read the saved error rather than treating the product as released. A partial file is not the customer's new release until finalization succeeds.

Editing metadata and deciding access

Edit an existing product safely

1. At `/publisher`, use **Find a product** and choose the intended record. Select **Metadata**. Confirm the product ID above the title so you do not edit a similarly named book.
2. Edit **Product title** using 3–120 characters and **Description** using 20–5,000 characters. The seller name remains fixed. Changing these fields edits the catalog record; it does not edit text inside your PDF, EPUB, ZIP, or previously downloaded customer files.
3. Set **Category** if the product was misclassified. Choose from the seven listed categories. Category names are not free text in this screen.
4. Edit **Price (USD)**, between `0` and `100000`. The server rounds to cents. This changes the offered price for subsequent checkout; it does not rewrite an already recorded order total.
5. Review **Formats**. Use 1–100 characters, for example `PDF, EPUB (ZIP download)`. The browser allows 120 characters but the server accepts at most 100. This field is descriptive: typing `PDF, EPUB` does not create an EPUB or add a second download. A new release may reset the displayed primary format to its filename extension, so review the wording after uploading a new ZIP or other original.
6. In **Discovery tags**, enter concise terms separated by commas, for example `gardening, printable, beginner, PDF`. The server permits up to 12 tags of at most 50 characters each; the screen permits 500 characters overall. A comma creates a new tag. Avoid a long sentence or repeated search terms.
7. Review **Include with the Digital Pass**. Checked products are available to active eligible members when published. Clearing it makes the product purchase-only for membership access. An individual purchase remains a separate entitlement; changing member inclusion is not a refund tool.
8. Review **Visibility** and **Publish at (UTC)** together using the publication procedures below. A nonempty future schedule keeps the product a draft until its scheduled publication.
9. Select **Save product details**. Wait for **Product details saved. Your collection is up to date.** If an error appears, correct the rejected field; unsaved typing is not a stored change.
10. Select **View product** to inspect the resulting product view. If you need to judge what an ordinary visitor sees, use a signed-out/private browser separately: an administrator can inspect drafts and access files that a visitor cannot. Do not confuse administrator visibility with public publication.

Publish a finished draft immediately

1. Verify that a complete original has been uploaded and finalized. A metadata-only draft cannot be published or scheduled. The server explicitly asks you to upload and finalize an original first.
2. In **Metadata** at `/publisher`, clear **Publish at (UTC)** if the field contains a schedule you no longer want.
3. Set **Visibility** to **Published** and select **Save product details**. Confirm the status beside the editor title changes to **published**.

4. Alternatively, open **Product catalog** at `/admin` and select **Publish** on the draft's row. This also requires a complete original and clears an existing publication schedule.
5. Open the product through the storefront and confirm the title, price, description, membership indication, and any preview. Public listing and a protected original download have different access rules.

Schedule a future publication in UTC

1. Finish and finalize the original before scheduling. Open the product's **Metadata** at `/publisher`.
2. Decide the intended UTC date and time. Although the input looks like a browser date/time picker, its label is **Publish at (UTC)** and the application treats the entered digits as UTC. It does not interpret them as your computer's local time.
3. Convert any local launch time yourself before entry. For example, a launch at 14:30 in India Standard Time corresponds to 09:00 UTC on the same date. For locations with daylight-saving changes, use the correct offset for that launch date. Avoid guessing from today's offset.
4. Enter a future date and time in **Publish at (UTC)**. Select **Save product details**. A valid schedule forces the product to **draft**, even if the visibility selector had shown **Published**.
5. Read the scheduled time beside the editor heading. It is displayed with a UTC indication. Confirm the intended calendar date as well as the hour.
6. Understand the execution model: due schedules are processed when the server receives a subsequent API read request. This is not a guaranteed external timed announcement or email campaign. Check the storefront after the scheduled time rather than promising second-perfect publication independently of server availability.
7. To remove the schedule, clear **Publish at (UTC)** and save. Decide whether **Visibility** should remain **Draft** or become **Published**. Clearing the schedule alone is not an instruction to publish.
8. To postpone the launch, enter a new future UTC time and save. If you use **Publish** or **Unpublish** in `/admin`, that explicit status change clears the stored schedule; check the studio again after a visibility change.

Unpublish without deleting purchases or the release archive

1. Open `/admin` → **Product catalog**, find the published record, and select **Unpublish**. Alternatively use `/publisher` → **Metadata**, set **Visibility** to **Draft**, clear any schedule as appropriate, and select **Save product details**.
2. Confirm the product status becomes **draft**. The ordinary catalog no longer offers it as a published listing.
3. Retain the distinction between a listing and a purchase. Unpublishing does not erase existing customer purchase history or automatically revoke individual paid access. Existing entitlements are handled separately, including confirmed refunds. Membership access depends on the eligible published collection.
4. Keep the release archive when providing updates. There is no product-delete, release-delete, rollback, or file-archive button in these publishing screens. The historical list is the archive; publish a new uniquely labeled release when correcting a file. Do not claim that an older release vanished from customer history merely because the latest file changed.

Public preview samples

Upload or replace a separate public sample

1. Open the correct product at `/publisher` and select **Preview**. Read the explanation that a sample is separate from the paid original.
2. Prepare a genuine MP3, WAV, or OGG audio clip; MP4 or WebM video; PDF; or plain UTF-8 `.txt` sample. The maximum preview size is 50 MiB. Text previews must also be at most 1 MiB and contain plain text rather than HTML/script markup. The generic browser file picker may offer more audio/video formats than the server accepts; use the exact supported types above.
3. In **Preview file**, accessible as **Public product preview file**, choose the sample. An EPUB or ZIP is not an accepted inline preview. Use a short PDF excerpt for an EPUB book if appropriate.
4. Read and tick **I understand this preview is public.** Anyone viewing a published product can access that sample without buying. Upload only material you have permission to share publicly, and never attach the complete paid book here.
5. Select **Save public preview**. Wait for **Public preview saved. Customers can try this product before purchasing.** If the bytes do not match a genuine supported format, correct the sample rather than merely renaming it.
6. If a preview already existed, the new successful upload replaces it. Read **Public [kind] preview**, its MIME type, and **Open public preview**. Open the preview and confirm it is the excerpt you intended.
7. Open **View product** and inspect **A closer look.** A visitor sees the sample. An entitled account may see **Your full content** and **Public preview** as separate choices where the original supports browser viewing. Verify the distinction before publishing a paid book.

Remove a public sample

1. At `/publisher`, select the product and open **Preview**.
2. In the existing preview card, select **Remove**. Wait for **Preview removed from the product.**
3. Reopen the product view and confirm that the public sample is absent. This removes the sample; it does not delete the paid original, the product, customer purchases, or archived original releases.

Private activation-key inventory

Add genuine software activation keys

1. Use this feature only for a product that actually requires activation and for which you hold a valid stock of distributable keys. A book normally needs no activation keys. The screen does not generate legal licenses, copyright ownership, serial-number validity, or permission to resell third-party software.
2. Open the correct product at `/publisher` and select **License keys**. Read **In inventory**, **Assigned**, and **Available** before adding stock.
3. In **Private activation keys**, paste one genuine key per line. The server accepts 1–500 distinct nonempty keys in a submission, each 5–200 characters, with an overall input limit of 80,000 characters. Blank lines are ignored and duplicate keys are not added as extra inventory.

4. Recheck that these keys belong to this product. There is no supported browser action to reassign an already delivered key or to edit a pasted inventory entry afterward. Do not put keys in the description, tags, changelog, public preview, or support message.
5. Select **Add private keys**. Wait for **License inventory updated. Keys remain private until assigned.** Read the updated counts. Adding keys also marks the product as requiring a license key.
6. Entitled customers use **Claim my key** or **Show my key** in the product experience. The server assigns an available key to that account and returns the same assignment on later retrieval. Adding stock does not broadcast every key to all customers.
7. Keep your own controlled inventory records and monitor **Available** before selling activation-dependent software. Stored inventory keys are encrypted, and public catalog responses omit them, but you still need the server's private encryption material and data preserved by the backup process described elsewhere in this manual.
8. If stock is exhausted, add valid keys and ask the customer to try again. Do not promise a generated replacement, activation with the software vendor, or automatic third-party provisioning: those services are not implemented by this inventory screen.

Bulk changes to existing catalog records

Prepare an update file without creating duplicate products

1. Open `/publisher` and select each intended product to copy its existing ID from the editor heading into your working document. **Bulk update** edits existing records; it does not create new products or upload originals, covers, previews, or keys.
2. Make a small first batch containing only the fields you intend to change. The documented browser fields are `title`, `description`, `category`, `price`, `format`, `tags`, `subscriptionIncluded`, and `status`, plus the required existing `id`. Omit a field to preserve its current value. The seller remains RTR Digital.
3. Use JSON when you need clear types. A valid example is `[{"id":"REPLACE-WITH-EXISTING-ID","price":14,"tags":["gardening","printable"],"subscriptionIncluded":false}]`. Replace the example ID with a real existing ID before submission. JSON booleans are `true` and `false`, without quotation marks; `price` is a number and `tags` is an array of strings.
4. A JSON file can contain the array directly or an object with a `products` array. Include 1–100 records and use each product ID only once. An unknown ID, duplicate ID, invalid value, or product lacking an original when publication is requested rejects the import.
5. For CSV, place the column names in the first row. A simple example is `id,price,status`, followed on the next row by `REPLACE-WITH-EXISTING-ID,14,draft`. Do not submit the placeholder as if it were a real ID.
6. In CSV, write tags separated by `|`, for example `gardening|printable|PDF`. Write `subscriptionIncluded` as `true` or `false`. Use quotes around fields containing commas or line breaks, and double an internal quotation mark. An unclosed quoted field is rejected.
7. Do not add unnecessary empty columns. An included blank value is still a supplied value: a blank price can become zero during CSV parsing, and a blank required text field can fail validation. Remove the column entirely when preserving that field is your intention.

8. Stay within the same field limits as the individual metadata editor: title 3–120 characters, description 20–5,000, format 1–100, valid category, price 0–100000, and up to 12 tags of 50 characters each. The browser file picker accepts CSV/JSON files up to approximately 500 KB.
9. Do not use this documented bulk workflow for `scheduledAt`, arbitrary payload fields, media paths, order data, or user privileges. Scheduling has its own **Publish at (UTC)** field. Supplying an explicit `status` in an import clears that product's existing schedule.

Validate and apply a bulk update

1. At `/publisher`, select **Bulk update**. The dialog **Bring the collection up to date.** opens.
2. Expand **Supported fields and CSV format** and compare your intended fields with the displayed guidance. Check that your IDs are real IDs from this installation.
3. Use **CSV or JSON file** to select the prepared file, or paste the exact text into **Product updates**. A chosen file is parsed into the text area; it is not applied merely by choosing it.
4. Read the entire **Product updates** content. Recheck prices, booleans, IDs, and every `status`. Save an external record of the prior values you will need if you later choose to reverse the edit; there is no undo button.
5. Select **Validate & update catalog**. This button validates and writes in one action; it is not a dry-run-only preview. The server processes the batch as one transaction, so a validation failure prevents partial application of that batch.
6. On success, the dialog closes and a message reports the number of product records updated together. Search for several changed records and inspect **Metadata** or **View product** to confirm the intended outcome.
7. If an error appears, keep the dialog open, correct the indicated record, and submit the corrected batch. Do not remove validation merely to force publication of a product that has no original. Close the dialog with **Close bulk update** if you want to abandon the unapplied text.

Customer feedback, reports, and support

Moderate a product review

1. Open `/publisher`, select the relevant product, and select **Reviews**. Review entries show the customer's name, rating out of five, date, body, and **Public** or **Hidden** status.
2. Read the full review and any **Customer report · open** or other report entry attached to it. A report and the review's visibility are separate records. Resolving a report does not itself hide the review.
3. If the review should temporarily leave the public page, select **Hide review for review**. The screen records the standard moderation note **Hidden pending administrator review.** and preserves the original review record.
4. Confirm the review now shows **Hidden** and inspect the **Last moderation note**. The publishing screen does not provide a free-text moderation-reason editor or permanently delete the review.
5. If a hidden review should be public again, select **Restore review**. The screen records **Restored after administrator review.** and returns the entry to public review results.

6. When you have dealt with an open report, select **Resolve** beside that report. Confirm its status becomes resolved. This preserves report history. The current browser workflow does not provide an administrator **Reopen report** control.

7. Revisit **View product** and its **Reviews** tab to check public visibility. Use balanced moderation decisions and the written policies of your business. This screen's existence does not mean customer reports are automatically generated or that every report arrives by email.

Work the support inbox

1. Open **Admin workspace** at `/admin` and select **Support inbox**. It initially shows open requests through **Needs attention**.

2. Use **Search support requests** to search the customer's email, subject, or message. Select **All requests** if an expected item is absent from the open queue; select **Resolved** for completed items.

3. Openly read the request card's status, date, short request number, subject, message, and email. Match an order reference against the commerce records when the question concerns a payment or missing entitlement. Do not assume two people with similar names share an account.

4. Carry out the authorized resolution in the appropriate tool or workspace. The support card itself offers status changes only. There is no reply composer, attachment upload, internal discussion thread, or automatic outgoing response button in this inbox.

5. If you need to contact the customer, use your separately configured business support process and appropriate authorization. Changing a request's status is not evidence that an email was sent. The site's support form saves messages for administrator review, and email delivery depends on separate configuration.

6. After resolving the matter, select **Mark resolved**. Wait for **Support request marked as resolved.** The request moves out of **Needs attention** and remains in **Resolved** and **All requests**.

7. If the issue returns, find it under **Resolved** or **All requests** and select **Reopen request**. Wait for **Support request reopened.** It becomes open again.

8. Use **Refresh dashboard** for newly submitted requests. If the queue fails to load, retry before concluding that the customer submitted nothing. Never ask a customer to send passwords, recovery codes, or a full payment-card number in the support message.

Administrator boundaries and final checks

Know which changes require another screen or platform work

1. Use **Metadata** for catalog text, categories, prices, tags, visibility, member inclusion, and schedules. Use **Releases** for original file versions, **Preview** for public samples, **License keys** for private activation inventory, and **Reviews** for moderation. Those are the browser-editable publishing features.

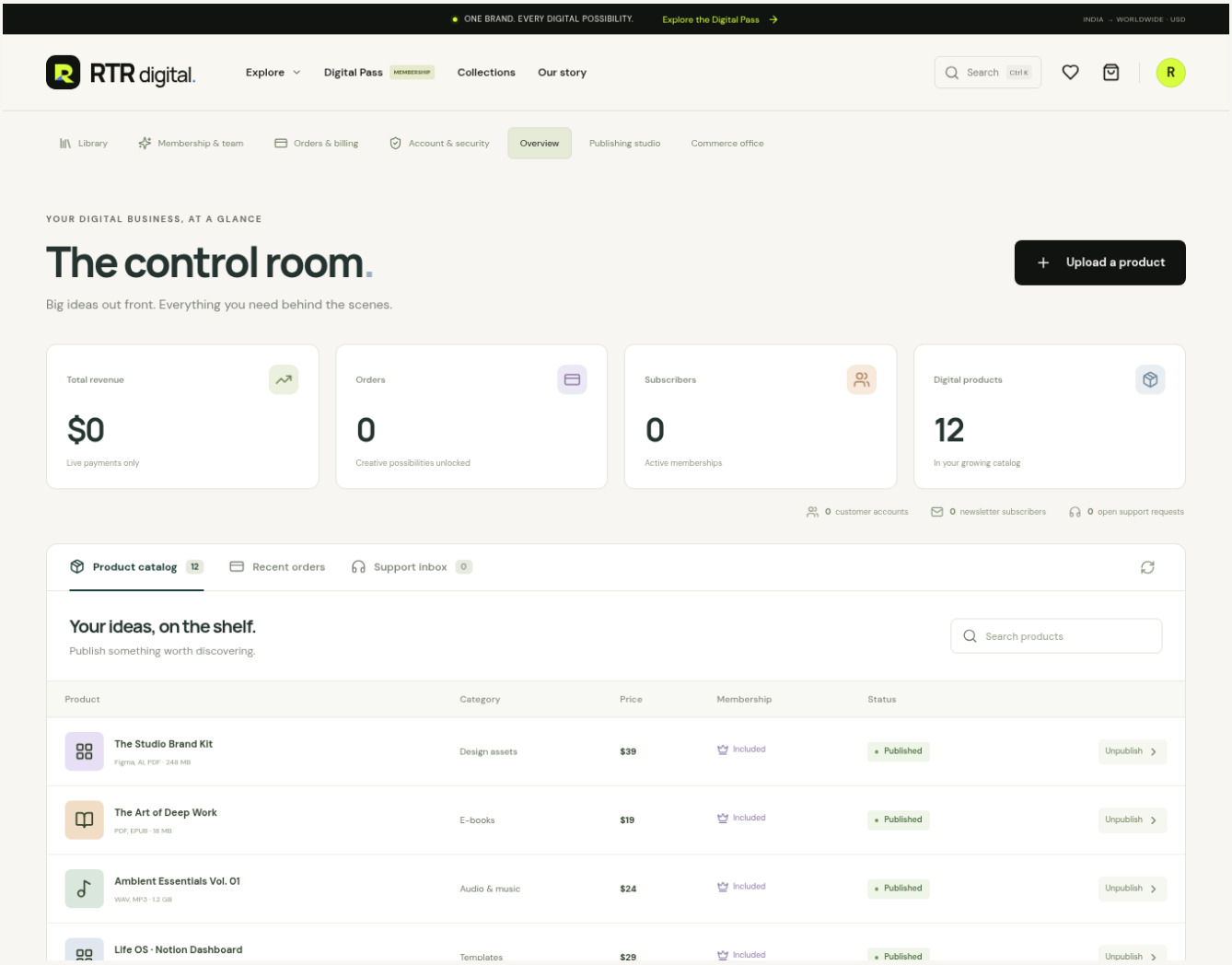
2. Use the `/admin` creation dialog when creating a product with a cover. Treat replacement/removal of a cover, deleting a product, deleting an old release, restoring a database backup, changing the brand/seller identity, and editing the category list as platform/development work; the current publishing UI does not provide those controls.

3. Use **Commerce office** at `/operations` for payment methods, promotions, financial reports, seller receipt settings, audit information, and administrator refund decisions. Do not try to express a refund by changing a product price to zero or unpublishing it.
4. Treat upload maximums, chunk sizes, pending-upload storage, real administrator provisioning, email delivery, payment-provider credentials, scheduled jobs/hosting, backups, and storage capacity as platform configuration. These are not toggles in **Metadata**.
5. Before announcing a release, confirm the product is actually published, its intended original is downloadable through an authorized customer account, the public preview contains only a sample, activation-dependent software has available genuine keys, and any promised membership inclusion matches the checkbox. These are practical publication checks, not claims of external provider verification.
6. When finished on a shared computer, select **Open account menu** → **Sign out**. Keep privileged sessions, locally saved originals, activation keys, and exported account information private.

Source references: administrator upload, catalog, library and support UI; publishing editor, imports, releases, previews and moderation UI; navigation and authentication UI; standard upload and visibility validation; publishing validation, archives, preview formats and key inventory; draft creation, resumable sessions and configurable limits.

SCREEN REFERENCE

04.1 / Admin overview

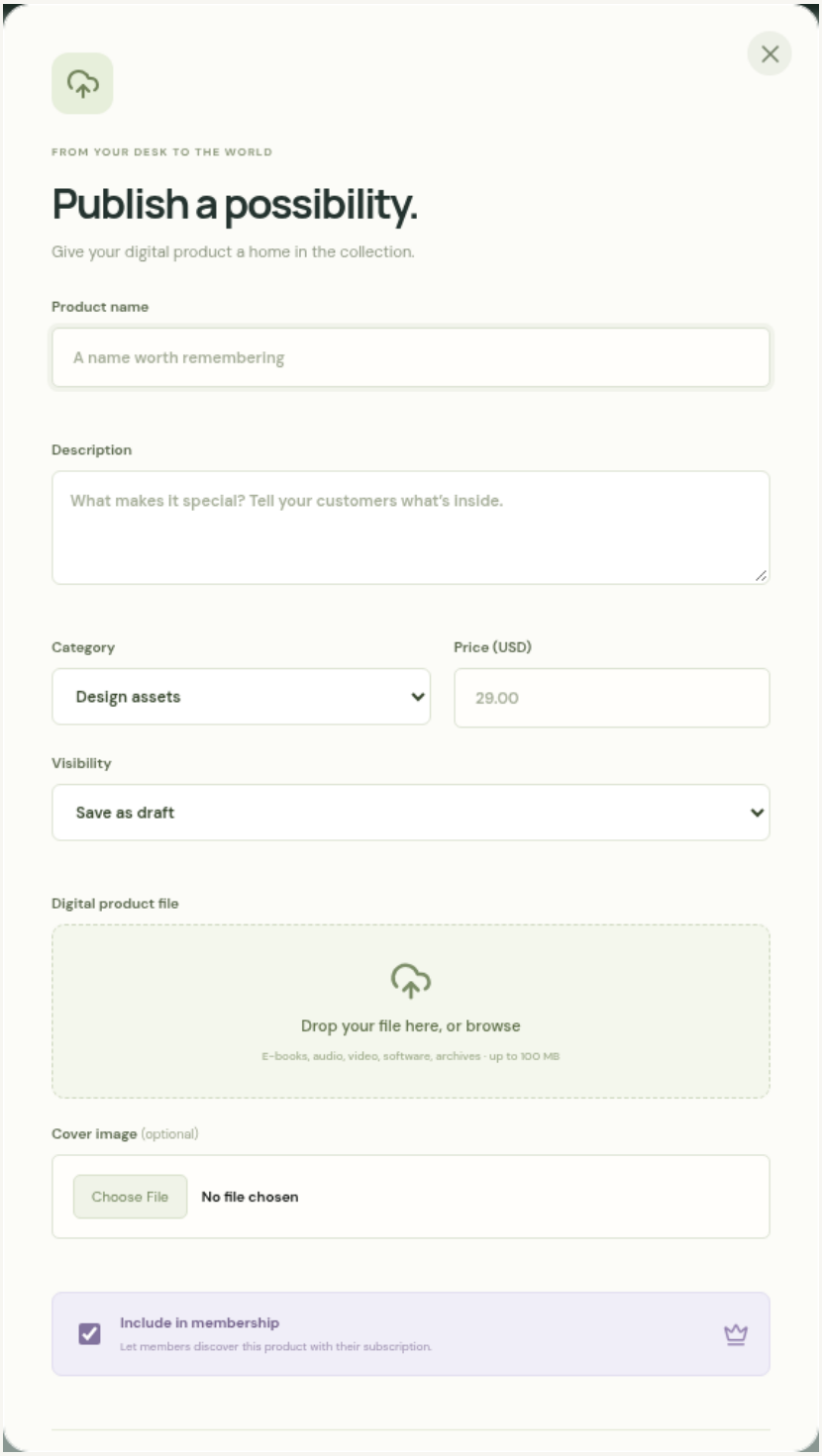


The administrator control room at /admin. Use Upload a product for the standard original-and-cover workflow.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

04.2 / Standard upload

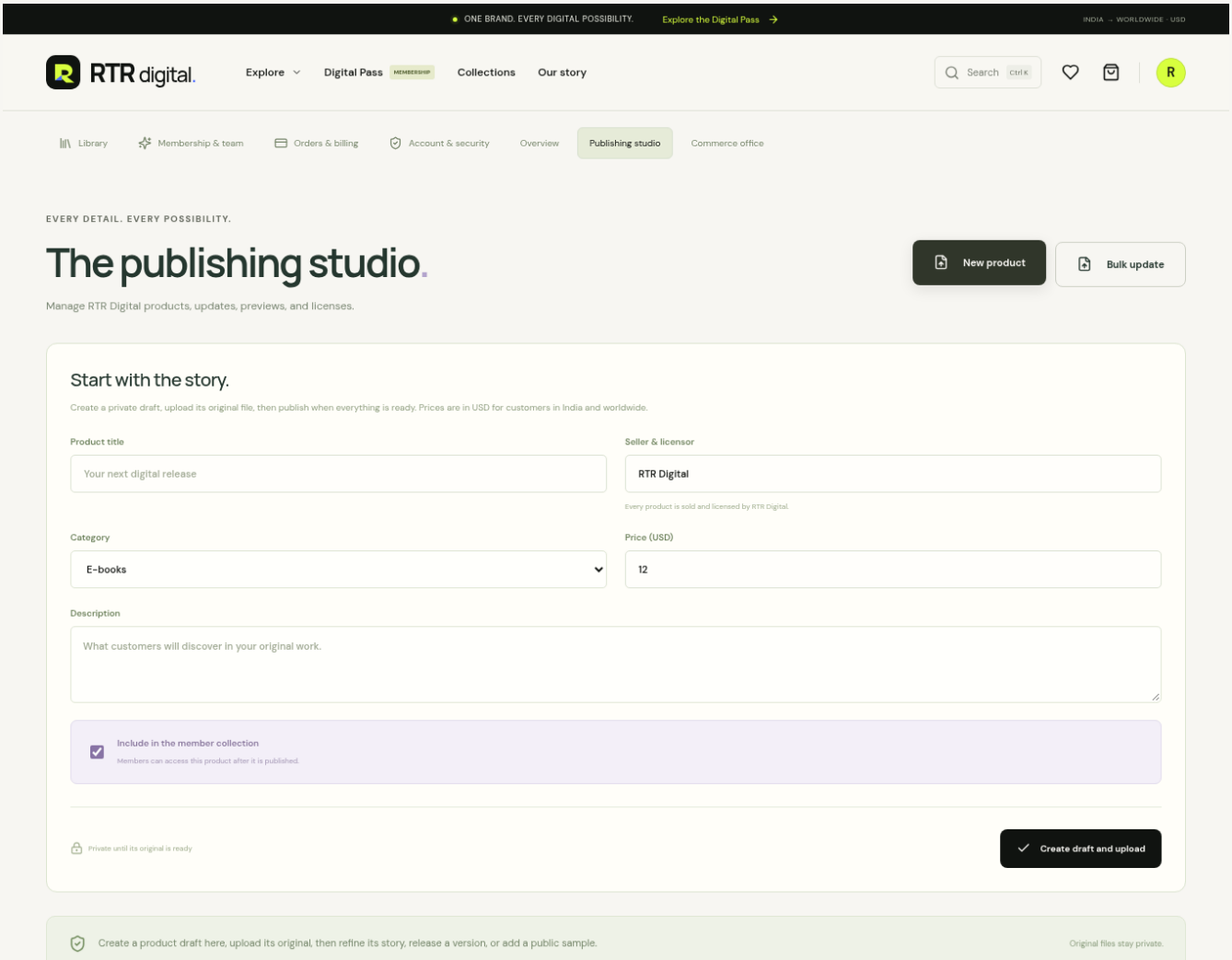


The standard upload form supplies a cover and an original. The form scrolls when the browser viewport is short.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

04.3 / New product draft

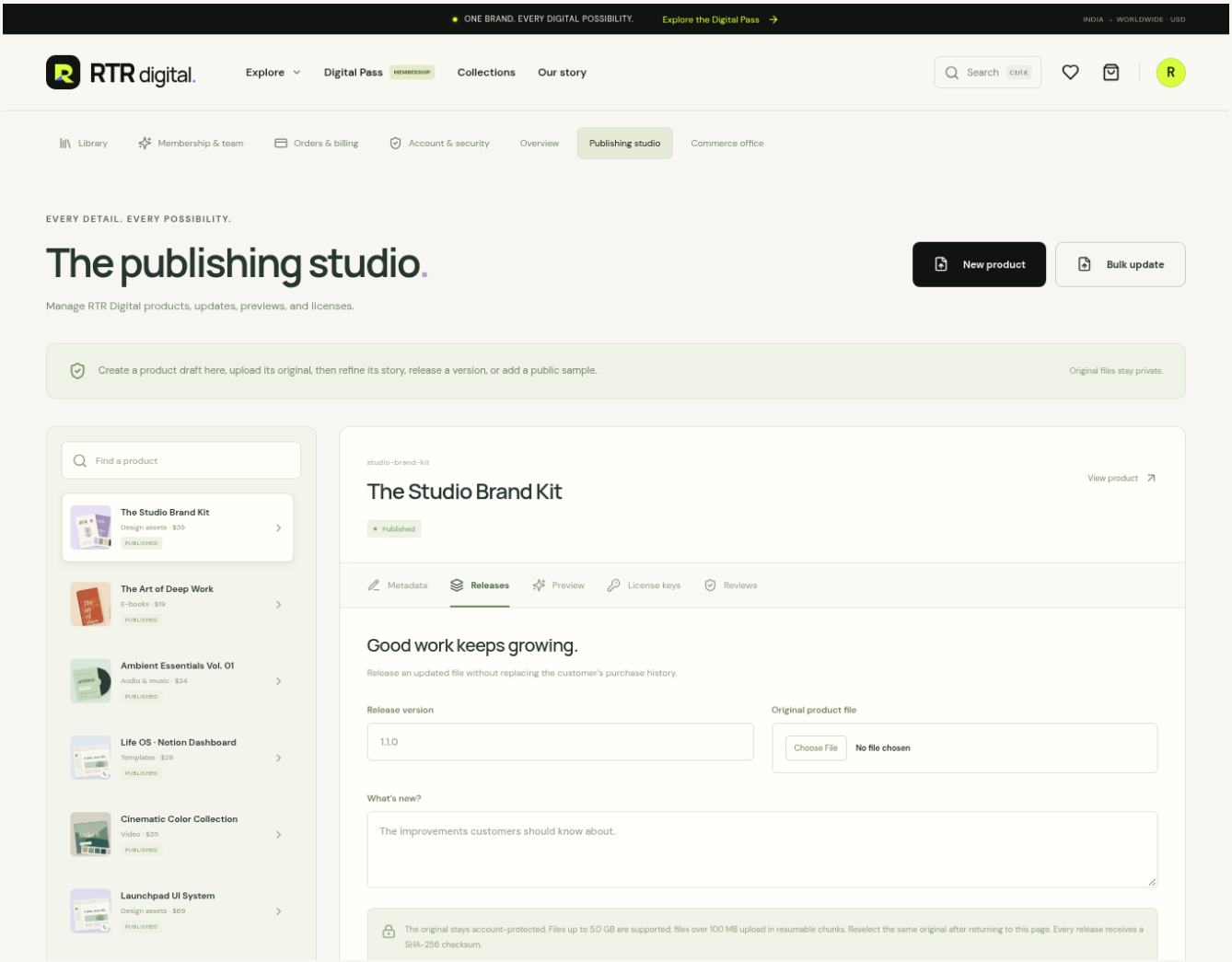


Publishing studio: New product creates metadata and a draft. An original must still be uploaded before publication.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

04.4 / Publisher releases



Publishing studio: Releases manages originals, version labels and change notes. Complete the upload before publishing from Metadata.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 05

05 Customers, accounts and memberships

This chapter follows the actual customer screens: discovering a product, saving a shortlist, signing in, protecting an account, opening paid content, buying or managing a Digital Pass, and sharing Studio access. It also explains how an owner can guide a customer without promising controls or services the application does not provide.

The account menu is the main starting point after sign-in. Select the circular initial labeled **Open account menu**, then choose **My library** (`/library`), **Account & security** (`/account`), **Membership & team** (`/members`), or **Orders & billing** (`/billing`). The public plan-comparison page is `/membership`; it is different from the signed-in membership management page at `/members`.

Finding, saving and comparing products

Browse and search the collection

1. Select **Explore** in the main navigation, or open `/explore`. The collection shows product cards with titles, formats, file sizes, prices in USD, and categories.
2. Select a category such as **E-books**, **Audio & music**, or **Templates**. Choose **All products** to return to the general category. The displayed results count changes with the current filters.
3. In **Search the collection**, type a useful title word, subject, or format, such as `garden`, `PDF`, or `audio`. Search checks the title, seller name, category, format, description, and discovery tags. Clear the search using **Clear search** to remove that part of the filter.
4. Alternatively, select the header **Search** control or press **Ctrl K** on Windows/Linux or ⌘ **K** on a Mac. In the search dialog, enter **Search products** and select **Search**, or choose a suggested category. The collection opens with the selected search or category.
5. Open **Filters** to adjust **Maximum price** or tick **Included with Digital Pass**. The price range runs from zero to $150 in $5 steps and initially uses $150. This actual Explore screen filters out products priced above $150; **Reset filters** returns to that same maximum rather than creating an unlimited price range.
6. Use **Sort by** to choose **Most popular**, **Top rated**, **Price: low to high**, or **Price: high to low**. Popularity is based on the product's recorded download count; it is not an independent recommendation or proof of quality.
7. Select **Reset filters**, **Clear all**, or the individual active-filter chips when the current combination is too narrow. If there are no results, try fewer search words and check the category and price settings.
8. Select a product's title, artwork, or **Quick view**. The product view opens at `/products/[product-id]`. Read its description, **File format**, **Digital download** size, **One-time purchase** price, and seller before adding it to your bag.

9. Read **Details** and **License & access**. Check device/software compatibility yourself where relevant. A displayed format does not mean every phone can open it, and a membership badge does not mean every product in the store is included.
10. If a banner says **Showing the sample collection. The store service is unavailable.**, select **Try again**. The fallback sample collection is not confirmation that a live order, download, or provider connection is functioning.

Save and revisit a wishlist

1. On a card, select the heart labeled **Save [product title] to wishlist**. In a product view, the heart is labeled **Save to wishlist**. The filled heart indicates the saved state; a message says the find was saved for later.
2. Select the header heart labeled **View wishlist**. The collection changes to **Your little list of possibilities.** and shows saved products that match its current catalog filters.
3. If an expected item is absent, check the category, search, price, and inclusion filter. A saved ID cannot make an unpublished, absent, or currently filtered product appear as an available listing.
4. Select the filled heart again to remove a product. The corresponding accessible label becomes **Remove [product title] from wishlist** or **Remove from wishlist**.
5. While signed out, the wishlist is stored in that browser. After sign-in, the browser's saved list is merged with the account wishlist and written to the account. Allow a moment for the account data to load before assessing it.
6. On a shared computer, remember that browser storage is local to that browser profile. Use your own browser profile for private shopping. Signing out clears the application's current wishlist, but private browsing or clearing site data also affects locally stored selections; neither action refunds an order or deletes the account itself.

Compare up to three products

1. Open `/compare`, or select **Compare digital finds** in the footer. The page **Find your kind of extraordinary.** opens.
2. Use **Add a product to compare** and choose a product from **Choose a digital product…**. Repeat for up to three products. At three, the selector says **Your shortlist is full**.
3. Compare **Category**, **Price · one-time**, **Digital formats**, **Download size**, **Seller & licensor**, **Digital Pass access**, and **What’s inside**. **Included while your plan is active** differs from **Individual purchase only**.
4. Select **A closer look** or the artwork to open a product. Comparing a product does not purchase it, add it to the bag, or create an entitlement.
5. Use **Remove [title] from comparison** to remove one selection, or **Start fresh** to clear the comparison. The shortlist is stored in the browser and can remain on that computer; it is not a private account-wide comparison history.

Accounts and sign-in

Register a customer account

1. Select **Sign in** in the header, then **Create account** in the form. The heading changes to **Make a little space for you.**
2. Enter **Your name**, between 2 and 80 characters. Use the name you want displayed in your account and any submitted reviews.
3. Enter **Email address** carefully. Use an address you control and can verify. Membership invitations are tied to the invited email, so register with that address if joining a team.
4. Enter a unique **Password**, at least eight characters. For consistency with account security and recovery, use 8–128 characters. Save it in your own password manager; support does not need to see it.
5. Select **Create your account** and wait for success. The account initial appears in the header. If you were moving from the bag or a plan choice, the application can continue to checkout after sign-in.
6. Open **Open account menu** → **Account & security** at `/account`. Check the spelling of the email and complete verification below. Registering creates a customer account even when the customer buys the Creator plan; it does not create publishing permissions.
7. If the form offers **Customer experience** under the demo controls, understand that it uses a shared demonstration account. Use sample information there. For personal verification, trials, security changes, or deletion, use your own eligible account rather than assuming a shared demo account behaves like a private production account.

Sign in and sign out

1. Select **Sign in**, enter **Email address** and **Password**, then select **Welcome back**.
2. If an authenticator is enabled, the form asks for **Authentication code**. Enter a current code from the authenticator or one unused recovery code, then select **Verify & sign in**. A password alone does not complete that account's sign-in.
3. If a code was already used, wait for a new authenticator code or use a different unused recovery code. Keep the device time accurate; time-based codes change every 30 seconds.
4. Confirm the expected name and email under **Open account menu** before purchasing. Checkout also states **Signed in as [email]**; use this to avoid buying on the wrong shared-browser account.
5. To end the current session, select **Open account menu** → **Sign out**. The site returns to the signed-out header. The shopping bag is browser storage and survives sign-out; remove its contents separately if privacy on that computer matters.

Verify an email address

1. Open `/account` → **Profile** through **Account & security**. Read **Verified email** or **Not verified yet**.
2. If unverified, select **Send verification**. Read the delivery message on that screen. A configured provider can deliver account email; an unconfigured production installation cannot deliver verification or recovery email until the operator connects delivery.

3. In your own mailbox, open the verification message and use its account link. The verification link is single-use and expires after 24 hours. If it is expired, request a new one from the profile screen.
4. On the account action screen, select **Verify email address**. Return to `/account` and confirm **Verified email**. Merely requesting the message does not verify the address.
5. On a development installation with **PRIVATE DEVELOPMENT MAILBOX**, open the relevant message there and select **Open account action**. This is a private development preview, accessible to the account and an administrator; it is disabled in production. It does not mean an actual external email was sent.
6. If delivery is unavailable, ask the owner to configure account email rather than repeatedly creating accounts. The browser profile screen does not contain provider-credential settings.

Recover a forgotten password

1. Select **Sign in** → **Forgot your password?**. Enter the account's **Email address** and select **Request recovery link**.
2. Read the response and check the account mailbox when delivery is configured. A neutral success response does not establish that any arbitrary address belongs to an account. On a development installation, use only the authorized private development-mail workflow.
3. Open the recovery link promptly. It is single-use and expires after 30 minutes. Request a new link if necessary rather than reusing an old one.
4. Enter **New password** and **Confirm new password** with the same unique 8–128-character value. Select **Save new password**.
5. Use **Return to sign in** or **Back to sign in** as shown, and sign in with the new password. A completed reset revokes the account's existing sessions. Keep any enabled authenticator/recovery access available for the subsequent sign-in.

Security, profile and privacy

Change the profile name or email

1. Open `/account` → **Profile**. Edit **Your name** using 2–80 characters, or edit **Email address** if changing the account address.
2. For a name-only correction, select **Save your details**. **Current password (required only for email changes)** is not needed merely to correct the displayed name.
3. For an email change, enter the current password in that field. If MFA is enabled, also enter a fresh **Authenticator or recovery code**. Check the new address before saving, especially if it is used for team invitations or purchase correspondence.
4. Select **Save your details** and wait for success. An email change requires verification of the new address. Complete **Send verification** and the account action again; the previous address's verified state does not automatically transfer.

Change a known password

1. Open `/account` → **Security**. Under **A fresh password.**, enter **Current password**.

2. Enter a unique 8–128-character **New password** and repeat it in **Confirm new password**.
3. If shown, supply a **Fresh authenticator or recovery code**. Use an unused code rather than one just consumed by another security action.
4. Select **Change password**. Successful change signs out other sessions, so another device may need the new password. This current session remains available for you to inspect security settings.

Enable an authenticator and save recovery codes

1. Open `/account` → **Security**. In **An extra layer of you.**, confirm the status says **Two-factor authentication not enabled**.
2. Enter **Confirm your password** and select **Set up authenticator**. The application displays a setup key and an **Open an authenticator app** link.
3. Add a time-based account named RTR Digital in your authenticator app using the displayed key, or use **Open an authenticator app** if your device supports that link. The screen provides a setup key; do not wait for an invented QR-code control.
4. Keep the setup key private and finish within ten minutes. If setup expires, start again. Make sure your device clock is set accurately.
5. Read a new six-digit code from the authenticator, enter **Six-digit authenticator code**, and select **Confirm authenticator**.
6. Confirm **Two-factor authentication enabled**. The screen now shows eight recovery codes under **Save these recovery codes now.** Each code works once, and the list is not shown again after leaving this screen.
7. Select **Copy recovery codes**, then save them in a protected place separate from the device that runs the authenticator. If clipboard access fails, select and save the codes manually. Do not put them in a support message, shared team document, or public screenshot.
8. On a later sign-in, use the authenticator's current code at **Authentication code**. If that device is unavailable, use one unused recovery code and mark it as consumed in your saved record. The account shows how many unused recovery codes remain.
9. If you intentionally need to disable MFA, return to **Security**, enter **Current password** and a fresh **Authenticator or recovery code**, then select **Disable two-factor authentication**. This is a security change requiring both factors; the app does not provide a support shortcut that simply reveals your existing secret.

Revoke sessions on other devices

1. Open `/account` → **Sessions**. Read the active sessions list and its timestamps/device information.
2. Identify the current row, labeled **This browser** and **Current**. Other rows are labeled **Another signed-in session**.
3. Select **Revoke** on a particular other session you no longer need. That session loses authenticated access; the row is not a payment or order record.
4. Select **Sign out other sessions** to revoke all sessions except this one. This is useful after losing access to an old device. If you suspect the password is known elsewhere, also change the password using **Security**.

5. Use **Open account menu** → **Sign out** when you also want to end this browser's session. Revoking other sessions does not remove locally downloaded books from those devices.

Read notices and set optional preferences

1. Open `/account` → **Notifications**. Read each title, body, and date. Account/security notices can appear here even when external mail is not configured.
2. Use the per-item check control, labeled **Mark [title] as read**, or select **Mark all read**. Marking a notice read does not cancel a membership or resolve a support request.
3. Under **Choose what reaches you.**, choose **New finds & product updates**, **Membership reminders**, and **A little creative inspiration**. Each checkbox saves when changed; there is no additional save button for these preferences.
4. Read the saved-state/error feedback. Turning off the newsletter removes the email from its sign-up list. Essential security and billing communications remain necessary. Preferences do not by themselves prove that a marketing email service or automated campaign is configured.

Export records before closing an account

1. Open `/account` → **Privacy**. Under **Take your records with you.**, select **Download account data**.
2. Save the resulting account-data download privately. It includes the profile, orders, preferences, wishlist, support records, reviews, and notices. It excludes passwords, session cookies, authenticator secrets, and recovery codes.
3. Understand that this is a record export, not a download of every purchased original. Download any entitled books/files separately through **My library** while the account and access are still active.
4. Read **Close your account.** fully before using it. Closure removes personal profile/preferences/sessions, reviews, private messages, learning notes, and invitations; loses library access; and closes any Studio workspace you own, ending teammates' shared access.
5. Deal with a live subscription first. The server requires a live subscription to end before account deletion; canceling future renewal may still leave a paid active period. Use the actual membership state rather than assuming account deletion cancels or refunds a payment.
6. Financial records keep an anonymized reference where retention is necessary. A hashed trial-eligibility marker may remain for abuse prevention. Closure does not promise to erase every financial fact, remove files already downloaded elsewhere, or create a right to another trial.
7. If you decide to close the account, enter **Current password**, a **Fresh authenticator or recovery code** when required, and exactly `DELETE` in **Type DELETE to confirm**. Select **Permanently close account** only when you intend this irreversible result.
8. If the action is blocked, read the reason. Shared demo accounts cannot be deleted, and the last real administrator cannot delete itself. Resolving those platform/account constraints is different from simply retrying the button.

Shopping bag, checkout and order records

Build and review a bag

1. Open the desired product and select **Add to bag**. The message **Added to your bag.** appears. If already present, the button says **View in your bag**; the bag contains product selections rather than adjustable quantities of the same digital product.
2. Select the header bag labeled **Shopping bag, [number] items**. Read **Your little bag of ideas**, each title and price, and **Subtotal**.
3. Remove an unwanted item with **Remove [product title]**. Adding to the bag or wishlist does not reserve a price, charge money, or give download access.
4. Understand the privacy boundary: the bag is kept in this browser's site storage and can survive sign-out. It is not an account-private server cart that automatically follows you to another device. Use your own browser profile and check the bag before checkout.
5. Select **Make them yours**. Sign in or register if prompted. On checkout, confirm **Signed in as [email]**, the exact products, the USD total, and any membership alternative you prefer before proceeding.

Complete the intended checkout and verify the outcome

1. In checkout, review the summary and use **Promo code → Apply** only if you have a valid promotion. Read the returned discount and **Total**; typing a code without applying it does not establish a discount.
2. Under **Choose your payment method**, select an available method. The application can offer Stripe, PayPal, Razorpay, bank transfer, or demo checkout depending on the operator's setup. **Unavailable** means the required setup is missing; the manual cannot make that provider live.
3. Read the terms checkbox. Individual products require agreement to digital-content terms. A Stripe membership renews until canceled; PayPal, Razorpay, and bank-transfer memberships buy one prepaid term without automatic renewal. The terms and total change with the selected method.
4. Tick the agreement only after reviewing it. Then use the displayed action: **Continue to secure payment** for Stripe, **Continue to PayPal**, **Continue to Razorpay**, **Get bank transfer instructions**, or **Complete demo checkout**.
5. For an online provider, finish the provider's secure flow and return to the store. For bank transfer, follow the separate payment instructions and await cleared-funds verification. Creating instructions or notifying the store that you paid does not itself prove funds arrived.
6. Check the order/payment state and **My library** after confirmation. If the payment is pending or under review, follow the payment-activity procedure in the payments chapter; do not create repeated orders just because a return page is slow.
7. A demo checkout explicitly collects no money and creates test access. It does not prove that real card charging, PayPal, Razorpay, international acceptance, taxes, or external email delivery have been verified.

Open a receipt or request a refund

1. Select **Open account menu** → **Orders & billing** at `/billing`. Select **Refresh orders** if you need the newest status.
2. Match the order using its full ID, titles or Digital Pass plan, date, provider label, status, and USD total. **Demo · no money collected** differs from a live provider payment.
3. Select **Open payment receipt** on that order. The receipt opens in a new tab and can be printed or saved through the browser. It records the order total in USD; it is a payment receipt, not a statutory GST/export tax invoice.
4. If a refund is needed and the order offers the action, select **Request a refund**. Enter **Reason for refund**, between 10 and 2,000 characters, describing the concrete issue and relevant details. Avoid full card numbers, passwords, or unrelated sensitive information.
5. Select **Submit refund request**, or **Cancel request** to close the unsent form. Success says the request is saved for administrator review. Submission does not guarantee approval or move money.
6. Revisit the same order to read **Refund request: [status]**, the reason, and any **Administrator note**. The order retains its request; repeatedly submitting does not create an unlimited queue of separate requests for the same payment.
7. Access follows the confirmed payment/refund state. A requested refund is not necessarily a completed refund, and a bank refund requires the owner's real banking process. For a membership, canceling renewal and refunding an order are separate actions.

Opening purchased products and learning content

Download the protected original

1. Select **Open account menu** → **My library** at `/library`. If signed out, select **Sign in to your library** and use the account that made the purchase or holds eligible membership/team access.
2. Select **My products**. Use **Search your library** to search title, category, or seller/creator text. Clear the field if it hides the expected item.
3. Find the card and check its format and file size. Select **Download**, accessible as **Download [title]**, and allow the browser to save the file.
4. Wait for **Your download is ready. Happy creating!** and check the browser's download location. A large original can take time; this card's download implementation receives the file before starting the local save. Do not assume it provides the administrator's resumable-upload controls to the customer.
5. Open PDFs with a suitable PDF reader, EPUBs with an EPUB reader, ZIPs with archive software, and other formats with their appropriate applications. If an archive contains several files, extract it before using those files.
6. If a download fails, read the message and confirm the correct account, completed order, active membership, or active Studio workspace. A pending payment and an expired membership do not supply current membership access.
7. Individual purchased entitlements remain independent of membership expiry, subject to confirmed refunds and the applicable terms. Membership-only access lasts while eligibility is active. Locally downloaded files do not remove the obligation to follow the product license.

View full content, save notes and return to a saved place

1. Select a library product's title or artwork. In **A closer look.**, entitled accounts can view supported original PDF, audio/video, or safe text content directly. Unsupported formats such as EPUB and ZIP direct you to the download instead.
2. When both are available, use **Your full content** or **Public preview** to choose. The label below distinguishes **YOUR FULL CONTENT · ACCESS PROTECTED BY YOUR ACCOUNT** from the public sample. Do not mistake a short sample for a missing paid original.
3. For a PDF, use the embedded reader or **Open your full PDF**. A visitor's corresponding link is **Open PDF preview**. Browser PDF toolbar details vary; the application does not provide a universal page-bookmark button.
4. For audio/video, use the player's controls and seek bar. Protected media supports byte-range delivery for seeking while entitled. This does not promise offline streaming or a dedicated mobile app.
5. For eligible **Courses**, **Video**, **Audio & music**, and **E-books**, enter **Your notes** with up to 10,000 characters. For a PDF, write a bookmark such as `Resume at chapter 3, page 24` because PDF page position is not automatically tracked.
6. Tick **Mark as completed** when appropriate, then select **Save my place**. Audio/video position updates while you play, but you must select **Save my place** to persist the current position, notes, and completion choice.
7. Wait for **Your place is saved. Pick up wherever inspiration finds you.** On returning with access, saved full-audio/video position is restored when media metadata loads. Public-preview playback is not the saved learning position.

Retrieve an activation key or a specific release

1. In an entitled software product that requires activation, find **Your license. Your possibilities.** and select **Claim my key**. The server assigns one private inventory key to your account if stock is available.
2. Select **Copy license key**, or select and copy it manually if clipboard access fails. Keep it private and follow the software's own legitimate activation instructions; the store does not automatically activate third-party software.
3. When returning later, use **Show my key** to retrieve the account's existing assignment. A key is not permission to redistribute original files or a substitute for the legal usage terms.
4. Under **Good things keep getting better.**, read **Version [label]**, changelog, date, size, and **Latest release**. Expand **SHA-256 file integrity** when you need its fingerprint.
5. Select **Download version [label]** for a specific archived release or **Download the latest release** for the current one. Both require access. Compare a checksum only when you know how to calculate the same SHA-256 value locally; it verifies matching bytes, not content quality or legal rights.

Share an eligible customer review

1. Open the product and select **Reviews**. Read existing feedback and any message explaining that sample ratings are illustrative.
2. Sign in with an account that has product access. Choose **Your rating** from one to five stars and enter **Your experience**, between 10 and 2,000 characters.

3. Select **Share your review** and wait for confirmation. An access check applies even though a signed-in form is visible. Administrators can hide and restore reviews while preserving their records.
4. If you need to flag a review or another content concern, use **Support** at `/support` with the product and relevant review details. The current product screen has no **Report review** button, although the server has report-handling support for connected workflows.

Choosing and managing a Digital Pass

Choose a plan and billing period

1. Open `/membership`. Choose **Monthly** or **Yearly** before selecting a plan. **Yearly** is initially selected; the large annual-card figure is a monthly equivalent, not the whole annual charge.
2. Compare the actual USD prices and rights below. Read the applicable product license too; plan wording does not grant third-party trademark rights or permit redistribution of original assets.

Plan	Monthly charge	Annual charge	Annual monthly equivalent	Stated use
Explorer	$9	$84	$7	Personal use, individual account
Creator	$19	$180	$15	Commercial use and client projects, one creator
Studio	$39	$372	$31	Project usage by a team of up to five

3. Check whether the desired products are marked **Included with Pass**. Membership unlocks eligible published products, not every product automatically. Individual purchases remain a separate option.
4. Select **Choose Explorer**, **Choose Creator**, or **Choose Studio**. Sign in if prompted, then review the plan, interval, provider, total, and renewal/prepaid wording at checkout.
5. Finish the intended payment process. Stripe membership renews monthly or yearly until canceled. PayPal, Razorpay, and confirmed bank-transfer memberships are prepaid fixed terms that expire automatically. Available methods depend on the installed provider configuration.
6. Open `/members` → **Overview** to read current status, period/preview end, billing, and payment mode. Use **Refresh membership** if necessary. A customer Creator or Studio membership does not make the account a store administrator.

Cancel, resume or change a recurring membership

1. Open `/library` and select **Membership**, or use the banner's **Manage membership**. Read the current plan, status, billing cycle, and date.
2. For a recurring membership, select **Cancel renewal** when you intend to stop the next renewal. Access continues until the paid period ends; this does not automatically refund the current order.
3. Confirm the status says **Ends after this billing period** and review **Access until**. If you change your mind before expiry, select **Keep my membership** to resume normal renewal.
4. To change an eligible recurring plan, choose **Plan** and **Billing cycle** under **A different pace?**, then select **Review plan change**. Review the new terms and any billing adjustments in Stripe before confirming there. The store does not promise an immediate free upgrade.
5. On a demo membership, the button is **Change demo plan**. It applies the test change immediately, restarts the demonstration period, and collects no payment. Do not extrapolate those test terms to live billing.

6. For a Stripe membership, **Payment methods & invoices** opens the provider billing portal. Provider-side invoices are separate from the store's printable payment receipts and do not establish statutory tax-invoice compliance by themselves.
7. For a **Prepaid membership · no automatic renewal**, there is no renewal to cancel/resume. Read its expiry date and choose a new term after it ends. A trial similarly offers **Manage free preview**, not a recurring billing cancellation.

Start or end an available seven-day preview

1. Open `/members` → **Overview** using your own account. If eligible, read **Seven days. See what sparks.** It offers seven days of Creator access with no card, payment, or automatic renewal.
2. In production, verify the account email first. The operator must have enabled real trials, and the collection must contain an eligible published original product. One trial is allowed per account/email; shared demo accounts are not eligible. A missing offer is not necessarily a broken button.
3. Tick the displayed acknowledgement that the trial expires automatically without a charge or renewal. Select **Start my seven-day trial**, or **Start my seven-day preview** when the screen explicitly describes a demonstration preview.
4. Read the resulting membership state and **Preview ends** date. Explore included products through **My library** during that access period. This preview does not automatically become a paid subscription.
5. To end early, select **End my free preview** → **Confirm end preview**. Select **Keep exploring** if abandoning that confirmation. Ending the preview removes trial access while individual purchases remain independent.
6. Read reminders under **Keep it in view.** and **Billing activity** for recorded membership events. Those in-app reminders are not evidence that an external reminder email was sent.

Studio workspaces and invitations

Create a workspace and reserve a teammate's seat

1. Sign in as the account holding an active Studio plan and open `/members` → **Your team**. A workspace requires Studio eligibility; a Creator plan does not supply five team seats.
2. Enter **Your workspace name**, 2–80 characters, and select **Create your workspace**. Use a fictional example such as `Quiet Garden Studio` when testing. The workspace banner identifies the owner and whether shared access is active.
3. Read the seat count. Studio has five total seats including the owner. Unexpired pending invitations reserve seats too, so an owner plus two members and two pending invitations uses all five places.
4. In **Invite a curious mind**, enter the teammate's exact email and select **Create invitation**. Invitations are disabled if the workspace is inactive or all seats are used/reserved.
5. Under **For your teammate’s eyes only.**, select **Copy link**. The private link is displayed once, bound to that invited email, single-use, and expires in seven days. The application explicitly says **No email is sent automatically.**
6. Share that private link with the intended teammate through your normal authorized communication channel. Do not post it publicly. If you lose a still-pending link, use the revoke/reinvite workflow; there is no **Resend invitation email** button.

7. Check **Possibilities on their way.** for the pending address and expiry. To correct the wrong address or free a reserved seat, select **Revoke** beside that invitation and create a new one for the correct email.

Accept an invitation with the intended account

1. Open the private `/members?invite=…` link supplied by the owner. If signed out, select **Sign in to continue**.
2. Sign in or create an account with the exact invited email. In production, verify that email before acceptance. If already signed in as someone else, sign out and use the intended account rather than sharing credentials.
3. Read **Join [workspace name].**, the owner's name, invited email, and expiry. Select **Accept invitation** only for the intended workspace.
4. Wait for **You’re in. Good things happen together.** The **Your team** view opens with your member row. Open **My library** for included products while the owner's Studio plan remains active.
5. An account can belong to only one workspace. An expired, revoked, consumed, wrong-email, or conflicting-workspace invitation needs resolution by the owner/account holder; repeatedly opening the link does not bypass those checks.

Rename, remove seats, leave or close a workspace

1. As owner, open `/members` → **Your team**, edit **Workspace name**, and select **Save name**. This changes the display name; it does not transfer ownership or change billing.
2. To remove another member, select the icon labeled **Remove [name] from team**. Select **Confirm removal**, or **Keep** to abandon the change. Confirmed removal ends that person's shared access and frees the seat; their independent purchases remain with their account.
3. To revoke an unused pending invitation, select its **Revoke** control. The reserved seat becomes available. Revoking an invitation differs from removing an already joined member.
4. As a member who wants to leave, select **Leave workspace** → **Confirm leave**, or **Stay** to cancel. Leaving ends this workspace's access, while your personal purchases and personal membership remain separate.
5. As owner who intends to close the team, select **Close workspace** → **Confirm close workspace**, or **Keep workspace** to cancel. Closing revokes team invitations and shared access; it does not cancel or refund the owner's own membership or purchases.
6. If the banner says **Access paused**, inspect the owner's Studio status. Team access follows that active plan. There is no owner-transfer, unlimited-seat upgrade, separate teammate billing, or file-sharing drive control in this workspace screen.

Asking for help

Submit a useful support request

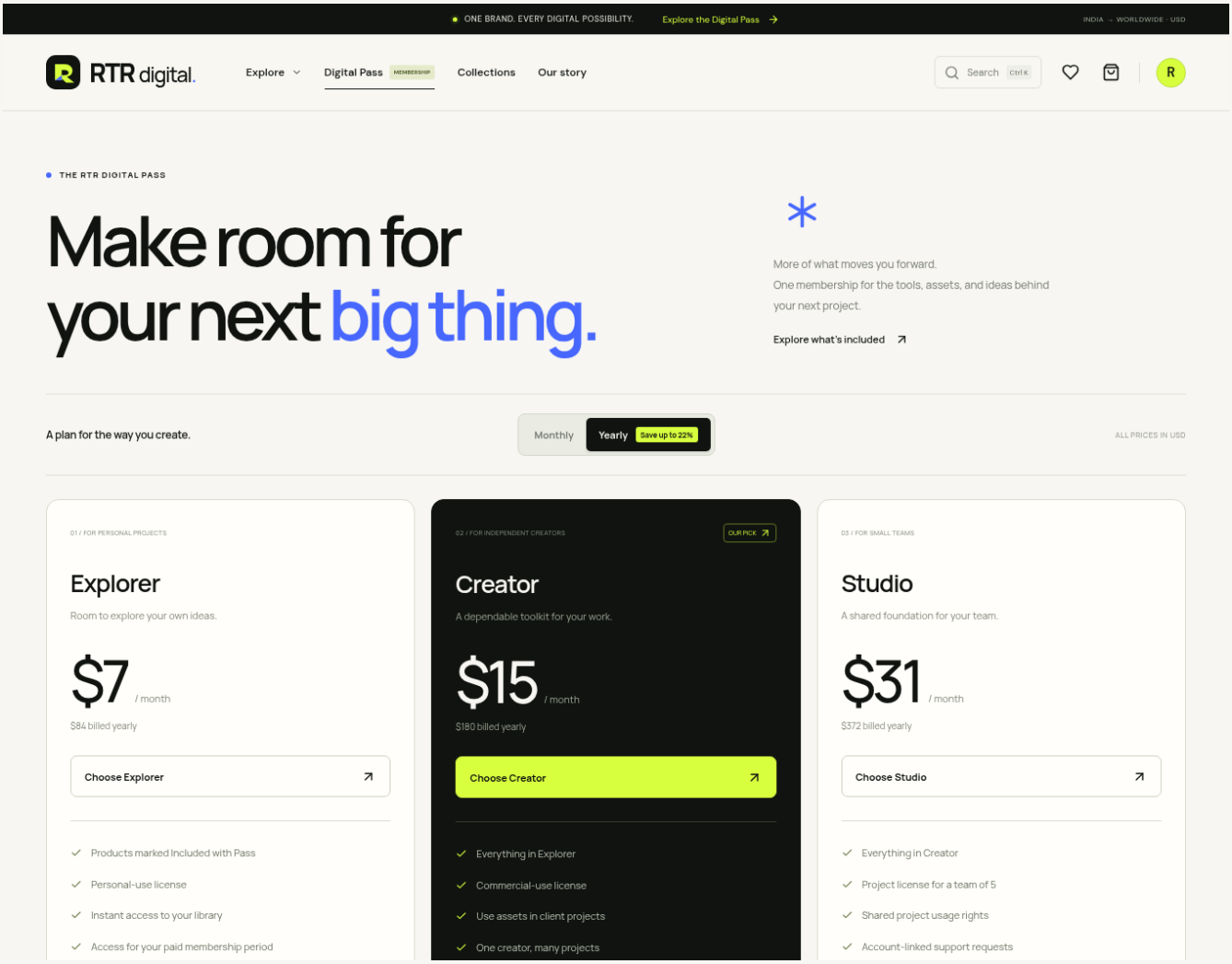
1. Open **Support** at `/support`. For a missing file, first check the correct account and **My library**; for renewal management, check **Library** → **Membership**.

2. Enter **Email address**, a 3–150-character **What’s it about?** subject, and **Your message** with 10–5,000 characters. Include the product title, full order reference when relevant, what happened, and the exact error wording.
3. Keep passwords, authentication/recovery codes, full card details, and private activation keys out of the message. The administrator needs enough context to investigate, not access to your security secrets.
4. Select **Send your message**. Wait for **Your message is saved in the administrator workspace.** and save the displayed ticket reference.
5. Use **Send another message** only when you intend a further request. The form stores requests for review; this preview explicitly says email delivery is not enabled, and the administrator inbox has status controls rather than a built-in reply conversation.

Source references: navigation, authentication, bag, wishlist, reviews and support UI; catalog filters and published plan prices; comparison UI and browser storage; profile, authenticator, sessions, preferences, export and deletion UI; library downloads and recurring membership controls; trials, workspaces and invitations UI; full content, notes, keys and archived-release UI; customer receipts and refunds UI; checkout wording and available payment methods; identity enforcement; trial/team rules; protected original viewing; publishing access and key rules; refund requests and receipt behavior.

SCREEN REFERENCE

05.1 / Membership plans

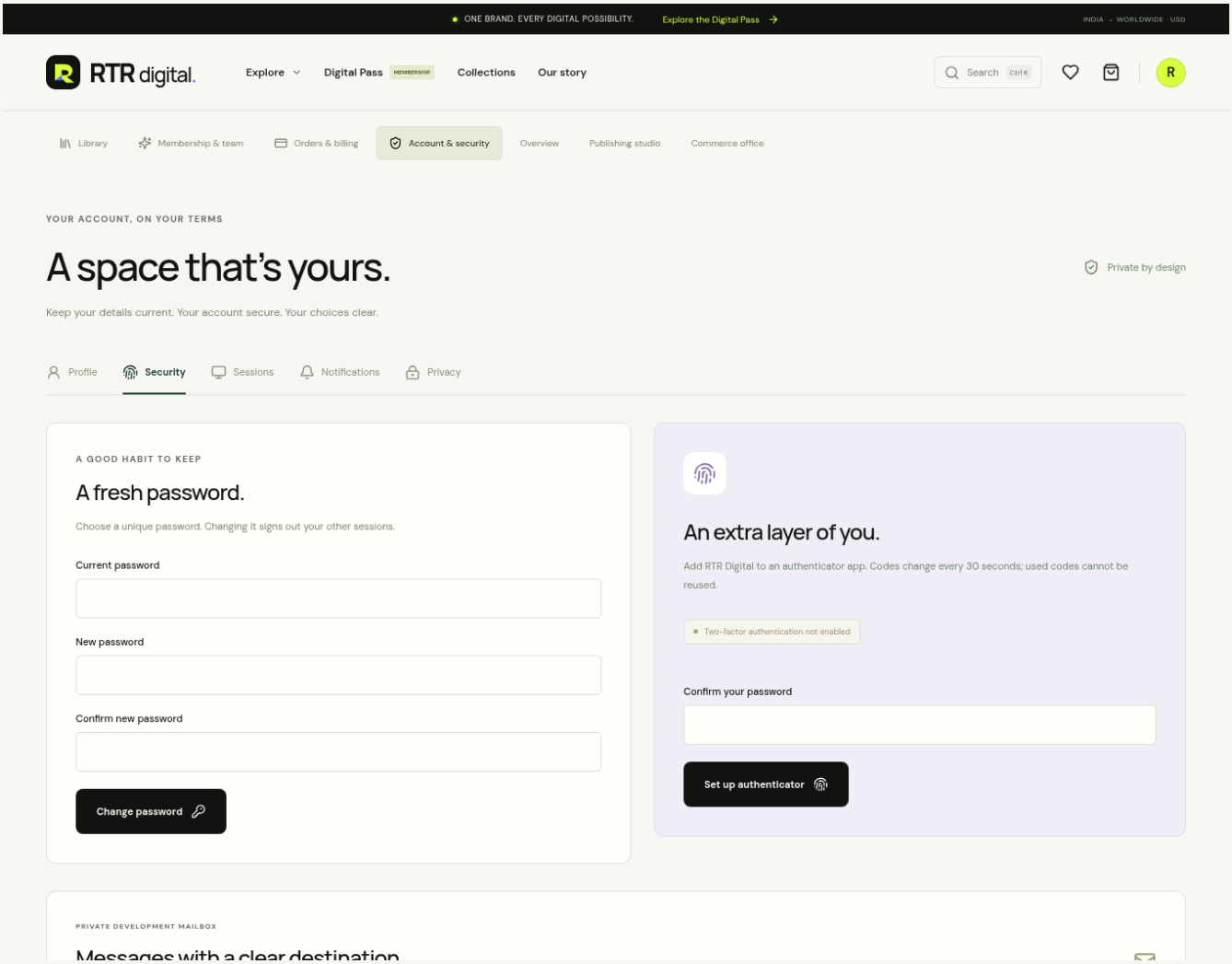


Yearly Digital Pass comparison. The monthly equivalent is a display calculation; the annual total is the charge.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

05.2 / Account security

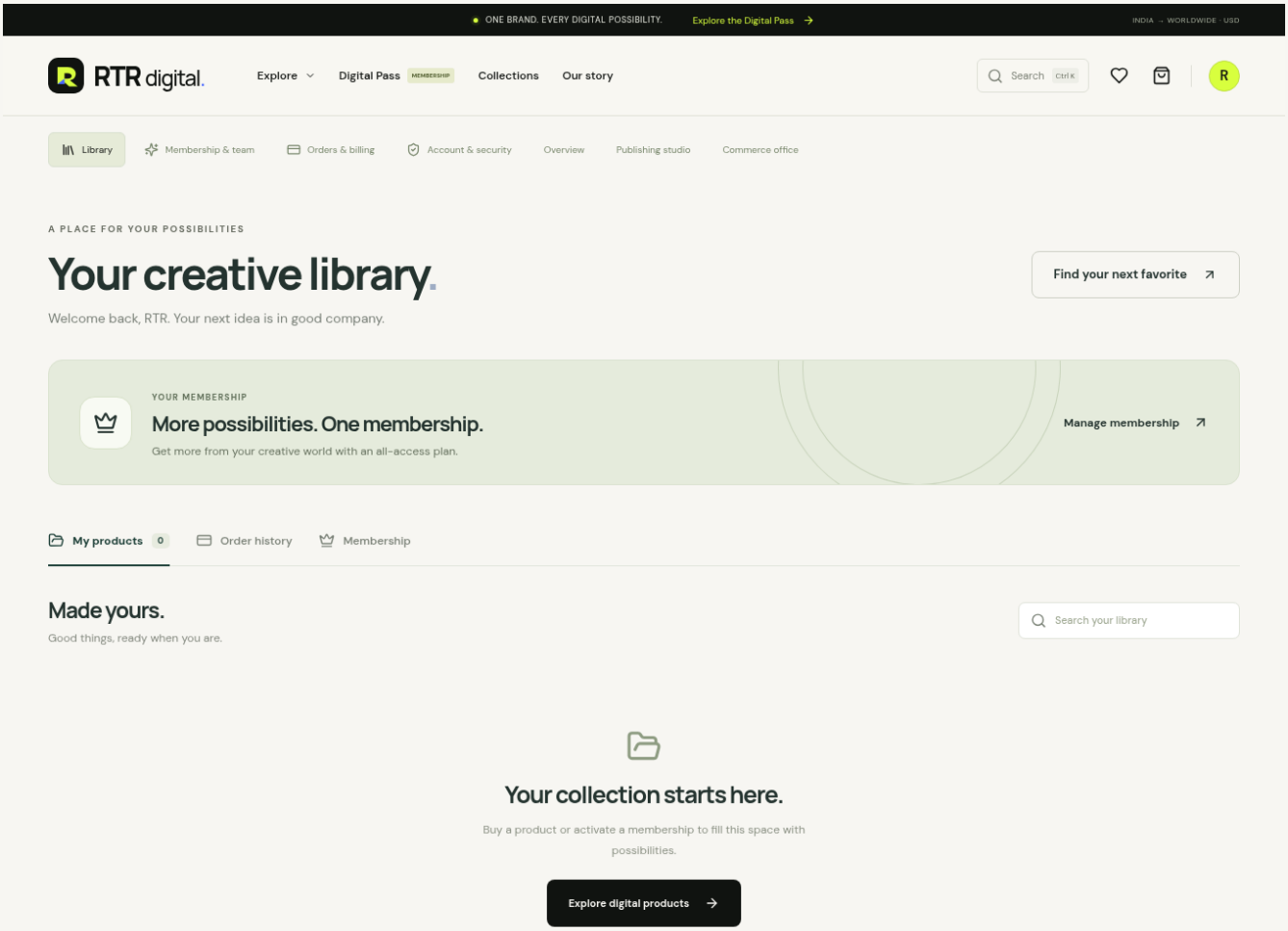


Account and security controls. Shared demonstration accounts are for orientation; configure your own production administrator.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

05.3 / Library



My library in a demonstration account with no individual purchases. Actual purchases and active membership rights determine customer access.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 06

06 Payments, bank accounts and finance

This chapter explains the payment and finance controls implemented in this copy of RTR Digital. The business is based in India, the storefront charges in USD, and RTR Digital is the seller and licensor. You must supply your own approved merchant accounts, actual receiving bank instructions, business details, and private credentials. The project does not include a merchant account, a connected real bank, or permission to charge anyone.

The website has four payment routes. A payment provider may offer several cards, wallets, or local methods within its own checkout, according to its approval and settings. That does not mean the website has a separate integration with every payment service.

Route	What the customer uses	Membership behavior	What permits access
Stripe	Stripe hosted checkout and methods eligible for your account, buyer, and currency	Automatic monthly or yearly renewal	A verified, signed Stripe event matching the store's recorded checkout
PayPal	PayPal approval followed by server capture and verification	One prepaid monthly or yearly term; no automatic renewal	The exact completed USD capture, original order identity, and expected merchant recipient
Razorpay	A Razorpay hosted payment link	One prepaid monthly or yearly term; no automatic renewal	The exact captured USD payment linked to the original store reference
Bank transfer	Your published wire/SWIFT receiving instructions	One prepaid monthly or yearly term; no automatic renewal	An administrator confirms exact cleared USD funds in the original receiving account

All examples below are illustrative placeholders. A value such as `<YOUR_PAYPAL_CLIENT_ID>` must be replaced privately with the value from your account. Example amounts explain a procedure; they do not authorize a transaction. External provider menus can change. These instructions describe the application's source and visible labels; no live provider dashboard, merchant approval, bank settlement, or real charge was verified while preparing this manual.

Prepare the business before enabling payment

Decide which accounts and bank routes you will use

1. Identify the legal person or entity operating RTR Digital. Gather the registered address, business contact, relevant tax identifiers, bank account ownership evidence, and documents required by each provider. A trading name and a legal bank beneficiary can differ; use the actual registered information requested by the provider and bank.
2. Decide which of the four routes you intend to offer initially. Enabling every route creates more accounts and reconciliation work. You can launch with the routes for which your business has completed approval and operating tests.
3. Apply through each provider's official website for an account eligible for your India-based business and digital products. Complete its identity, business, website, product, and bank verification steps. Request any separate approval required for international payments, USD transactions, or the payment-link product you intend to use.
4. Ask the provider how your USD sales settle: the settlement currency, conversion rate basis, fees, payout delay, reserve or hold rules, refunds, disputes, and export documentation. A price shown in USD on RTR Digital does not promise that an Indian bank receives a USD balance.
5. Connect and verify your actual payout bank inside that provider's merchant dashboard. Follow the provider's verification process. Do not enter an online-banking password, card PIN, or one-time banking code into RTR Digital's payment settings.
6. Separately ask your bank for written inward international transfer instructions for the receiving account you will publish for direct transfers. Confirm that the account is permitted to receive the intended digital-product receipts and ask how you can verify the exact cleared USD amount.
7. If your bank automatically converts receipts or deducts fees in a way that prevents you from establishing the exact received USD amount required by the store, resolve the receiving arrangement before offering wire checkout. The store has no partial-payment or alternative-currency approval override.
8. Record private support contacts for the provider and bank, the account owner, and the person responsible for daily reconciliation. Keep a secure record of which merchant account and bank belong to each route. Do not put secret keys or full bank details into public product descriptions or this manual.

Understand the two different bank settings

1. For **Stripe, PayPal, or Razorpay payouts**, open the respective merchant website. Add or change the payout/settlement bank there. A change in RTR Digital's bank-transfer form does not change any provider's payout bank.
2. For **direct customer wire transfers**, open RTR Digital's **Commerce office → Payment connections** and edit **Your receiving account.** These instructions tell a customer where to send money; the website does not log into the bank, initiate a debit, monitor a bank API, or withdraw funds.
3. When changing a provider payout bank, complete the provider's verification and inspect its payout status in that external dashboard. The buttons **Open Stripe dashboard**, **Open PayPal dashboard**, and **Open Razorpay dashboard** open external websites; they do not complete verification automatically.

4. When changing wire instructions, plan for customers who already received the old details. Old pending orders retain their own saved bank version. Keep the old account available and monitored while those orders and possible late transfers are resolved.

Open and understand Payment connections

Reach the payment controls

1. Sign in to your private administrator account. Use your own production administrator for a live store. The shared development account is not an operating identity for a real business.
2. Open your account menu and choose **Commerce office**. The administrator workspace navigation also contains **Commerce office**. Its application address is `/operations`; the visible navigation label is **Commerce office**.
3. In the Commerce office tabs, choose **Payment connections**. The neighboring tabs are **Overview**, **Promotions**, **Refund requests**, **Seller details**, and **Audit trail**.
4. Read the provider cards. **Credentials configured** means the required saved values exist, or the server has the required Stripe configuration. It does not prove external account approval, a successful connection test, or a verified payout bank. **Setup required** means configuration is incomplete.
5. Check the environment badge. PayPal has an explicit **Sandbox / Live** choice. Razorpay follows the `rzp_test_` or `rzp_live_` Key ID. Stripe is **Server configuration**, managed by the person deploying the server.
6. Click **Refresh connections** after another administrator or your deployer changes settings. Refreshing reloads saved values and hides revealed bank details. Save or record unfinished edits before refreshing because the form is rebuilt from the saved settings.
7. Scroll to **Review. Verify. Save.** The field **Current administrator password** verifies sensitive changes. Enter your current RTR Digital administrator password, not a provider password. Click **Save payment settings** only after reviewing the whole form.
8. If the page displays **Shared development demo**, read its exception carefully: password verification is bypassed for that shared development account. Production disables this shortcut. A simulated cleared wire recorded by that account does not grant live purchase access.

Preserve, replace, disable, or remove saved credentials

1. To keep saved PayPal or Razorpay credentials while changing another setting, leave their credential inputs blank. The placeholders **Saved · leave blank to keep** and **Saved securely · leave blank to keep** indicate that values are already held on the server.
2. To replace a credential, paste the complete replacement into the corresponding blank input. A masked ending such as `••••1234` is a display hint, not a credential to retype. Saved provider secrets cannot be revealed through this page.
3. To temporarily stop new PayPal checkouts, turn off **Offer PayPal**, authenticate, and save. To stop new Razorpay checkouts, turn off **Offer Razorpay**, authenticate, and save. This preserves credentials for future use and does not repay or cancel existing orders.
4. To remove PayPal's current saved credential set, check **Remove saved PayPal credentials when I save**, enter your current administrator password, and save. The control removes Client ID, Client

secret, and Merchant ID and disables the route. The corresponding Razorpay control removes Key ID and Key secret and disables that route.

5. Do not select a removal checkbox during an ordinary replacement. Removal explicitly takes precedence for those fields. After a successful save, the form reloads; check the switches, badge, and placeholders again.

6. Keep old provider accounts and credentials available while their outstanding payments, refunds, and disputes remain unresolved. Existing PayPal and Razorpay orders retain encrypted original provider settings. Changing the current form does not silently move an old order to a different merchant recipient.

Set up Stripe with your deployer

Stripe credentials are not editable in Payment connections. A novice owner should prepare the account and give the following concrete configuration requirements to the person managing deployment. Send secrets through the deployment platform's private secret manager or another agreed secure channel.

Configure the account, keys, and webhook

1. Sign in to the official Stripe dashboard for your own business. Complete its onboarding and confirm current India account availability and approval for your intended international digital sales. The website cannot bypass Stripe's eligibility decisions.

2. Work in Stripe test mode first on an isolated test deployment. Obtain the secret API key for that mode. The server uses `STRIPE_SECRET_KEY`. This application does not ask you to paste a Stripe publishable key into Payment connections. Keep test credentials out of the real store: Stripe configuration recognition does not validate the key's live/test mode, and the application's recorded `stripe` route is included in its paid-route reporting. A test key must not be used to produce supposed real sales records.

3. Establish the final public HTTPS origin, for example `https://shop.your-domain.example`. Set `PUBLIC_URL` to that origin. Use your real domain for deployment; the example domain is a placeholder. Do not include login credentials, a query string, a fragment, or a subdirectory path.

4. Create a Stripe webhook destination pointing to the real deployed endpoint `https://YOUR-REAL-DOMAIN/api/stripe/webhook`. Replace the domain. Do not register `/billing`, a dashboard URL, or the browser checkout success page as the webhook destination.

5. Subscribe that destination to the implemented checkout, subscription, billing, and refund events listed below. If your provider interface groups events, verify the actual event names included in the subscription.

6. Copy that destination's signing secret into `STRIPE_WEBHOOK_SECRET`. A webhook signing secret belongs to a particular destination and mode. A test destination's signing secret is not a substitute for the live destination's secret.

7. Have your deployer set the three values in the server runtime secret/environment settings and restart or redeploy the server. Creating a local `.env` file alone does not configure a hosted deployment; the supplied application also does not automatically load `.env` without the documented runtime option.

8. Return to **Commerce office → Payment connections**, click **Refresh connections**, and inspect the Stripe card. Configuration recognition is a first check. Complete the end-to-end test procedure later in this chapter before considering the route operational.

9. In Stripe, enable only payment methods your business is eligible to accept and that work for the buyer and USD checkout. Do not advertise a particular wallet or INR-only local method merely because Stripe supports it in some other country or currency.
10. When switching to live service, repeat the mode-specific key and webhook setup with your approved live account and final HTTPS domain. Retest with your own authorized low-value purchase under your operating policy. This manual does not perform that purchase.

Event name	Purpose in this application
`checkout.session.completed`	Fulfill a matching paid checkout
`checkout.session.async_payment_succeeded`	Fulfill a matching delayed payment after success
`checkout.session.expired`	Release checkout-related reservations and membership locks
`customer.subscription.updated`	Reconcile authoritative subscription status, period, and mapped plan changes
`customer.subscription.deleted`	Reconcile an ended subscription
`invoice.paid` and `invoice.payment_succeeded`	Reconcile successful membership billing/renewal
`invoice.payment_failed`	Reconcile billing that needs attention
`invoice.payment_action_required`	Reconcile billing requiring customer action
`charge.refunded` and `refund.updated`	Reconcile confirmed full refunds

The server verifies the Stripe signature, known checkout intent, expected amount, USD currency, and required subscription state. A buyer returning to `?checkout=success` does not itself grant access. Duplicate signed deliveries are recorded and handled without creating another purchase grant.

Configure recurring plan changes, the portal, and optional tax

1. Decide whether customers will use Stripe's customer billing portal for provider-managed membership changes. Configure the portal in your Stripe account, including the actions and membership prices you intend to permit. A working key alone does not create that portal configuration.
2. Create or identify the correct USD recurring prices in the same Stripe account and mode. Match each price to the actual application plan and monthly/yearly interval. The display name **Explorer** has the internal name `starter`.
3. Give your deployer the matching Price IDs for `STRIPE_PRICE_STARTER_MONTHLY`, `STRIPE_PRICE_STARTER_YEARLY`, `STRIPE_PRICE_CREATOR_MONTHLY`, `STRIPE_PRICE_CREATOR_YEARLY`, `STRIPE_PRICE_STUDIO_MONTHLY`, and `STRIPE_PRICE_STUDIO_YEARLY`. A Price ID resembles `price_...`; it is not a product title or an API secret.
4. Recheck every mapping against plan, amount, currency, interval, account, and mode. These mappings let authoritative provider plan changes map back to application rights. Initial website checkout builds

its own trusted price details; these mappings are specifically important when provider-managed price changes are offered.

5. Test an actual allowed plan change in the test portal and inspect the resulting application membership. An unrecognized provider price requires administrator mapping and review; do not promise automatic Studio/team privileges for an unmapped price.

6. Decide tax handling with your tax adviser. The default `STRIPE_AUTOMATIC_TAX=false` does not configure tax collection. If appropriate registrations and Stripe Tax configuration are complete, your deployer can set `STRIPE_AUTOMATIC_TAX=true` and restart. Stripe then calculates applicable tax at hosted checkout and billing-address collection is requested.

7. Test the final tax-inclusive hosted total and recorded receipt. A storefront estimate is not a tax determination. Enabling Stripe Automatic Tax does not configure equivalent tax handling for PayPal, Razorpay, or direct wires and does not turn the site's payment receipt into a GST/export tax invoice.

Connect PayPal

Obtain and enter a matching PayPal credential set

1. Open the official PayPal developer site using the business account that should receive RTR Digital's money. Create or identify its REST application. For a first test, use a sandbox business merchant and separate sandbox buyer.

2. Obtain the application's **Client ID** and **Client secret** for the intended environment. Also obtain the **Merchant ID** for that same business recipient. Do not use a buyer account's identity, an email address in place of a merchant ID, or credentials copied from another business.

3. In RTR Digital, open **Commerce office → Payment connections → PayPal**. Select **Sandbox** for sandbox testing or **Live** for the approved live business. The environment and all credentials must agree.

4. Enter **Client ID** as `<YOUR_MATCHING_PAYPAL_CLIENT_ID>`, **Client secret** as `<YOUR_MATCHING_PAYPAL_CLIENT_SECRET>`, and **Merchant ID** as `<YOUR_MATCHING_PAYPAL_MERCHANT_ID>`, replacing these placeholders privately. There are no working values supplied with the project.

5. Turn on **Offer PayPal**. Leave the removal checkbox unselected. Enter **Current administrator password** in the save bar and click **Save payment settings**.

6. Confirm that the saved page shows the intended environment and **Credentials configured**. Then test a purchase in that environment. The store creates its USD order, obtains buyer approval, and captures/verifies server-side before access.

7. During your test, check that the receiving merchant in PayPal is the business you intended. The implementation checks the recorded recipient Merchant ID as well as the exact USD amount, order reference, custom ID, invoice ID, and final completed capture.

8. Before production, enter the complete live credential set and select **Live**. Production refuses sandbox payments as production entitlements. A development payment does not become a live order when you switch the dropdown.

Change the PayPal account or payout bank

1. For a payout-bank change within the same merchant account, open **Open PayPal dashboard** and complete PayPal's bank-change process. Do not change Client ID or Merchant ID solely because the payout bank changed.
2. For a different receiving PayPal business account, obtain its Client ID, Client secret, and Merchant ID together. Review outstanding payments and refunds belonging to the old merchant before switching.
3. Replace all three application values, choose their matching environment, authenticate, and save. Recheck new checkout using the new merchant. Old pending orders continue to use their encrypted original merchant settings.
4. Avoid revoking old application credentials until outstanding original-account verification and refunds are resolved. Revocation at PayPal can make those operations require manual provider support, even though the website preserved the original settings.

Connect Razorpay

Configure test or live keys

1. Open the official Razorpay merchant dashboard for your business. Complete activation and the required bank and business checks. Ask specifically about eligibility for international USD payments through the hosted payment-link route used here.
2. Generate or locate a matching **Key ID** and **Key secret**. Test Key IDs begin with `rzp_test_`; live Key IDs begin with `rzp_live_`. Keep the complete generated secret securely when the provider makes it available.
3. In **Commerce office → Payment connections → Razorpay**, enter the complete **Key ID** and **Key secret**. For illustration, the Key ID field's format is `rzp_test_...` or `rzp_live_...`; the ellipsis is not a usable value.
4. Turn on **Offer Razorpay**, leave **Remove saved Razorpay credentials when I save** unselected, enter **Current administrator password**, and click **Save payment settings**.
5. Check the **Test / sandbox** or **Live** badge. There is no separate Razorpay environment dropdown; the saved Key ID determines the mode. A valid-looking live key does not prove approval for USD or international transactions.
6. In your test deployment, complete a hosted payment-link test and return to the store. The server retrieves the payment link and captured payment and verifies the trusted reference, USD currency, full amount, and captured state. A customer-provided transaction number or unsigned callback alone cannot unlock access.
7. Enter your approved live key pair before production. Production requires the live prefix and does not accept test payments as live entitlements. Verify the actual merchant account, eligible methods, fees, and settlement currency in Razorpay.
8. If checkout is configured but USD payment-link creation fails, inspect the merchant account's international/currency/product permissions and provider error. Repeatedly replacing a correct key cannot grant missing merchant approval.

Change the account or settlement bank

1. Change a Razorpay settlement bank in **Open Razorpay dashboard**, following its verification process. The website's direct-wire bank fields do not affect Razorpay settlements.
2. If replacing the receiving merchant account, obtain and replace both Key ID and Key secret from the new account. Authenticate and save, then test new checkout against that account.
3. Retain access to the old account while its payments and refunds remain open. Existing payment snapshots retain their original encrypted key pair for verification and refund processing. Provider-side revocation can still require manual reconciliation.

Publish and change direct bank instructions

Prepare each bank field from your bank's instructions

Use the bank's actual written international receiving instructions. Do not derive a SWIFT code from a bank name, invent an IBAN for an Indian account, or substitute a domestic routing code for a correspondent-bank requirement.

Exact form label	What to enter
Beneficiary / account holder	The beneficiary name recorded by the bank; confirm any difference from the RTR Digital trading name
Bank name	The receiving bank's name supplied in its instructions
Account number	The actual receiving account number, including significant leading zeros
SWIFT / BIC	The exact international code required by the bank
IFSC	The supplied Indian branch code when applicable
IBAN (if applicable)	Only an actual IBAN issued for this receiving arrangement
Routing number (if applicable)	Only the routing number required for this transfer route
Intermediary / correspondent bank	Any required bank name, SWIFT, account, or routing details, copied accurately
Bank address	The bank/branch address requested in the receiving instructions
Beneficiary address	The registered beneficiary address associated with the account
Additional transfer instructions	Bank-approved reference, charges, or route instructions, including information the sender must supply

1. Compare the bank's instructions with your business records before opening the edit form. Confirm the beneficiary, currency route, account, branch, and correspondent details with the bank through a known contact.
2. Open **Commerce office → Payment connections** and scroll to **The details that bring funds home. / Your receiving account.** Enter the applicable fields above. The website requires

beneficiary, bank name, and account number or IBAN before bank transfer can be enabled, but that minimum is not proof that an international wire can reach the account.

3. Leave inapplicable fields empty. Do not insert invented examples to satisfy appearance. If your bank requires an intermediary, include it; an otherwise complete receiving account may still be insufficient without those instructions.

4. Review **Additional transfer instructions** for the sender's reference and charges obligations. Explain any bank-approved requirement needed for the exact USD amount to arrive. The customer-facing payment already provides its own unique store reference and amount.

5. Turn on **Offer bank transfer**, enter **Current administrator password**, and click **Save payment settings**. Check the **Instructions version** displayed after saving.

6. Create a pending wire order in your isolated test environment and inspect the customer view. Check every displayed detail against your source document. This tests publication and order snapshots; it does not verify an actual bank connection.

7. Before accepting real wires, have the bank independently confirm the published route and establish how you will reconcile a real cleared receipt. The website cannot validate account ownership or settlement merely from these text fields.

Reveal or update saved account numbers

1. To inspect the current saved account, enter **Current administrator password** in the save bar, then click **Reveal saved bank details**. The saved account number, IBAN, and routing number appear in **Saved receiving account** where present.

2. Compare the revealed details privately. Click **Hide saved bank details** when finished. They also hide automatically after **60 seconds**, when the page reloads its data, or when the administrator identity changes. Revealing is recorded in the audit trail.

3. To retain the current account number, IBAN, or routing number during an ordinary edit, leave that sensitive input blank. To replace it, enter the complete replacement. The revealed details do not automatically become editable values.

4. Edit ordinary text fields such as Bank name or SWIFT directly. Those fields save their current visible text; clearing one removes that text. Review all fields when changing the receiving bank so that a new account does not retain an old branch or intermediary description.

5. If moving to a route that must remove previously saved sensitive fields, use **Remove saved receiving account and disable bank checkout when I save** first, authenticate, and save. Then enter the complete correct new account set, re-enable **Offer bank transfer**, authenticate, and save again. Blank sensitive inputs preserve old values, so blanking an obsolete IBAN alone does not remove it.

6. To pause new wires while retaining the current account, turn off **Offer bank transfer** and save without selecting removal. Existing pending orders still retain their original instructions. Disabling checkout is not a notice to a customer who has already copied bank details.

Change banks with pending orders

1. Review the bank reconciliation queue before changing details. Record outstanding payment IDs, original receiving account endings, instruction versions, deadlines, and expected amounts in your private operational records.

2. Confirm when the old bank account will stop receiving funds. Plan to monitor it throughout outstanding orders' seven-day instruction window and resolve possible late arrivals before closure.
3. Save the replacement bank details as described above. Each save publishes a version for future orders. A pending order keeps the snapshot issued when it was created; the site does not silently replace its beneficiary or redirect its payment.
4. Check one old pending order and one new test order. The old queue row should retain its **Original receiving account** and version; new instructions should show the replacement details.
5. Reconcile old-account receipts against their old orders. Do not treat receipt in a different account as proof of the original instructions without investigating. If a customer needs revised instructions or has sent late funds, handle the case directly through your support procedure and bank.
6. Keep historical snapshots and financial records intact. Do not edit the database to rewrite a past order's bank details, amount, or seller merely to make it resemble the new configuration.

Understand the customer payment flow

Guide a customer through checkout and payment activity

1. The customer signs in, chooses products or a membership, and opens checkout. Memberships are purchased separately from individual products. The quote and USD total are calculated by the server.
2. If using a promotion, the customer enters **Promo code**, clicks **Apply**, and checks the new **Total**. For a Stripe membership, the discount display says **each renewal** when applicable.
3. The customer selects **Pay securely with Stripe**, **Pay with PayPal**, **Pay with Razorpay**, or **Pay by bank transfer**. An unavailable route shows **Unavailable** or setup guidance. **Explore with demo checkout** appears only when enabled for development and makes no real charge.
4. The customer reads and checks the digital-content license and terms declaration. For Stripe memberships, it explains renewal until cancellation. For other membership routes, it explains a prepaid fixed term with no automatic renewal and access after verification.
5. For Stripe the action is **Continue to secure payment**; PayPal and Razorpay use **Continue to PayPal/Razorpay**. For a wire the customer chooses **Get bank transfer instructions**. Do not instruct a customer to send money before the store has created the pending order and issued its reference.
6. On return, the customer opens **Orders & billing** at `/billing`. Completed orders and receipts appear above **Payment activity**; pending PayPal, Razorpay, and bank orders appear in Payment activity. Stripe fulfillment depends on its signed webhook flow.
7. The customer opens the relevant payment row and checks **Payment reference**, **Payment ID**, total, purchase, and **Payment expires**. Returning from a provider does not prove payment. **Refresh provider payment** asks the server to verify that provider's actual record.
8. For a wire, the customer copies the private **Bank transfer instructions**, sends the exact displayed USD amount with the payment reference, and accounts for bank or intermediary fees. Instructions are disclosed to the signed-in order owner, not as a public catalog of your bank account.
9. After sending a wire, the customer may choose **Report a bank transfer**, enter **Bank transfer reference**, add a **Transfer note (optional)**, and click **Submit transfer details**. The reference must be at least three characters. It is a customer claim, not bank confirmation.

10. The message remains **Transfer details reported. Your payment remains pending until cleared funds are verified. Refresh transfer status** reads the store's current record. Neither reporting nor refreshing tells your bank to transfer funds or authorizes access.
11. Once verified, the payment shows **Paid** and **Payment confirmed**, with the paid order and **Open your library** when appropriate. A paid membership that conflicts with another active paid term can require review instead of automatically replacing the active membership.
12. If the status is **Expired**, **Rejected**, **Payment setup failed**, or **Cancelled** after the customer sent funds, ask for the payment reference and investigate. Do not ask them to pay again solely because the browser return failed.

Reconcile bank transfers

Confirm an exact cleared USD payment

1. Open **Commerce office → Payment connections** and scroll to **Clear funds. Then grant access.** Click **Refresh connections** to reload the queue before beginning.
2. Find the row using the store payment/order reference and the customer's identity. Read **USD expected**, **Order reference**, **Original receiving account**, its version, and **Transfer deadline**. A status of **Customer reported payment** is not confirmation.
3. Sign in to your bank through its official website or approved banking channel. Locate the actual cleared receipt. Match the intended beneficiary and original account, bank transaction reference, customer/remitter information where available, and store reference.
4. Establish the exact amount received in USD. For an expected `25.00 USD`, a pending remittance, a screenshot for `25.00`, an INR credit without a verified corresponding USD receipt, or `24.00 USD` after fees does not satisfy the store's confirmation rule.
5. Check that the bank statement reference has not been used for another order. Each incoming bank receipt can fulfill only one order; do not split one bank reference across separate order confirmations.
6. Click **Review transfer**. For an unexpired order the form opens **Confirm a cleared payment**. Enter **Cleared amount received (USD)** using the exact amount established in the bank, and **Unique bank statement reference** using the real statement entry.
7. Enter **Current administrator password**. Check **I checked RTR Digital's bank statement and verified that this exact USD amount has cleared into the original receiving account.** Check it only after doing that work.
8. Click **Confirm cleared payment & grant access**. The shared development account instead shows **Record demo cleared payment** and records a simulation. Do not mistake a demonstration toast for a real settled purchase.
9. Wait for success, refresh the relevant records, and confirm the order/access result. The server records settlement identity and prevents that receipt from fulfilling another order. A retry of an already confirmed matching payment does not create another purchase.
10. If an error appears, preserve the bank evidence and payment reference. Correct an input error if identified. Investigate any already-used reference or conflicting status before retrying; never create a different reference to bypass duplicate protection.

Reject, expire, or investigate a transfer

1. For an unexpired row that cannot be accepted, click **Review transfer**, then **Reject transfer instead**. Read **Reject this transfer** and **This records a rejection and keeps the order unpaid.**
2. Enter **Reason / note (optional)** with a useful factual explanation, enter **Current administrator password**, and click **Reject transfer**. Use **Cancel** or **Close review** if more investigation is needed. Rejection does not send money back.
3. For **Expired · manual review**, read the deadline and late-funds warning. The review opens a rejection path; that expired order cannot automatically grant access. Standard wire instructions expire after seven days. PayPal and Razorpay orders have a shorter, approximately 30-minute payment window.
4. If funds arrive late, inspect the bank/provider record and contact the customer through your normal support channel. Agree the legitimate resolution, such as returning the money through the original bank/provider or a separately documented supported purchase. Do not overwrite expiry or historic amounts in the database.
5. For short payments, other currencies, wrong references, unmatched recipients, or duplicate receipts, retain the payment as unresolved while investigating. The form deliberately has no partial/other-currency grant control.
6. If a second membership payment is recorded while another paid membership is active, inspect both orders and current entitlements. The application preserves the paid order for review rather than silently replacing the active term. Resolve or refund the conflicting payment without charging again as a diagnostic step.

Create and manage promotions

Create a promotion with deliberate limits

1. Open **Commerce office → Promotions**. Review the preloaded **WELCOME10** and **RTR20** codes. They are enabled seeded examples with 10% and 20% discounts, broad scope, no expiry, and no usage cap unless your installation has changed them. Review or **Pause** them before announcing live prices.
2. Click **Create promotion**. Enter **Promotion code**, using 3–40 letters, numbers, underscores, or hyphens. For illustration, `LAUNCH15` is a proposed code; check that it does not already exist. The server stores codes uppercase.
3. Enter **Discount (%)**, for example `15`, within the positive percentage range up to 100. A 100% code can produce a zero total that some routes do not support; test your chosen route rather than advertising a universal free checkout.
4. Choose **Applies to**: **Products & memberships**, **One-time products**, or **Memberships**. Set **Minimum subtotal (USD)**, for example `20.00`, or leave the displayed `0` if there is no minimum.
5. Set **Starts at (your local time)** and **Ends at (your local time)**. The browser converts these local selections into stored timestamps. Confirm the computer's timezone and use a clear published timezone in campaign communications. Leave dates blank only if immediate start/no expiry is intentional.
6. Set **Maximum redemptions**, for example `100`, and **Redemptions per account**, for example `1`. These must be positive whole numbers; blank means no corresponding limit. A code can pass its date range yet fail its account or global cap.

7. Review the checkbox **Allow on memberships. The discounted price applies to every renewal.** It is selected initially. For a Stripe membership, allowing the code establishes a discounted recurring price, not just an introductory first payment.
8. If you want one-time product use only, choose that scope and clear membership allowance. A membership purchase with membership allowance disabled is rejected even if another scope selection seems broad.
9. Click **Save promotion**. Review its row: code, minimum, percentage, scope, **Recurring eligible / One-time only**, date range, redemptions, account limit, and enabled state.
10. Test the promotion with an eligible checkout and an ineligible one. Check the exact rounded USD total shown before payment. Do not infer the charged total by subtracting a percentage from a receipt's reference item prices later.

Pause codes and understand reservations

1. Click **Pause** beside a code to stop future eligible use. Click **Enable** to restore it; original date and usage limits still apply. The supplied table offers pause/enable, not an edit form for rewriting the original code's terms.
2. To change an offer's percentage, dates, or limits, create a new clearly named code and pause the old one according to your campaign policy. Do not rewrite old order totals. Existing Stripe membership contracts retain their established discounted recurring price unless changed through legitimate billing operations.
3. Understand that the displayed **Redemptions** count reflects completed order usage. The server also reserves limited uses for concurrent pending checkouts, preventing several buyers from taking the final available redemption simultaneously.
4. For a pending wire, the reservation can last through its seven-day payment window. A code can temporarily report its limit reached even when the completed-use count appears lower. Examine pending payments before assuming the counter is broken.
5. Completion records the redemption. Rejection, expiry, or failed creation releases the relevant pending reservation. Do not delete legitimate pending orders or financial records to make a promotion available again.

Handle refunds carefully

Refund requests are a review workflow. Submitting a request does not promise approval, transfer money, or immediately revoke access. The supplied controls handle full-order refunds; there is no partial-refund amount editor for provider approvals.

Understand the customer's request

1. The customer opens **Orders & billing**, locates the order, and chooses **Request a refund**. They enter **Reason for refund** with 10–2,000 characters and select **Submit refund request**. **Cancel request** closes the unsubmitted form.
2. The customer sees the saved request status and later administrator explanation with the order. Once a request exists, another request form is not offered for that same order.

3. As administrator, open **Commerce office → Refund requests**. Choose **All requests**, **Open requests**, **Approved**, or **Declined**. Match order ID, customer, amount, route, reason, and any existing decision note.

4. Confirm the original paid transaction and whether a refund has already happened externally. Check your policy and the relevant provider/bank record before choosing an approval action.

Approve or decline a Stripe, PayPal, or Razorpay refund

1. On a new request, click **Review refund**. Read **Confirm this refund?** and the provider-specific explanation. A real provider approval asks the original provider to refund the original payment; a pure demo order reverses only simulated records/access.

2. Click **Confirm refund** once you have completed the review. This is a financial action for a real paid order. The provider amount is the original full payment, not an amount typed by the administrator.

3. Wait for the result and refresh records. **Approved** means the workflow has confirmed the necessary refund result and applied its access reversal. A pending or uncertain response must not be treated as completed repayment.

4. For **processing**, inspect the provider's refund state. The application retains access while repayment is pending or unconfirmed. Use **Retry reconciliation** to retrieve or reconcile the original refund after checking the provider; do not create an unrelated second refund to make the page change.

5. For a Stripe membership refund, check subscription cancellation as well as repayment. **provider refunded pending cancel** means repayment was issued but cancellation still requires reconciliation. Use **Retry reconciliation** after investigating; the refund must not be issued again just because cancellation failed.

6. PayPal refund verification checks the original captured payment, expected full USD amount, and completed refund. Razorpay checks the original payment, full amount, currency, and processed refund. The original encrypted provider settings remain relevant after current account changes.

7. If a Razorpay request times out or its response is lost, open the original merchant dashboard and inspect that payment's refunds before any further action. The application records an attempt and searches for its stable receipt. When uncertain it reports that the prior attempt awaits reconciliation and **will not be submitted twice**.

8. Keep the Razorpay guard intact. Do not delete the attempt record, invent a new order, or submit a separate manual provider refund as a blind retry. Escalate an unresolved provider state with the original payment/refund references through your support process.

9. To decline a new request, choose **Decline with explanation**, enter **Decision explanation** in 5–1,000 characters, and click **Record decision**. The explanation is visible to the customer. Only a new requested refund can be declined; an already processing repayment is not something this decision control cancels.

10. Use **Keep reviewing** to close the form without a decision. After completion, confirm the relevant order status and access. Refunding one order does not invalidate another independent purchase of the same product.

Return a wire payment through the bank, then record it

1. Review the paid wire order and refund request. Verify the full original USD amount and the legitimate repayment destination with the customer through your established support process. Investigate any request to repay a different party or bank detail before proceeding.
2. Send the actual refund through your bank's official channel. Follow the bank's instructions and retain its transaction evidence. RTR Digital does not initiate this transfer and its approval button cannot send the money.
3. Wait until the bank refund is completed and verified. Establish the exact full original USD amount returned. A proposed transfer, saved beneficiary, pending bank instruction, or partial refund is insufficient for the site's declaration.
4. Return to **Commerce office → Refund requests**, find that order, and click **Review refund**. Read the notice beginning **First return the full amount through your bank.**
5. Enter **Amount already refunded (USD)** with the exact original full positive amount, for example `25.00` for an original `25.00 USD` order. Enter **Bank refund transaction reference** with the actual completed repayment's unique bank reference.
6. Use the full real reference. Server validation requires 5–120 characters beginning with a letter or number, with letters, numbers, spaces, periods, slashes, underscores, or hyphens. A reference already recorded for another wire refund is rejected.
7. Enter **Current administrator password**. Check **I already returned the full [displayed amount] USD to the customer through the bank and verified the completed refund.** Do not check this for a transfer that is still pending.
8. Click **Confirm refund**. The site records the completed bank repayment, audit information, and related access reversal. It does not send another bank payment. Verify the approved request and refunded order afterward.
9. If recording fails after the bank already paid, retain the completed repayment evidence and correct or investigate the record. Do not send the money a second time. A shared development demo bypasses its labeled password check but is not a model for live bank operations.

Seller details, receipts, reporting, and records

Keep accurate seller information

1. Open **Commerce office → Seller details**. **Seller & licensor** is fixed as RTR Digital. Enter your actual **Registered address**, applicable **Tax registration ID / GSTIN**, and working **Support email**.
2. Review **Receipt prefix**, initially `RTR`. This is a payment-receipt reference prefix, not an automatic government tax invoice numbering system. Click **Save seller details**.
3. Confirm the resulting details on a new test receipt. Historical paid receipts retain captured seller and financial information. Changing the current address, tax ID, prefix, or prices does not rewrite earlier financial snapshots.
4. If a historic receipt needs a legal correction, follow your accounting process for a separate correction or statutory document. Do not edit the database to overwrite the original transaction or assume changing Seller details corrects the past receipt.

5. Arrange applicable GST, export, place-of-supply, VAT, or sales-tax documentation with your adviser or configured tax service. The site's **payment receipt is not a statutory tax invoice**. India-based operation and USD pricing alone do not determine tax treatment.

Read receipts and finance figures correctly

1. Customers open **Open payment receipt** on an order in **Orders & billing**. The receipt opens in a new browser tab and can be printed through the browser. Access is restricted to the owner or authorized administrator.
2. Check the order identifier, seller snapshot, issue date, status, mode, recorded total, and any displayed promotion/tax breakdown. Item rows may show reference prices before promotion; the immutable order total is the authoritative payment total.
3. Recognize **DEMONSTRATION — no money was charged** and **Demo · no money collected**. Sandbox/development PayPal, Razorpay, and wire completions are recorded as demo activity rather than live revenue. Do not include them in real sales books as cash receipts.
4. Open **Commerce office → Overview**. **Live gross receipts** sums recorded paid-route orders; **Confirmed refunded revenue** reflects confirmed full refunds; **Net live revenue** subtracts the latter from the former.
5. Treat these as store records, not your available bank balance, profit, or settlement statement. Provider fees, currency conversion, reserves, payout timing, and bank charges need reconciliation using the provider/bank statements.
6. Read **Live monthly recurring value** as recorded recurring Stripe contract value normalized monthly. A yearly contract is divided by 12; the annualized display multiplies monthly value by 12. It is not cash collected this month or a guaranteed forecast.
7. Exclude prepaid PayPal, Razorpay, and wire membership terms from recurring MRR. They can appear among active live memberships while having no automatic future charge. Demo memberships and no-card trials are separate from live recurring contract value.
8. Read **Open refund requests** as requests requiring attention, including processing and pending cancellation states. Do not reduce your reconciled cash for an unconfirmed refund merely because the customer submitted a request.

Export and close the day's records

1. Click **Refresh records** in Commerce office. On **Overview**, choose **Export order records · CSV**. The export contains **Order**, **Date**, **Mode**, **Status**, **Customer ID**, and **Total USD**.
2. Save the file in a private accounting location with a dated filename. Filter mode and status before analysis; an order export includes record history and is not a provider payout report.
3. Open **Audit trail**. The screen shows the latest 150 administrator/account activity records. Select **Export audit · CSV** to obtain the available full audit export with **Date**, **Action**, and **Details**.
4. Match paid orders and approved refunds to provider transaction reports and actual bank entries. Document fees, exchange differences, held payouts, late transfers, pending refunds, and unresolved cases separately rather than altering order totals.
5. Record who investigated each discrepancy and the next action. Reference store IDs and provider/bank references without copying passwords or secret credentials into a shared spreadsheet.

6. Archive required receipts and external statements under your retention policy. The site's audit trail records application activity; it cannot prove that an external bank action occurred unless you obtain the bank's evidence.

Preserve keys and verify operation before launch

Back up settings and original pending-payment snapshots

1. Have your deployer back up the complete private data directory consistently with the database and its relevant SQLite state. Include uploads and all associated keys according to the deployment chapter.
2. Specifically preserve `.payment-encryption-key` with its matching encrypted data. This original private 32-byte key decrypts saved gateway credentials, bank instruction versions, and pending-payment snapshots. It is not interchangeable with a new generated file of the same name.
3. Keep encrypted backups private and access-controlled. Do not place the key, database, `.env`, or provider secrets into the public frontend, a distributable ZIP, a support screenshot, or a repository commit.
4. Before a migration or account rotation, verify that an isolated restoration can read the original payment settings and pending orders with their matching key. Restoring only the database without that key is insufficient.
5. Distinguish application credential rotation from encryption-key replacement. Updating a provider key for future orders does not justify replacing `.payment-encryption-key`. Old PayPal/Razorpay snapshots deliberately retain the original merchant credentials for their verification and refunds.
6. After rotation, test a new order and an outstanding original-account order in a controlled environment. Keep provider access for older payments until they close. If a key is lost or provider access is revoked, use the original bank/provider records and qualified support to resolve affected payments; do not manufacture successful store states.

Owner's first operating test checklist

Use separate test accounts, synthetic bank instructions, provider test environments, and an isolated deployment first. Do not mix test histories with production accounting. For any later real transaction, use only your own approved account and a transaction authorized under your business process.

1. Verify the basics: real licensed originals are published, final seller/contact details are saved, the correct HTTPS domain is configured, and the intended provider/account/mode appears. A sample file must not be sold as a live original.
2. Complete one successful one-time checkout for each enabled provider in its test environment. Confirm exact USD total, original merchant recipient, paid order, private receipt, and appropriate library access.
3. Cancel or fail a second checkout. Confirm no paid order/access is granted by the cancellation page. Return from a provider without successful payment and verify the same rule.
4. Interrupt the browser return after a successful test payment. Reopen the relevant account's records and refresh/reconcile. Confirm the legitimate original payment is recovered without instructing the buyer to pay again.

5. Retry the same pending checkout and repeat status refreshes. Confirm no duplicate order grant or second charge. Have your technical tester verify duplicate webhook delivery and mismatched amount/currency/recipient rejection with controlled fixtures, not another person's real account.

6. Test a wire instruction snapshot, buyer reference submission, and administrator review. Confirm reporting remains pending, a wrong/short amount cannot grant access, an exact cleared test record can complete once, and the bank reference cannot fulfill a second order.

7. Change the test wire bank while an order is pending. Confirm its old snapshot remains and a new order receives the replacement instructions. Verify reveal requires the applicable administrator password and hides after 60 seconds.

8. Test expiry and late-payment handling in the controlled test process. Confirm an expired order cannot automatically grant access and the customer is told to contact support if money was already sent.

9. Test a Stripe recurring membership, allowed mapped portal change, renewal success/failure, cancellation, and access period. Test a PayPal/Razorpay/wire prepaid term separately and confirm no automatic renewal is created.

10. Test promotion scope, local-time date boundaries, minimum spend, per-account and global caps, concurrent reservations, and the wire hold. Confirm a Stripe membership discount applies to each displayed renewal as promised.

11. Submit a refund request as the test customer. Approve and decline separate appropriate cases. Confirm a completed refund reverses the relevant grant, an independent purchase remains valid, and pending/unknown provider refunds retain access until confirmed.

12. Test the Razorpay uncertain-response guard and Stripe pending-cancellation reconciliation with a technical tester's controlled fixtures. Confirm retries reconcile the original refund without blindly sending a second repayment.

13. Test the wire refund form using synthetic completed-bank evidence. Confirm exact amount, unique reference, administrator password, and explicit declaration are required. Understand that the real operating procedure pays through the bank first.

14. Export order and audit CSVs, inspect receipts, and reconcile demo versus live reporting. Verify prepaid terms do not enter recurring MRR and historic totals/seller snapshots remain unchanged after current settings change.

15. Restore a backup in an isolated environment and check original pending credentials and bank versions. Finish your own external provider/bank settlement checks before announcing a working live route. Bundled local tests and screenshots do not establish that your real merchant account has passed these steps.

Official provider reading and source authority

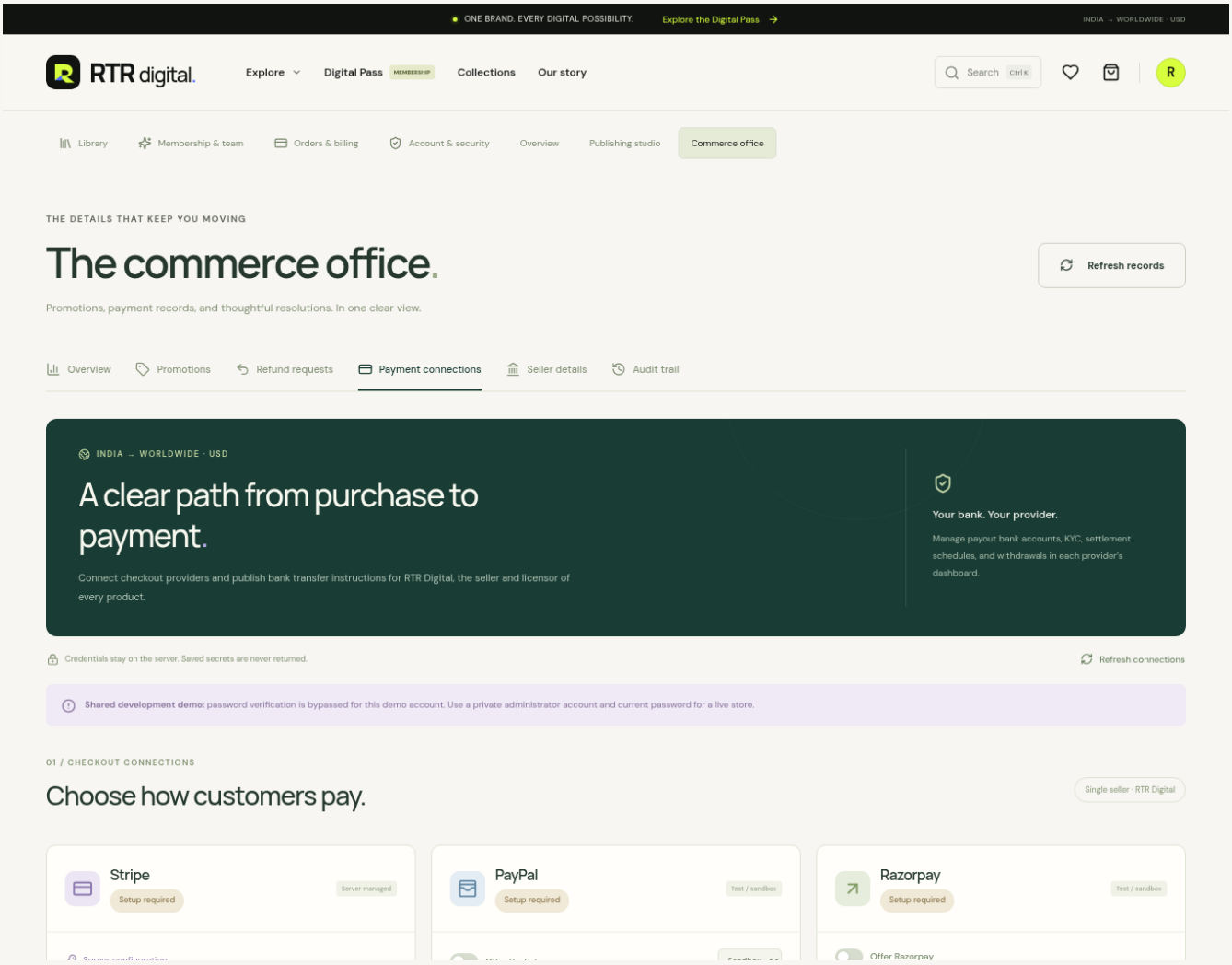
The following are official starting points for the external tasks. The application's dashboard buttons link to Stripe payouts, PayPal business money management, and Razorpay settlements. Provider menu names and account-specific availability should be confirmed in your own account. Automated retrieval of the documentation links was attempted during preparation but the environment's network proxy returned HTTP 403 tunnel errors; their current page contents and live dashboard layouts were not verified.

Provider/task	Official reference
Stripe API keys	https://docs.stripe.com/keys
Stripe webhook setup and signing	https://docs.stripe.com/webhooks
Stripe international payments for India	https://docs.stripe.com/india-accept-international-payments
Stripe customer portal	https://docs.stripe.com/customer-management/activate-no-code-customer-portal
Stripe Tax	https://docs.stripe.com/tax
Stripe payout settings opened by this app	https://dashboard.stripe.com/settings/payouts
PayPal REST applications	https://developer.paypal.com/api/rest/
PayPal sandbox	https://developer.paypal.com/tools/sandbox/
PayPal business money settings opened by this app	https://www.paypal.com/businessmanage/account/money
Razorpay API authentication	https://razorpay.com/docs/api/authentication/
Razorpay international payments	https://razorpay.com/docs/payments/international-payments/
Razorpay settlements	https://razorpay.com/docs/payments/settlements/
Razorpay settlement dashboard opened by this app	https://dashboard.razorpay.com/app/settlements

The application-specific procedures were checked against `src/App.tsx`, `src/features/PaymentSettings.tsx`, `src/features/PaymentCheckout.tsx`, `src/features/CommerceOffice.tsx`, `server/index.mjs`, `server/modules/payments.mjs`, `server/modules/commerce.mjs`, `server/modules/memberships.mjs`, `server/payment-policy.mjs`, `.env.example`, and the supplied payment/deployment guides. Source-derived behavior is the authority for the labels and restrictions described here. This chapter does not record an actual API credential entry, outgoing email, charge, wire, refund, or externally verified payout.

SCREEN REFERENCE

06.1 / Payment connections

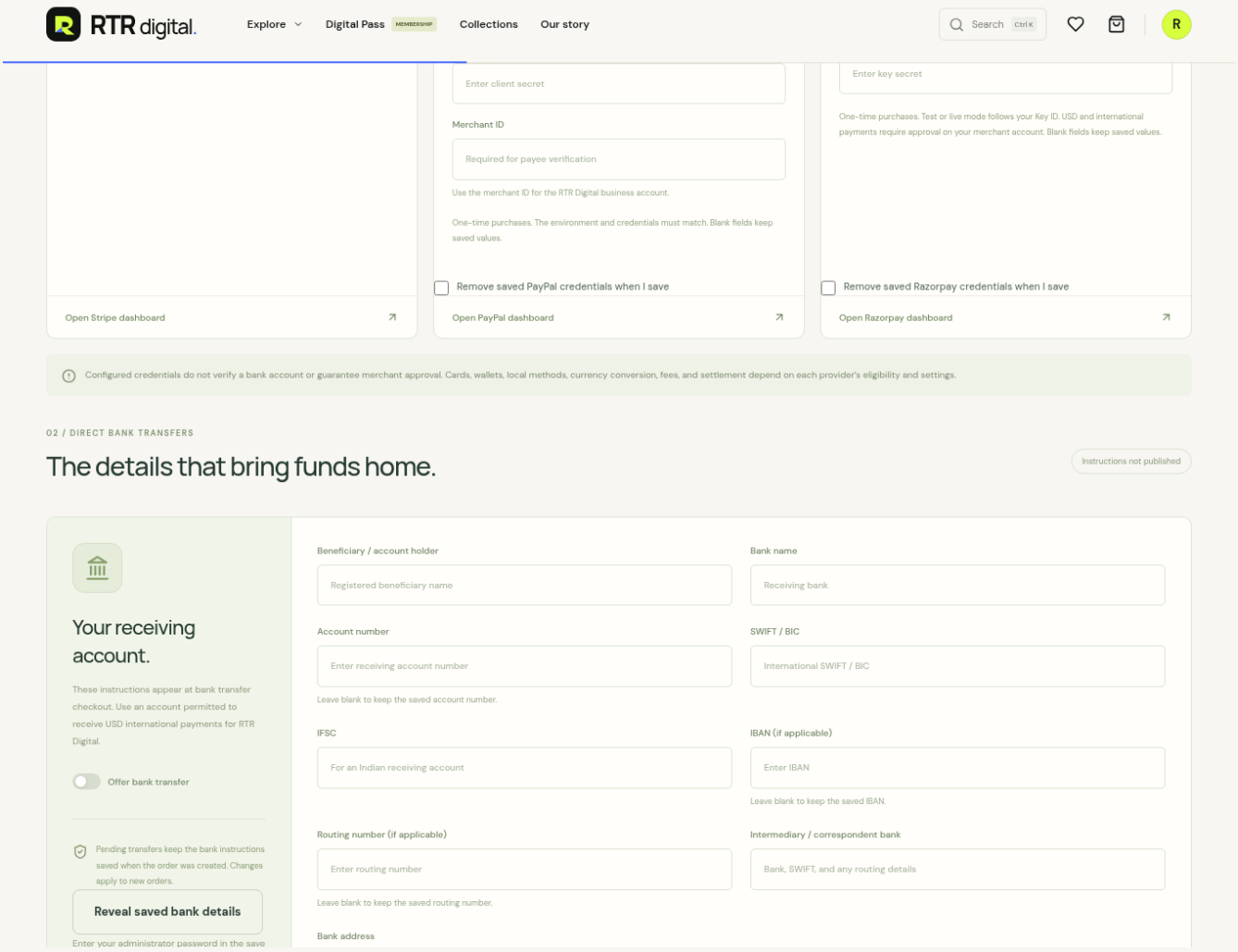


Commerce office → Payment connections in development. The visible demo warnings do not establish a live provider connection.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

06.2 / Wire bank settings

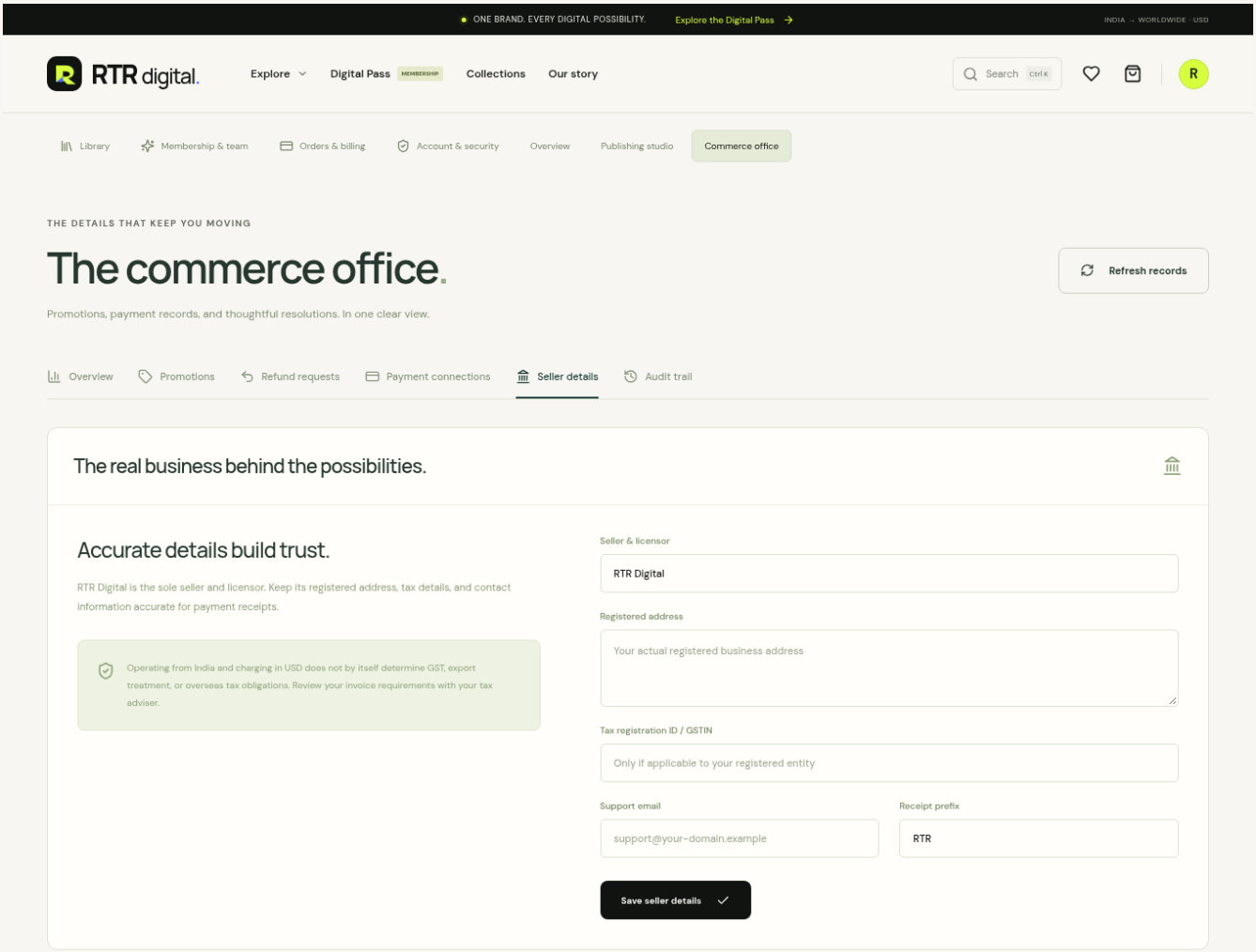


Wire receiving instructions. The training form deliberately has no real bank details; enter your verified information privately.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

SCREEN REFERENCE

06.3 / Seller details



Seller details: RTR Digital is fixed as seller/licensor. Supply your actual legal, tax and contact information.

Captured from an isolated development demonstration on 24 October 2026. Use the numbered procedures in this chapter for complete instructions.

CHAPTER 07

07 Hosting and launch

This chapter takes the complete RTR Digital project ZIP from your computer to a public HTTPS website. The recommended route is one Ubuntu 24.04 LTS virtual private server, Docker Compose, a persistent application data volume, and Caddy for HTTPS. You manage the server; the hosting company supplies the virtual machine, network and billing account.

These are instructions for you to carry out in your own accounts. Preparing this manual did not purchase a domain, create a server, change DNS, provision payment accounts, send email or verify a live transaction. The examples deliberately contain replacement values. Do not publish the examples unchanged.

1. Understand what you are hosting

RTR Digital is a running application with a database. The React frontend becomes files in `dist/`, while the Node.js server supplies accounts, permissions, administration, checkout, private uploads and authenticated downloads. In production, the server serves both the frontend and API on port 3001. The Dockerfile builds the frontend for you.

The database is SQLite, using Node's built-in SQLite driver and write-ahead logging. Use Node.js 24.2 or later when running outside Docker. The included Dockerfile uses the Node 24 image family. Keep one application instance attached to one persistent data directory. Increasing a hosting dashboard's instance count creates a different architecture; do not enable automatic horizontal scaling for this deployment.

Your live installation has three distinct parts:

Part	Example location	What happens if you lose it
Application source and build instructions	`/opt/rtr-digital/app`	Recover from the matching release archive, then rebuild.
Deployment configuration and secrets	`/opt/rtr-digital/compose.yaml`, `Caddyfile`, `.env.production`	Recreate only from a trusted private copy; lost secret values may require provider recovery.
Private persistent application data	Docker volume `rtrdigital_rtr-data`, mounted at `/app/data`	Customer records, orders, uploads and encryption keys require a matching backup. A fresh ZIP does not recover them.

Caddy has its own persistent certificate/configuration volumes. These simplify certificate renewal and restarts. They do not contain the customer database. A backup of the website's source code is not a backup of its operating business data.

1.1 Where you should host it

A general purpose VPS from DigitalOcean, Amazon Lightsail, Akamai/Linode or Hetzner can support this arrangement when the chosen product provides an Ubuntu VM, persistent local disk, a public address, and open inbound HTTP/HTTPS. Compare the plans and regional availability in each provider's current console. Provider offerings, names, prices, taxes and bandwidth allowances change; no price quoted elsewhere in this project should be treated as a current offer.

For an India-based operator, compare an available India region with Singapore and other nearby regions. Test latency from your expected customers, and check the provider's actual data-location and backup options. A nearby server helps latency; it does not itself satisfy privacy, financial or tax obligations. Global downloads also consume outbound bandwidth, so read the transfer allowance and overage policy before selling large files.

For a modest first deployment, two virtual CPUs, about 4 GB RAM and an SSD volume with substantial free space are a reasonable starting point to evaluate. A build can temporarily use more memory than an idle running app. A small plan may run the site but fail during image builds. Size disk from your original files, retained releases, expected growth, temporary upload reservations, image builds and backups. A 5 GiB upload limit does not mean a 5 GB server is usable. This is planning guidance, not a tested concurrency or sales-capacity promise.

A managed service with a paid persistent disk is another option, described later in this chapter. A static site host alone is unsuitable. GitHub Pages, a static Netlify deployment or a static Vercel deployment cannot preserve this application's SQLite database and private uploads or run its complete backend. Putting only `dist/` online may display pages while sign-in, checkout and downloads fail.

Typical shared WordPress/cPanel hosting is also insufficient unless the specific plan supports a continuously running Node 24.2+ application, private durable writable storage, environment variables, HTTPS reverse proxying and adequate request/download limits. An option labeled “Node.js” is not enough if the available runtime is old or disk is replaced on every deployment. Obtain the host's explicit technical answers before buying such a plan.

1.2 Record the values you will use

Keep a private launch notebook in your password manager or another controlled location. Record the provider account, server name, region, public IP address, SSH username, domain registrar, DNS service, renewal dates and location of your backup instructions. Store secret values in the password manager, not a public issue or support screenshot.

This chapter uses the following placeholders:

Example	Replace with
`realdomain.example`	Your registered root domain, such as the actual domain you bought. `.example` domains cannot serve a real public store.
`203.0.113.10`	Your server's actual public IPv4 address. This documentation address will not reach your server.
`owner@realdomain.example`	An email inbox you control for the initial administrator.
`deploy`	The non-root Linux account created in this chapter.

Example	Replace with
`RTR-Digital-Project.zip`	The filename of your complete project archive.

Use the same final HTTPS origin in the application and payment/email configuration. Decide now whether the store will live at the root domain or a subdomain such as `store.yourdomain.com`. The commands below use a root domain. If you choose a subdomain, substitute it consistently in the DNS record, Caddyfile, `PUBLIC_URL`, provider return URLs and webhook URLs.

2. Prepare your computer and project ZIP

2.1 Confirm that you have the actual application

Download the complete project ZIP supplied with the project. Keep an untouched copy in a private release archive on your computer. Extract another copy for inspection. On Windows, right-click the archive and choose the extraction option. On macOS, opening the ZIP normally extracts it. Open the extracted folder and look for `package.json`, `package-lock.json`, `Dockerfile`, `server/index.mjs`, `src/`, `shared/` and `public/`.

The project delivery matters here: the repository's `main` branch may contain only `LICENSE`, while a download branch contains the release ZIP and two guide files. Cloning `main` and running Docker in that folder will not produce a working app. If your folder contains only a license or the delivery artifacts, extract the full application ZIP first. Do not substitute a similarly named repository you found through a search.

You can later maintain the extracted source in a private Git repository. That repository must contain the real project files and lockfile. Keep `.env.production`, live `data/`, encryption keys, merchant credentials and customer exports out of Git. A public repository is not a secret store, even if you later delete the sensitive commit.

2.2 Open the terminal you will use

On Windows 10/11, open PowerShell or Windows Terminal. Recent Windows installations include the OpenSSH client. Run:

```
ssh -V
scp
```

The second command may print usage information and exit with a nonzero result because no files were supplied; that still confirms the program exists. If Windows says the command is unknown, install the OpenSSH Client optional feature using Microsoft's current instructions, or use WinSCP for file transfer and a supported SSH terminal for server commands.

On macOS, open Applications → Utilities → Terminal. On Linux, open your terminal application. Run `ssh -V` to check the SSH client. The server commands in this chapter use Linux shell syntax. Run those commands after signing into the server; do not paste them directly into PowerShell and expect Linux package management to work on Windows.

Throughout this chapter, “on your computer” means the terminal before connecting by SSH. “On the server” means the terminal after a successful SSH login. The command prompt usually changes after login. Run `hostname` if you are unsure which machine you are operating.

2.3 Create an SSH key

An SSH key lets your computer prove its identity to the server. Keep the private half on your computer. The public half is safe to place in your VPS account. If you already have a suitable key and know how it is protected, reuse it according to your own access policy.

On your computer, run:

```
ssh-keygen -t ed25519 -C "rtr-digital-server"
```

Read the proposed filename. If a key already exists at that filename, do not overwrite it. Choose another filename such as `id_ed25519_rtr`, or cancel and inspect your existing keys. Set a strong passphrase when prompted and store it in your password manager. The passphrase protects the private key if the file is stolen; it is different from the server's login password.

Display the public key. With the default filename, PowerShell uses:

```
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
```

macOS/Linux uses:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the entire single line beginning `ssh-ed25519`. Do not copy the file without `.pub`. If you chose another filename, use that filename in these commands and later `ssh -i` commands. Save the private key securely; losing the only working key can lock you out until you use the hosting provider's recovery console.

3. Buy the domain and create the VPS

3.1 Register and protect your domain

Choose a domain registrar you trust, check that the spelling is correct, purchase the domain and enable renewal reminders or automatic renewal with a valid payment method. Enable two-factor authentication on the registrar account. Keep its recovery codes private and accessible if your phone is lost.

Domain registration and web hosting can be with different companies. A domain points to a server through DNS; buying a domain does not upload the website. The place where you edit DNS is whichever service controls the domain's current authoritative nameservers. It may be the registrar or a separate DNS provider. If the registrar says another service manages the nameservers, edit the records at that service.

Do not change nameservers casually when the domain already handles business email. A nameserver change can remove effective MX, SPF, DKIM, DMARC and verification records unless you recreate them at the new DNS service. For this deployment, changing the relevant website A record is usually sufficient.

3.2 Create the server

In your chosen hosting console:

1. Create and verify your account, add billing details, and enable two-factor authentication.
2. Open the VM/instance creation page. The exact label may be “Droplet,” “Instance,” “Linode,” “Server” or something similar.

3. Choose Ubuntu 24.04 LTS, normally the standard 64-bit image rather than a preconfigured WordPress image. These commands assume an Ubuntu server with `sudo` access.
4. Choose your evaluated region, CPU/RAM plan and persistent SSD size. Read bandwidth and backup charges before confirming.
5. Add the public SSH key created above through the console's SSH key option. Give it a recognizable name.
6. Name the server, for example `rtr-production-01`. Enable provider backups if you want that extra recovery layer; they do not replace the consistent application backups in the next chapter.
7. Create the instance and wait until its status indicates it is running.
8. Copy its public IPv4 address to your private notebook. If the provider offers a reserved/static address, understand how to retain it across instance replacement. A changed public IP requires a DNS update.

If there is a provider network firewall, allow inbound TCP 22 for SSH from your current public IP where practical, and TCP 80/443 from the internet. UDP 443 is optional for HTTP/3. Do not open 3001 publicly. If your home IP changes, update the SSH allow rule before attempting to reconnect. IPv6 requires matching firewall rules if you publish IPv6 DNS.

3.3 Make your first SSH connection

On your computer, replace the address and use the initial username shown by the hosting company:

```
ssh root@203.0.113.10
```

Some providers use `ubuntu` or another initial account instead of `root`. Use their documented account. If you selected a nondefault private key filename, supply it explicitly, for example:

```
ssh -i ~/.ssh/id_ed25519_rtr root@203.0.113.10
```

On the first connection, SSH shows a host fingerprint. Compare it with the fingerprint available through the provider's console or trusted server-access documentation, then accept it if it matches. A changed fingerprint on a later connection deserves investigation: rebuilding the VM can change it, but an unexplained change should not be accepted automatically.

You should now see a Linux shell. Check the host and operating system:

```
hostname
cat /etc/os-release
```

3.4 Create a normal deployment user

If the provider already created a normal `sudo` user, you can use that account consistently instead. For an initial root account, create `deploy` on the server:

```
adduser deploy
usermod -aG sudo deploy
install -d -m 700 -o deploy -g deploy /home/deploy/.ssh
cp /root/.ssh/authorized_keys /home/deploy/.ssh/authorized_keys
chown deploy:deploy /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
```

`adduser` asks for a password and optional descriptive fields. Choose a strong password; you may need it when using `sudo`. The copy command assumes your provider installed your public key in root's

`authorized_keys`. If it did not, place your public key's complete line into the new user's `authorized_keys` using a text editor. Copy the public key only.

Keep the current connection open. Open a second terminal on your computer and test:

```
ssh deploy@203.0.113.10
sudo -v
```

Enter the `deploy` user's password if requested. Continue in this second connection after `sudo` succeeds. Do not disable any working access method until the replacement has been tested. Once key login and provider console recovery are confirmed, you can follow Ubuntu's SSH hardening instructions to disable password/root SSH login if appropriate. Check the effective SSH configuration and test a new connection before closing the existing one.

4. Prepare Ubuntu and install Docker

4.1 Update the server and set its firewall

On the server as `deploy`, run:

```
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install -y ca-certificates curl unzip rsync dnsutils python3 gnupg ufw
sudo timedatectl set-timezone UTC
timedatectl status
```

An upgrade may recommend a reboot. Reboot during this initial setup if required, wait for the VM to return, and reconnect with SSH. UTC makes application logs and payment-provider timestamps easier to compare. The server's time must remain synchronized for certificates and authenticator codes to work reliably.

Allow SSH before enabling the firewall:

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 443/udp
sudo ufw enable
sudo ufw status verbose
```

Read the confirmation prompt. Keep your SSH session open and test another SSH login after enabling it. These commands assume SSH uses the normal port 22 and Ubuntu's `OpenSSH` profile. If you changed SSH's port, allow the actual port first. The UDP rule is optional and corresponds to HTTP/3; ordinary HTTPS uses TCP 443.

Docker-published ports can interact with Linux firewall rules differently from ordinary processes. Do not rely on UFW to make an accidentally published application port private. This manual's Compose file publishes only Caddy's 80/443 ports, and the provider firewall should also expose only the intended services.

4.2 Install Docker Engine and Compose from Docker's repository

Read Docker's official Ubuntu installation instructions at https://docs.docker.com/engine/install/ubuntu/ before applying these commands to a future Ubuntu release. The following recipe targets Ubuntu 24.04 LTS. The documentation website could not be fetched from the preparation environment, so this manual does not claim a live comparison against its latest page or a completed VPS installation.

On a fresh server, add Docker's signing key and repository:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker version
sudo docker compose version
sudo docker run --rm hello-world
```

The multi-line `tee` command must include the final `EOF` on its own line. Docker's guide discusses conflicts with distribution-provided packages; follow that section if this is an existing server. Do not uninstall packages blindly on a server already running other workloads.

This manual uses `sudo docker ...` deliberately, so it does not require adding `deploy` to the `docker` group. Membership in that group effectively grants root-level control over the host. If your operational policy chooses group membership, understand that consequence and keep commands consistent.

The test container prints a confirmation and exits. It does not host your application. If installation fails, resolve the package or network error before proceeding. Repeatedly adding alternative installation scripts can leave a confusing mixed installation.

5. Upload the complete source

5.1 Create the deployment directories

On the server:

```
sudo install -d -m 0750 -o deploy -g deploy /opt/rtr-digital
sudo install -d -m 0750 -o deploy -g deploy /opt/rtr-digital/app
```

These are source/configuration folders. The customer data will live in a Docker volume. An update to the source folder must not replace that volume.

5.2 Copy the ZIP from your computer

In a terminal on your computer, use your real local path. A Windows example is:

```
scp "C:\Users\YOURNAME\Downloads\RTR-Digital-Project.zip" deploy@203.0.113.10:~/rtr-project.zip
```

A macOS/Linux example is:

```
scp ~/Downloads/RTR-Digital-Project.zip deploy@203.0.113.10:~/rtr-project.zip
```

If you use a separate private key filename, add the appropriate `-i` argument. Wait until the transfer completes. For WinSCP, choose SFTP, enter the server's IP and `deploy` username, select your SSH key as supported by the current client, connect, and upload the ZIP to `/home/deploy/`. Verify the host fingerprint there too. Do not upload credentials to a public web directory as a transfer shortcut.

5.3 Extract and check the contents on the server

Back in the server's SSH terminal:

```
unzip -l ~/rtr-project.zip | head -n 30
rtr_extract=$(mktemp -d /tmp/rtr-source.XXXXXX)
unzip -q ~/rtr-project.zip -d "$rtr_extract"
find "$rtr_extract" -maxdepth 3 -name package.json -not -path '*/node_modules/*'
```

The last command tells you where the real project root is. Suppose it prints `/tmp/rtr-source.ABC123/RTR_Digital/package.json`. Set the source directory using the actual directory printed, without `package.json` at the end:

```
rtr_source="$rtr_extract/RTR_Digital"
test -f "$rtr_source/package.json"
test -f "$rtr_source/package-lock.json"
test -f "$rtr_source/server/index.mjs"
test -f "$rtr_source/Dockerfile"
```

If the ZIP has no enclosing folder, use `rtr_source="$rtr_extract"`. If a check fails, stop and correct the path or archive. Do not create a missing `package.json` by hand to make the check pass. Use only an archive from a trusted source, and inspect unexpected archive contents before extracting them onto a sensitive host.

For this first installation, copy source while excluding development dependencies, private data and secrets:

```
rsync -a \
  --exclude='node_modules/' \
  --exclude='data/' \
  --exclude='dist/' \
  --exclude='.git/' \
  --exclude='.env' \
  --exclude='.env.*' \
  --exclude='test-results/' \
  --exclude='playwright-report/' \
  --exclude='deliverables/' \
  "$rtr_source/" /opt/rtr-digital/app/
cp "$rtr_source/.env.example" /opt/rtr-digital/app/.env.example
cd /opt/rtr-digital/app
ls package.json package-lock.json Dockerfile server/index.mjs
```

The separate copy restores the harmless template excluded by the wildcard. Docker will rebuild `dist/` using the lockfile. Do not use this first-install procedure as an update recipe for a store with sales; the next chapter explains backups, versions and rollback.

If you instead use a private Git repository, place the complete source under `/opt/rtr-digital/app` and check the same files. Configure a read-only deployment credential where possible. Do not embed an access token into a clone command that will be retained in shell history. A Git push or clone does not automatically upload a customer's private data volume.

6. Create the production configuration

6.1 Understand the finished file layout

The deployment will look like this:

```
/opt/rtr-digital/
├── compose.yaml
├── Caddyfile
├── .env.production
└── app/
    ├── Dockerfile
    ├── package.json
    ├── package-lock.json
    ├── server/
    ├── shared/
    ├── src/
    └── public/
```

The Compose file belongs one directory above the project's Dockerfile. Its `build: ./app` depends on that layout. Putting `compose.yaml` inside `app/` without changing the build path produces a missing-directory error.

6.2 Create the private environment file

On the server:

```
cd /opt/rtr-digital
umask 077
nano .env.production
```

If `nano` is absent, install it with `sudo apt-get install -y nano`, then reopen the file. Paste this starting configuration, replacing the administrator email and password:

```
DEMO_MODE=false
ADMIN_EMAIL=owner@realdomain.example
ADMIN_PASSWORD=replace-with-a-long-unique-password-from-your-password-manager
STRIPE_AUTOMATIC_TAX=false
TRIALS_ENABLED=false

# Add these only when you configure their real services:
# RESEND_API_KEY=re_replace_me
# EMAIL_FROM=RTR Digital <accounts@realdomain.example>
# STRIPE_SECRET_KEY=replace_me
# STRIPE_WEBHOOK_SECRET=replace_me
# STRIPE_PRICE_STARTER_MONTHLY=price_replace_me
# STRIPE_PRICE_STARTER_YEARLY=price_replace_me
# STRIPE_PRICE_CREATOR_MONTHLY=price_replace_me
# STRIPE_PRICE_CREATOR_YEARLY=price_replace_me
# STRIPE_PRICE_STUDIO_MONTHLY=price_replace_me
# STRIPE_PRICE_STUDIO_YEARLY=price_replace_me
```

Use a unique password of at least 12 characters. For straightforward environment-file syntax, a long password-manager-generated string of letters, numbers, hyphens and underscores is convenient. Values containing dollar signs, quotes, spaces or comment characters require careful Compose environment-file handling; check Docker's syntax rather than guessing. Do not put your password directly into a shell command where it enters command history.

In `nano`, save with Ctrl+O, press Enter to confirm the filename, and exit with Ctrl+X. Then restrict the file:

```
chmod 600 .env.production
```

Use a genuine inbox and an email address that has not already been registered as a customer in this database. Bootstrap settings create an administrator only when the address does not already exist. They do not promote an existing customer, reset an existing password or create a second copy of an account. Production rejects the supplied shared demo addresses. Use the owner's actual account from the beginning.

The Compose file below sets `NODE_ENV`, `DATA_DIR`, `PORT` and `PUBLIC_URL`. Keep those four values there for this recipe. Other settings belong in `.env.production`. If the same variable appears in both places, the explicit Compose `environment` setting wins. An `env_file` passes values into the container; it does not make those values available to arbitrary `${...}` substitutions while Compose parses its YAML.

Node does not automatically load a file named `.env` or `.env.production`. This deployment works because Compose's `env_file` supplies the environment. Outside Docker, use the host's environment controls or an explicit command such as `node --env-file=.env.production server/index.mjs` from the correct directory.

6.3 Create the Compose file

Run `nano /opt/rtr-digital/compose.yaml` and paste the following. Replace `https://realdomain.example` with the final HTTPS origin, without a trailing path:

```
name: rtrdigital

services:
  app:
    build: ./app
    restart: unless-stopped
    env_file:
      - .env.production
    environment:
      NODE_ENV: production
      DATA_DIR: /app/data
      PORT: "3001"
      PUBLIC_URL: https://realdomain.example
    expose:
      - "3001"
    volumes:
      - rtr-data:/app/data
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"

  caddy:
    image: caddy:2
    restart: unless-stopped
```

```
    depends_on:
      - app
    ports:
      - "80:80"
      - "443:443"
      - "443:443/udp"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data
      - caddy-config:/config
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"

volumes:
  rtr-data:
  caddy-data:
  caddy-config:
```

Use spaces, not tabs, for YAML indentation. The application has `expose`, which makes its internal port available on the Compose network; it has no public `ports` mapping. Do not add `3001:3001`. Caddy is the public entry point and forwards requests privately to the app.

The fixed project name `rtrdigital` makes the application's default Docker volume name `rtrdigital_rtr-data`. Keep that project name stable during upgrades. Using another project name or another named volume can start the app with a new empty database, which can look like lost records even while the original volume still exists.

The included Dockerfile runs as the `node` user, UID/GID 1000, and creates `/app/data` with appropriate ownership in the image. The volume must remain writable by that user. Do not solve permission problems by switching production to root or by making private data world-writable. The next step initializes the volume ownership explicitly.

The image names above follow maintained major-version families. Before a commercial launch, record the exact tested image versions or digests and use your update process to advance them. Neither `caddy:2` nor `node:24-bookworm-slim` is an immutable release identifier. Avoid switching the application to an unsupported Node major while resolving an unrelated issue.

6.4 Create Caddy's HTTPS configuration

Run `nano /opt/rtr-digital/Caddyfile`. For a root-domain store with optional `www` redirection, paste and replace both domain names:

```
realdomain.example {
    request_body {
        max_size 125MB
    }
    reverse_proxy app:3001
}

www.realdomain.example {
    redir https://realdomain.example{uri} permanent
}
```

Only include the `www` block if you will create DNS for `www` as well. If you use `store.yourdomain.com`, a single block for that actual hostname is sufficient. No manual certificate filenames or paid certificate purchase is required for the normal public-domain Caddy setup. Caddy obtains and renews HTTPS certificates when DNS and network access are correct.

The body limit allows a standard 100 MiB original and its optional 10 MiB cover in the same request, plus multipart overhead. The combined files alone can total 115,343,360 bytes; a `110MB` proxy limit is too small for that maximum combination. Large resumable releases are sent as multiple bounded requests, normally 8 MiB per chunk. Supporting a 5 GiB original does not require a 5 GiB reverse-proxy request limit. A larger proxy allowance does not override the application's own upload or disk limits. If you change chunk sizes or standard multipart behavior, revisit the relevant limits together.

6.5 Validate syntax and initialize the data volume

On the server:

```
cd /opt/rtr-digital
sudo docker compose config --quiet
sudo docker compose pull caddy
sudo docker compose build app
sudo docker volume create rtrdigital_rtr-data
sudo docker run --rm \
  --mount type=volume,src=rtrdigital_rtr-data,dst=/data \
  alpine:3.22 sh -c 'chown 1000:1000 /data && chmod 700 /data'
sudo docker compose run --rm --no-deps --entrypoint node app \
  -e 'const f=require("node:fs"); f.accessSync("/app/data", f.constants.W_OK); console.log("data directory is writable")'
```

The temporary Alpine container changes ownership of the volume's top directory, then exits. It does not initialize the application database. On a fresh volume this is sufficient. For a restored volume, preserve or correct ownership throughout the restored files as described in the recovery chapter. The final command runs under the image's normal non-root user and confirms write access without starting the server.

`docker compose config` without `--quiet` can print expanded secret values. Do not paste its full output into a ticket or public chat. `build` downloads dependencies using `npm ci`, compiles the frontend and constructs the image. The server does not need a separate public Vite process, and you do not run `npm run dev` on the public host.

If the build fails, read the first useful error. Missing files usually mean the wrong ZIP directory. A TypeScript error requires a source fix. An out-of-memory error requires build resources or a build pipeline change. Do not bypass the build by publishing a partial frontend with an unavailable backend.

7. Point the domain at the server

7.1 Add DNS records

In the authoritative DNS service for your domain, locate its record editor. Labels vary by provider. Add or update:

Type	Name/host	Value/target	Purpose
A	`@` or the blank root name	Your server IPv4 address	Root-domain store.
A	`www`	The same server IPv4 address	Optional `www` redirect.

For a subdomain deployment, use its label, such as `store`, instead of `@`. Some interfaces want a full hostname; read their example. A TTL around 300 seconds is useful during an initial change if the DNS service supports it. Existing cached records can remain in use until their prior TTL expires, and updates can take longer than expected across resolvers.

Remove or correct conflicting website records for those same hostnames. An old AAAA record points IPv6-capable visitors to an IPv6 server, even if the A record is right. Publish AAAA only if this VPS has working configured IPv6 and matching firewall rules. Leave email-related MX/TXT records in place. Do not put `https://`, a URL path or `:3001` into an A record.

If using a DNS/CDN proxy service, begin with ordinary DNS-only records while you establish the origin. A proxy adds its own TLS mode, request-size limits, caching and timeouts. Later enable it deliberately, using an appropriate secure origin mode and testing private downloads and large resumable uploads. Do not cache authenticated API or private original-content responses as public files.

7.2 Check the results

On the server or a computer with `dig`:

```
dig +short A realdomain.example
dig +short AAAA realdomain.example
dig +short A www.realdomain.example
```

Replace the names. The A answer should be your real server address; no AAAA answer is fine if you are not deploying IPv6. On Windows, `Resolve-DnsName realdomain.example -Type A` is a useful alternative. Check from another network or public resolver if your home network still sees the prior result.

DNS correctness is necessary for Caddy's certificate validation. TCP ports 80 and 443 must reach this server through both its provider firewall and host firewall. Another process cannot already occupy those ports. Restrictive CAA records or a corporate network restriction may also interfere with issuance. Diagnose those conditions instead of disabling HTTPS.

8. Start the website and complete the first login

8.1 Start and inspect the containers

On the server:

```
cd /opt/rtr-digital
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs --tail=100 app
sudo docker compose logs --tail=100 caddy
```

`-d` starts the services in the background. The application log should report production mode. The image's health check requests `/api/health`; allow startup time before treating a transitional health state as a fault. Caddy may need time to obtain certificates after fresh DNS changes.

Check the public endpoint from your computer, replacing the domain:

```
curl -fsS https://realdomain.example/api/health
```

On Windows PowerShell, use `curl.exe` if `curl` resolves to a PowerShell alias. The expected response includes:

```
{"status":"ok","service":"RTR Digital","demoMode":false}
```

Also open `https://realdomain.example/` in a normal browser. Check the address bar's connection information and certificate hostname. Opening the server IP directly is not the same check; the certificate belongs to the domain. A healthy API alone does not establish that uploads, payments, email, backups or your policies are ready.

If you configured `www`, open its HTTPS address and confirm that the browser ends at the root domain with the requested path retained. Check the HTTP address too; Caddy should redirect to HTTPS. Do not type card details or administrator credentials into a page showing a certificate warning.

8.2 Sign in as the owner

Open the website's sign-in control and use the administrator email/password from `.env.production`. The app created that account on first successful startup. You do not need to register through the customer form. Confirm that the administrator workspace, Publishing studio and Commerce office are available.

If the account signs in as a customer, the email probably existed before bootstrap. Changing `ADMIN_PASSWORD` does not alter that account's role. Do not delete the production database to retry. Investigate with a backup and an appropriate account-recovery/administration procedure. On a genuinely fresh installation, use a new unused owner address and re-create the app container after changing the bootstrap settings.

Once the administrator has been created and you have confirmed a fresh sign-in, remove `ADMIN_PASSWORD` from `.env.production`. You can remove `ADMIN_EMAIL` as well because the stored account persists. Save the password in your password manager, then apply the environment change:

```
cd /opt/rtr-digital
nano .env.production
sudo docker compose up -d --force-recreate app
```

A simple `docker compose restart app` does not load edited environment-file contents into a newly created container. Re-creation does. The named data volume remains attached, so the administrator and records persist. Confirm sign-in again before continuing.

Configure account email before relying on password recovery, and enable the owner's MFA through account settings after confirming the backup-key policy below. Store MFA recovery codes offline or in a protected password manager. They are different from encryption keys and SSH recovery credentials.

8.3 Keep encryption-key sources stable

On first startup, the app generates its private keys inside the data volume. Preserve `license-key-master`, `.payment-encryption-key` and, unless you deliberately configured an environment override, `.account-encryption-key`. They are paired with encrypted records in the database. Deleting a key to “reset” an error can make those records unreadable.

`ACCOUNT_ENCRYPTION_KEY` is an optional alternative source for account MFA encryption. If you choose it, set a high-entropy stable secret before the first MFA enrollment, keep it in the secret store and include it in protected configuration backups. Do not add, remove or change it on an existing installation without a

migration plan for already-encrypted MFA data. Leaving the default file-backed approach in place is simpler for a first installation with complete volume backups.

Never remove the payment or license keys to turn a provider off. Use the app's supported connection settings to disable a payment method. Keep the keys and historical records for legitimate recovery and reconciliation needs.

9. Turn a reachable website into a launch-ready store

9.1 Configure business identity and policies

In Commerce office, set the real business address, support contacts, tax information and receipt details. RTR Digital is the fixed seller and licensor. The “Creator” membership is a customer tier, and Studio is a customer team; these are not outside sellers.

Review terms, licensing, refunds, delivery, privacy, support expectations and any claims on the storefront against the actual business. Supply final product-specific licensing terms and required disclosures. A generated payment receipt is not automatically a valid GST invoice or export accounting document. An India-based business taking USD payments needs its own advice and registrations for the actual transactions and customer jurisdictions.

9.2 Publish original products

Production intentionally hides illustrative sample catalog products. An empty public catalog after deployment can be the correct result. In Publishing studio, create a draft, enter the title/category/description/USD price, upload your actual licensed original, and separately choose any public preview. If a custom cover is required, create the product through **Admin workspace → Upload a product** instead: that standard creation form has the cover picker and accepts an original up to 100 MiB. The studio has no cover-upload or cover-replacement control; studio-created products can use generated artwork. Publish only after the file is stored and the metadata is ready.

Use a small real test product first. Check its public product page, preview and category/search behavior. A preview is public. The full original should require an eligible purchase or membership. Log out and confirm that guessing a content URL does not make the original public. Also test an entitled customer's actual download and, for supported types, the original viewer.

Standard uploads are capped at 100 MiB by the application; resumable releases default to 5 GiB per file, 8 MiB chunks and a 10 GiB pending reservation budget. Actual free disk space can impose lower limits. Large software or archive files are not automatically malware-scanned. Establish a file-review/scanning process before distributing them, and retain your source/license evidence.

9.3 Configure account email

The implemented transactional email adapter uses Resend. It is not a generic SMTP settings screen. You need a sender identity that the service authorizes, a server API key, the correct public origin, and successful delivery tests. Resend console names may change; use its current domain-verification instructions for the exact records.

1. Open https://resend.com/ in your browser, create or sign in to your own account, and secure the account. Read the current account limits and approved sending requirements.

2. In its domain/sender management area, add a domain you control. Decide whether account messages should use the root domain or a dedicated sending subdomain. Use the identity the service actually verifies, rather than assuming a display name establishes domain ownership.

3. Keep the verification page open. It supplies DNS record types, hostnames, values and any priority fields. Open a second browser tab at the DNS service that currently controls your domain's authoritative nameservers.

4. Copy each required record exactly into DNS. A domain-management interface may append your domain to the hostname automatically. Follow its format so you do not accidentally create `record.yourdomain.com.yourdomain.com`. Preserve significant punctuation and the full record value.

5. Preserve existing business-email MX and verification records unless the provider's instructions specifically require a reviewed change. Do not create several separate SPF policy records for the same hostname; have the existing policy combined correctly if necessary. Sending-domain verification is separate from the website's A record.

6. Return to the sender-verification page, request its check, and wait until it reports the domain ready. DNS changes can remain cached; repeatedly inventing different values does not make propagation faster. If verification fails, compare hostname, type and full value before changing anything else.

7. Create an API key with permission to send for the intended service/domain, using the least access that supports your setup. Copy it into your password manager. Do not put it into React source, a frontend `VITE_` variable, a screenshot or a public Git commit.

8. On the VPS, open `/opt/rtr-digital/.env.production` privately with your editor. Add `RESEND_API_KEY` with your actual key and `EMAIL_FROM` with the authorized sender, for example `RTR Digital <accounts@YOUR-VERIFIED-DOMAIN>`. Replace the domain. The actual sender value must match the service's verified identity rules.

9. Confirm that Compose's `PUBLIC_URL` is your real final HTTPS origin. Account links must return to the live site, not to localhost, an old staging domain or a placeholder. Save the configuration and recreate the application with `sudo docker compose up -d --force-recreate app` from `/opt/rtr-digital`.

10. Register a controlled customer account using an inbox you can actually read. Request verification through the account screen. Check the inbox and spam folder, then inspect the email provider's delivery event if the message is absent. A successful API response and actual inbox delivery are different observations.

11. Before following the message, inspect its hostname and confirm it uses your HTTPS site. Open it and confirm the account becomes verified. Request a password reset for the same controlled account, follow the new recovery message and set a new unique password. Confirm the old password no longer signs in and the new one does.

12. Record the test date, recipient account and observed result without saving secret links or passwords in a public report. If delivery fails, check the provider event, sender authorization, domain status, server configuration and the specific error before retrying repeatedly.

Do not infer working recovery from the presence of an API key. Production does not expose development mail previews as an email fallback. Support form intake is stored locally; it does not by itself send a personal support reply or marketing campaign. An outbound sender also does not create a business inbox: arrange a working support mailbox separately and publish that address in your actual business contact information.

9.4 Configure and test payments

Follow the payment/bank chapter and the supplied payment guide. Configure only providers for which your actual merchant account has been approved. Stripe secrets are runtime environment settings. PayPal, Razorpay and wire instructions are configured through the administrator's payment connections, with private encrypted storage. Gateway settlement bank accounts are managed in the provider's merchant dashboard.

Use provider test/sandbox modes first. Check success, canceled payment, failed payment, duplicate/retried callback and fulfillment behavior. Configure the correct webhook endpoint and verify its signing secret and provider delivery log. For Stripe, the endpoint is `https://your-real-domain/api/stripe/webhook`. A successful redirect back to the website does not itself establish payment or grant access.

Check the distinction between recurring Stripe membership and fixed-term PayPal/Razorpay/wire membership. Configure Stripe price mappings and the Billing customer portal if using live self-service changes. Do not advertise automatic renewal for prepaid methods. Keep optional no-card trials off until verified email and original eligible membership content are functioning.

Before enabling real sales, verify your merchant's permitted currencies, India/international eligibility, payout schedule, refund process and customer-facing statements in the actual provider account. A “configured” capability indicator in the app checks configuration presence, not account approval or settlement. This manual and the supplied local tests do not prove that any external provider can charge or settle for your particular business.

9.5 Test persistence and recovery before announcing the launch

Create a controlled real registered test account, add a draft product and upload a small original. Restart/recreate the application and confirm that those records remain. Then perform the complete backup and isolated restoration procedure in the next chapter. The restoration drill should show that the database, originals and keys are present together.

Do not discover on launch day that your provider's disk is ephemeral, your named volume changed, your backup omitted hidden files, or you cannot find the MFA recovery codes. Record the drill's date, the backup filename and the results. Keep test payment activity clearly identified in your operating records.

10. Managed alternative: a persistent Render web service

Use this route instead of the VPS recipe if you prefer a provider to manage the operating system and TLS. It still requires you to manage the application, database, disk capacity, secrets and backups. Read Render's current persistent disk and Node-runtime documentation before selecting a plan. Do not assume a free or ephemeral service preserves SQLite between deployments.

10.1 Prepare a private source repository

Extract the complete application ZIP on your computer and put that application's actual root files in a private Git repository. Check that `package.json`, `package-lock.json`, `server/`, `src/`, `shared/`, `public/` and `Dockerfile` are present. Do not connect the delivery repository's license-only `main` branch. Do not commit live `data/` or production environment files.

Choose a tested currently supported Node 24 patch release that satisfies the app's minimum of 24.2.0. Specify it using Render's currently documented runtime-version mechanism, such as `NODE_VERSION`, and

verify the selected version in build/runtime logs. Avoid relying on a platform default that may still be an older Node version. Updating the version later is an application update that needs validation.

10.2 Create one native Node web service

In the provider console, create a Web Service, connect the private repository and select the source branch containing the full app. Choose a native Node runtime for this alternative. If the UI automatically selects Docker because the repository contains a Dockerfile, deliberately choose the native runtime where supported, or consult Render's current setup instructions. These native instructions do not describe Docker disk ownership.

Set the root directory to the application's actual repository root. Set the build command to:

```
npm ci && npm run build
```

Set the start command to:

```
node server/index.mjs
```

Use one running instance and a service plan that supports a persistent disk. Add a persistent disk mounted at `/var/data`, with space for originals, versions, temporary uploads and growth. Confirm that this mount is available to the running service and persists across deploys. Configure these environment settings through the provider's secret interface:

```
NODE_ENV=production
DEMO_MODE=false
DATA_DIR=/var/data
PUBLIC_URL=https://your-real-domain
ADMIN_EMAIL=your-real-unused-owner-email
ADMIN_PASSWORD=your-long-unique-initial-password
STRIPE_AUTOMATIC_TAX=false
TRIALS_ENABLED=false
```

Let the platform supply `PORT` if that is its documented contract; the app reads it and listens on `0.0.0.0`. Do not configure `API_PORT` to a conflicting value. Set the health-check path to `/api/health` where the service settings allow it. Add email and payment secrets only when you are ready to validate those integrations.

10.3 Deploy and attach the domain

Start the deployment, watch the build output, and inspect the service logs for production startup. Open the provider-issued HTTPS service URL and check `/api/health`. If your configured `PUBLIC_URL` is already the final domain, account/payment links should only be tested after that domain resolves and TLS is working.

Use the service's custom-domain controls to add your real hostname. The platform displays the exact DNS targets to create; use those targets rather than the VPS IP examples earlier in this chapter. Some providers use a CNAME for a subdomain and a special root-domain configuration. Wait for domain verification and certificate status to become valid, then set the final `PUBLIC_URL` and redeploy/restart through the supported settings workflow.

Sign in with the bootstrap administrator, confirm the role, remove the bootstrap password from the provider environment and deploy the changed settings. Test data persistence across a redeployment. Establish a backup procedure for the full persistent disk and configuration, including every key, using the provider's current supported shell/snapshot/export facilities and a consistent SQLite process. Do not

apply the Docker volume commands from the next chapter to a native managed service; there is no corresponding local Docker volume.

Provider snapshots and disk restore policies have their own retention and availability constraints. Read them, test a separate restore and keep an independent encrypted backup. A managed service can reduce server maintenance while leaving disk corruption, accidental deletion, compromised credentials and missing backups within your responsibilities.

11. Final go-live checklist

Work through this list before announcing the domain or accepting live payments:

- [ ] The full application source, not a license-only repository or frontend-only build, is deployed.
- [ ] The domain resolves to the intended service, HTTPS is valid, and any `www` redirect works.
- [ ] `/api/health` returns `status: ok` and `demoMode: false` from the public HTTPS address.
- [ ] There is one application instance, a durable private data location and no public application port 3001.
- [ ] The real owner can sign in, the bootstrap password has been removed from runtime configuration, and owner recovery/MFA procedures are recorded.
- [ ] All encryption key sources are preserved and included in protected recovery material.
- [ ] Original licensed products, covers, public previews and private downloads have been checked with logged-out and entitled users.
- [ ] Business details, license terms, refund/privacy/support policies and tax/export arrangements match actual operations.
- [ ] Verification and recovery email reach real test inboxes with correct HTTPS links.
- [ ] Each enabled payment method has been checked in the actual provider account and its supported test environment, including fulfillment, webhook verification and refunds where applicable.
- [ ] A consistent encrypted off-server backup exists and an isolated restore has succeeded.
- [ ] Someone is responsible for support, bank reconciliation, payment exceptions, monitoring, updates, storage and domain/provider renewals.

When these checks pass in your own environment, switch the intended payment configuration to the approved live mode, repeat controlled live verification as appropriate for the provider, and monitor the first real orders closely. Keep the operating log. Do not mark an external step complete merely because this document contains instructions for it.

12. Official references

Consult these sources for current provider-specific details. Console labels and procedures can change:

- Ubuntu server documentation: https://documentation.ubuntu.com/server/
- Ubuntu OpenSSH guidance: https://documentation.ubuntu.com/server/how-to/security/openssh-server/
- Docker Engine on Ubuntu: https://docs.docker.com/engine/install/ubuntu/
- Docker Compose: https://docs.docker.com/compose/

- Docker volumes: https://docs.docker.com/engine/storage/volumes/
- Docker firewall behavior: https://docs.docker.com/engine/network/packet-filtering-firewalls/
- Caddy automatic HTTPS: https://caddyserver.com/docs/automatic-https
- Caddy reverse proxy: https://caddyserver.com/docs/caddyfile/directives/reverse_proxy
- Caddy request-body limits: https://caddyserver.com/docs/caddyfile/directives/request_body
- Render Node version selection: https://render.com/docs/node-version
- Render persistent disks: https://render.com/docs/disks
- Render custom domains: https://render.com/docs/custom-domains

Continue with Operations, backups and recovery. Keep that chapter available offline as well as in the project, because an outage may make the website unavailable.

CHAPTER 08

08 Operations, backups and recovery

Your live store is an operating system, a running application, a private database, original files, encryption keys and several external accounts. Updating the visible pages is only one part of maintaining it. This chapter gives you repeatable operating procedures for the Ubuntu/Docker/Caddy deployment in the hosting chapter. For a managed native Node service, use the same consistency and recovery principles with that provider's disk and backup tools; the Docker commands do not apply to a native service.

The examples assume you are signed in over SSH as the `deploy` user, the deployment folder is `/opt/rtr-digital`, the Compose project is `rtrdigital`, the services are `app` and `caddy`, and the production data volume is `rtrdigital_rtr-data`. Replace `realdomain.example` with your own domain. A changed Compose project or volume name must be reflected in your operating notebook and every backup command.

1. Establish your operating notebook

1. Create a private folder or password-manager collection named **RTR Digital operations**. Store it somewhere you can reach when the website or your main computer is unavailable.
2. Record the domain registrar, DNS provider, host, server region, public address, SSH username, server recovery-console procedure and billing contacts. Record renewal dates and enable the relevant account security controls.
3. Record the deployed application release, Node version/image, Compose configuration, Caddy configuration, persistent-volume name and the date of your last successful restore drill. A backup filename without a successful recovery test is incomplete evidence.
4. Keep merchant-account identifiers, payout-bank records, email-domain settings, MFA recovery codes and credentials in protected storage. Do not put passwords or keys into a public issue, the storefront, a guide shared with customers, or a Git repository.
5. Decide how much data you could accept losing and how long an outage you could tolerate. These are business decisions: if losing a day of orders is unacceptable, a single nightly backup is insufficient. More frequent consistent backups or an engineered database architecture may be required.
6. Give yourself a calendar reminder for backups, disk checks, refunds, renewals, updates and a restore drill. Assign an authorized technical helper if you cannot perform server recovery yourself. Avoid making one lost phone the only way into every account.

2. Daily, weekly and monthly work

Daily checks while the store is trading

1. Open your public HTTPS homepage and `/api/health`. The API response should contain `status: ok` and `demoMode: false`. This checks reachability and application mode; it does not prove that checkout or a large download works.
2. Sign in with your owner account and review **Administrator**, **Publishing studio** and **Commerce office** as appropriate. Read new support tickets and refund requests. Check genuinely pending wire transfers against your bank, rather than confirming a customer screenshot.
3. Review the payment providers' dashboards for failures, disputes and payout warnings. A provider payout to your bank and a customer order inside RTR Digital are different records; investigate mismatches using the provider reference and application order reference.
4. Check that your latest planned backup exists, completed successfully and has an off-server protected copy. A server disk failure can destroy both the store and a backup kept only on that disk.
5. Watch storage after large uploads. Retained release versions and unfinished upload reservations use space in addition to the current file. Do not wait for the operating system to report a completely full disk.
6. Keep a short incident log: time in your chosen operating timezone, affected account/order, symptom, action and outcome. Do not paste secrets or unnecessary customer data into that log.

Weekly checks

1. Download an entitled original using a controlled customer account. Open a PDF or play a small audio/video sample if you sell those formats. Inspect the actual delivered file, not only its listing.
2. Review draft products, publication schedules, failed resumable uploads and license-key stock for software that needs keys. A public preview should still contain only the intended public excerpt.
3. Reconcile a sample of orders, refunds and payouts. Check whether a refund is pending or confirmed before changing customer access through another route. Keep provider evidence in your normal protected accounting records.
4. Review log sizes and disk usage. Review certificate/domain expiry alerts, delivery failures from your email provider, and security notifications from your host and merchant accounts.
5. Confirm that you can locate the matching application release and configuration for your newest backup. Keep a copy of the owner manual offline.

Monthly and after significant changes

1. Perform an isolated restoration check from a selected backup. Include original files and encryption keys, not just a database that happens to open.
2. Review access to hosting, Git, DNS, email and merchant accounts. Remove access that is no longer authorized using each service's own controls.
3. Evaluate supported security updates for Ubuntu, Docker, Node, Caddy and npm dependencies. Test application updates on a separate environment before production.

4. Review storage/bandwidth growth, backup retention and capacity. Test whether your own operating response meets the recovery targets you set.
5. Recheck seller contact information, public policies, current product rights and membership benefits. Historical receipts are preserved snapshots; changing today's seller settings does not rewrite old receipts.

3. Read useful server status without exposing secrets

Open your SSH terminal and run:

```
cd /opt/rtr-digital
sudo docker compose ps
sudo docker compose logs --tail=100 app
sudo docker compose logs --tail=100 caddy
df -h
sudo docker system df
sudo docker volume inspect rtrdigital_rtr-data
```

`compose ps` should show the application and Caddy running. The application health check can be transitional immediately after startup. `df -h` shows host filesystem capacity; `docker system df` also helps distinguish image/build-cache growth from application data. Volume inspection tells you which volume you are examining. It does not validate its contents or create a backup.

Do not paste an entire log into public support. Inspect it first for customer identifiers, private URLs and provider details. Do not run a command that prints the complete container environment or `.env.production` simply to prove that a variable exists. Check the relevant setting in your private editor or hosting secret controls instead.

To follow application output briefly:

```
sudo docker compose logs --follow --tail=50 app
```

Press Ctrl+C to stop following. This leaves the running container intact. In contrast, stopping the foreground `npm run dev` process on your local computer stops that local development server.

4. What a complete backup must contain

Back up the **whole private data directory**, including hidden files. At the default location it contains `rtr.sqlite`, SQLite journal/WAL-related files when present, uploads, versioned originals, preview/cover data, upload-session state and key files. The application creates or uses these important encryption-key sources:

Item	Why you must preserve it
`.account-encryption-key`	Decrypts file-backed account MFA secrets. If you deliberately use `ACCOUNT_ENCRYPTION_KEY` instead, preserve that unchanged secret in the protected configuration backup.
`license-key-master`	Protects stored product activation-key inventory. A newly generated key does not decrypt an old inventory.

Item	Why you must preserve it
`.payment-encryption-key`	Protects saved gateway credentials and bank-setting snapshots. Restoring only the database can leave these settings unusable.
`rtr.sqlite` and its consistent SQLite state	Accounts, orders, entitlements, settings, schedules, support records and other application records.
All original and archived files	The database points to private media; a database-only restore does not recreate a book or software archive.

Also preserve the matching source/release archive, `compose.yaml`, `Caddyfile`, `.env.production`, any external secret-store records, and your recovery instructions. Caddy's certificate volumes are separate from the business data volume. A new properly configured Caddy installation can obtain fresh certificates, subject to DNS and issuer limits; retaining host/provider snapshots can still help recover the entire machine.

SQLite uses write-ahead logging. Copying only `rtr.sqlite` while the server writes to it may omit committed transactions or produce an inconsistent collection of database and media files. A live filesystem copy is not the backup procedure below. The simplest procedure for this deployment pauses the application briefly and copies the entire volume while it is stopped. Arrange a maintenance window and avoid interrupting an active large upload.

5. Make a consistent cold backup, step by step

These commands run on the VPS. Keep this terminal open throughout the procedure; the shell variables do not survive closing it. Read the expected result after each command and stop to investigate unexpected output. Commands containing `sudo` can ask for your SSH user's sudo password.

Prepare before stopping the application

1. Confirm the volume name and your working directory:

```
cd /opt/rtr-digital
sudo docker compose ps
sudo docker volume inspect rtrdigital_rtr-data
```

2. Download the small utility image before the maintenance pause so the download does not extend downtime:

```
sudo docker pull alpine:3.22
```

3. Create a private backup parent directory owned by the current SSH user, then a uniquely named subfolder:

```
sudo install -d -m 700 -o "$(id -u)" -g "$(id -g)" \
  /var/backups/rtr-digital
umask 077
rtr_backup_dir="/var/backups/rtr-digital/$(date -u +%Y%m%dT%H%M%SZ)"
mkdir "$rtr_backup_dir"
```

The timestamp is explicitly UTC to avoid ambiguous filenames across servers. Your customer-facing operating notebook can also record the corresponding India time. Do not reuse a previous backup folder, and do not place this folder under `public/`, `dist/` or a downloadable website directory.

Stop, copy and restart

4. Stop only the application service:

```
sudo docker compose stop app
sudo docker compose ps -a
```

The app should be stopped/exited. Caddy remains available but requests may return a gateway error during this short maintenance window. Inform customers through your normal channels when scheduling maintenance; this command does not create an automatic maintenance page.

5. Archive the entire volume, including hidden key files, through a read-only source mount:

```
sudo docker run --rm --network none \
  --mount type=volume,src=rtrdigital_rtr-data,dst=/source,readonly \
  --mount type=bind,src="$rtr_backup_dir",dst=/backup \
  alpine:3.22 sh -c 'cd /source && tar -czpf /backup/data.tar.gz .'
```

Wait for the command to finish without an error. The archive includes the contents represented by `.`; it does not use a wildcard that skips hidden files. Its source mount is read-only. Do not run another application container against the same volume during this copy.

6. Restart the application whether the archive succeeded or failed, unless you are intentionally keeping the store offline to investigate a damaged disk:

```
sudo docker compose start app
sudo docker compose ps
```

If step 5 failed, mark the backup as failed and investigate after service is restored. Do not label a partial archive as a successful backup merely because the app restarted.

7. Save the matching deployment configuration and image inventory without printing secrets:

```
sudo cp -p compose.yaml Caddyfile .env.production "$rtr_backup_dir/"
sudo docker compose images > "$rtr_backup_dir/images.txt"
sudo chown -R "$(id -u):$(id -g)" "$rtr_backup_dir"
chmod 700 "$rtr_backup_dir"
chmod 600 "$rtr_backup_dir"/*
```

8. Test the archive structure, record its checksum and check the live service:

```
tar -tzf "$rtr_backup_dir/data.tar.gz" > "$rtr_backup_dir/file-list.txt"
sha256sum "$rtr_backup_dir/data.tar.gz" > "$rtr_backup_dir/SHA256SUMS"
chmod 600 "$rtr_backup_dir/file-list.txt" "$rtr_backup_dir/SHA256SUMS"
curl -fsS https://realdomain.example/api/health
```

Inspect `file-list.txt` in your private editor for `./rtr.sqlite`, original-media paths and the key files described above. An installation using the environment-supplied MFA key may not have the file-backed account key; confirm that the matching environment secret is in the protected configuration instead. `tar -tzf` and a checksum detect some corruption and transfer problems; they do not replace a restore drill.

Keep a protected off-server copy

9. Record the absolute backup-folder name in your notebook. From a terminal on your own computer, use `scp -r` or your SSH file-transfer application to copy that exact folder to a private location. The syntax is `scp -r deploy@YOUR_SERVER_IP:/var/backups/rtr-digital/EXACT_TIMESTAMP PRIVATE_LOCAL_FOLDER`. Replace all three placeholders; do not copy that literal example.

10. Use encrypted storage for this copy, and restrict access. The archive contains customer information and keys; the configuration contains secrets. A checksum verifies integrity, not confidentiality. Do not upload this backup to the public project-download branch or include it in a customer-facing ZIP.
11. Keep an additional protected copy independent of the production server and your main computer, according to your chosen backup policy. If using cloud storage, configure private access and encryption and test the recovery credentials.
12. Record success only after verifying the copied archive's checksum. On Windows PowerShell, use `Get-FileHash -Algorithm SHA256` with the actual local archive path; on macOS use `shasum -a 256`; on Linux use `sha256sum`. Compare the full hexadecimal value with `SHA256SUMS`.

For automated backups, have a qualified operator convert this sequence into a script with error checking, a restart-on-failure trap, notifications and retention controls. A cron entry that copies an active SQLite file is not an equivalent automation. Schedule the job to avoid uploads and account for its temporary downtime.

6. Restore a backup into a separate volume first

A restoration check must not begin by overwriting your live data. The following procedure makes a new volume and checks it without running the full web application or contacting merchant/email services. Use a trusted backup created by your own procedure. Do not extract an untrusted archive as root.

Create and populate the new volume

1. On the VPS or a separate recovery machine with Docker, make the protected backup folder available. Set the absolute path to your selected folder and choose a new, unique volume name:

```
rtr_restore_source=/var/backups/rtr-digital/REPLACE_WITH_EXACT_TIMESTAMP
rtr_restore_volume=rtrdigital_rtr-data-restore-REPLACE_WITH_DATE
test -f "$rtr_restore_source/data.tar.gz"
```

Replace both uppercase fragments before proceeding. Never choose `rtrdigital_rtr-data` as the restoration target in this procedure. Keep using the same terminal so the variables remain defined.

2. Verify the archive checksum against its accompanying record, then create the new volume:

```
sha256sum "$rtr_restore_source/data.tar.gz"
sudo docker volume create "$rtr_restore_volume"
```

Compare the checksum before extracting. Confirm the volume name was not used previously. `docker volume create` returns an existing volume unchanged, so it is not proof that the destination is empty. If the name already exists, choose a new unused name and inspect it before continuing.

3. Extract only into the new volume and set the ownership expected by the application image:

```
sudo docker run --rm --network none \
  --mount type=volume,src="$rtr_restore_volume",dst=/restore \
  --mount type=bind,src="$rtr_restore_source",dst=/backup,readonly \
  alpine:3.22 sh -c \
  'tar -xzpf /backup/data.tar.gz -C /restore && chown -R 1000:1000 /restore'
```

Do not relax private file modes to `777`. The ownership correction permits the image's normal UID 1000 user to access the restored files. Inspect any extraction error before proceeding.

Check SQLite without launching the store

4. From the deployment folder, obtain the application's existing image identifier and run a one-off database check with no network:

```
cd /opt/rtr-digital
rtr_app_image=$(sudo docker compose images -q app)
sudo docker run --rm --network none \
  --mount type=volume,src="$rtr_restore_volume",dst=/app/data \
  "$rtr_app_image" node -e '
const { DatabaseSync } = require("node:sqlite");
const db = new DatabaseSync("/app/data/rtr.sqlite", { readOnly: true });
console.log(db.prepare("PRAGMA integrity_check").all());
console.log("Foreign key issues:", db.prepare("PRAGMA foreign_key_check").all().length);
db.close();
'
```

The integrity result should be `ok`, and foreign-key issues should be zero. Investigate any other result. This command does not start `server/index.mjs`, and `--network none` prevents the utility container from reaching providers. The command needs the matching application image to exist locally; a new recovery host must first obtain/build the matching source image. Check that `rtr_app_image` contains one image identifier before using it.

5. Inspect the restored file inventory privately. Confirm the expected media and hidden keys exist. Compare at least one restored original's checksum with the original release record or a known reference. Check that the matching environment-supplied key is available if that installation used it.

6. Record the backup timestamp, restored volume name, checksum, SQLite results and media check. An offline check establishes that the recovered data can be read; it does not prove a public checkout works.

A complete browser drill needs a separately configured HTTPS staging deployment with controlled test accounts, no shared production data volume, and provider/email isolation. A restored database can contain encrypted **live gateway settings**, so removing environment variables alone does not make that database safe for an unrestricted staging server. Have your technical operator isolate outbound services and sanitize settings before starting the full recovered application. Never expose a restored production database publicly as an unprotected test site.

7. Use a restored volume for an actual recovery

Only follow this section when you have decided to recover the store, have verified the backup, understand which newer orders/changes would be lost, and have recorded a reconciliation plan. A backup from yesterday cannot contain today's sales. Payment providers may still hold real successful charges that are missing from that older database.

1. Preserve the current volume and configuration, even if they appear damaged. Capture a further cold backup if practical. Do not delete or extract over the original volume.

2. Stop public access and the application during the switch:

```
cd /opt/rtr-digital
sudo docker compose stop caddy app
cp -p compose.yaml compose.before-recovery.yaml
```

3. Edit the bottom `volumes:` section of `compose.yaml`. Keep the service mount as `rtr-data:/app/data`, but give that logical volume the exact restored engine name you checked:

```
volumes:
  rtr-data:
    external: true
    name: rtrdigital_rtr-data-restore-REPLACE_WITH_DATE
  caddy-data:
  caddy-config:
```

Replace the placeholder with the real restored volume name. `external: true` requires that named volume to exist; a typo should fail rather than create a new empty volume. Keep the Caddy volumes and project name unchanged.

4. Restore the matching protected configuration secrets if needed. Do not arbitrarily replace encryption keys with new ones. Use a compatible application release; a schema from newer code may not be compatible with older code.

5. Re-create the application against the selected existing volume:

```
sudo docker compose up -d --no-deps --force-recreate app
sudo docker compose logs --tail=100 app
sudo docker compose exec app node -e \
  'fetch("http://127.0.0.1:3001/api/health").then(r=>r.json()).then(console.log)'
```

Inspect the output. Caddy remains stopped, so the public store is still closed during these checks. Do not start a second app using the old and restored datasets behind the same public domain.

6. Resolve the recovery gap with your provider records before reopening checkout. Identify successful payments/refunds after the backup timestamp, pending subscription states, unprocessed webhooks and customer access affected by the gap. Use supported reconciliation or engineering assistance; editing random database rows can create duplicate entitlements or inconsistent refunds.

7. When the recovery is ready, start Caddy, test HTTPS, sign in, open a known order, inspect a product original and check owner account security. Keep the original volume intact until the recovered service has been verified and retained backups exist.

```
sudo docker compose start caddy
curl -fsS https://realdomain.example/api/health
```

8. Update your notebook and every future backup command to use the **new actual volume name**. The original `rtrdigital_rtr-data` example is no longer the active data source. Backing up the old name would silently preserve the wrong dataset.

8. Update code and containers without losing business data

1. Review the change and its release notes. Identify whether it affects only the frontend, server behavior, dependencies, environment settings or database structure. Test it on an isolated deployment before touching a store with customers.

2. Obtain the new complete source into a separate release folder. Do not unzip over the production folder before reviewing what would change. The release must not contain live `data/`, secrets or unrelated customer exports.

3. Make and verify a fresh cold backup and protected configuration copy. Record the current image identifier and give it a clearly dated local tag so a rebuild does not remove your only convenient code fallback.

```
cd /opt/rtr-digital
rtr_previous_image=$(sudo docker compose images -q app)
sudo docker image tag "$rtr_previous_image" \
  rtrdigital-app:before-REPLACE_WITH_DATE
```

Replace the date fragment. Record the image identifier and matching source archive. A local image tag is a code fallback, not a data backup.

4. Put the reviewed source into `/opt/rtr-digital/app` using your chosen deployment process, preserving the deployment-level files and data volume. Do not run an unreviewed synchronization command with a broad `--delete` against `/opt` or the private storage location.

5. Build before replacing the running application. A successful build does not change the database in the running volume:

```
sudo docker compose build app
```

6. During the planned change window, deploy the new image:

```
sudo docker compose up -d --no-deps app
sudo docker compose ps
sudo docker compose logs --tail=100 app
curl -fsS https://realdomain.example/api/health
```

The container may briefly be unavailable. Test owner login, a catalog page, a controlled customer's library, an original download and the changed functionality. Check that production mode remains active and records persist.

7. If only environment values changed, use `sudo docker compose up -d --no-deps --force-recreate app`. A simple `docker compose restart` keeps the container's existing environment and may leave changed values unapplied. If a frontend source file changed, rebuild; restarting the old compiled image does not compile your new React/CSS files.

8. Keep the previous release, image and backups until the change is stable. Do not immediately run Docker image pruning. Never use `docker compose down -v` or indiscriminate `docker volume prune` as routine maintenance: they can destroy the persistent business data.

For a code rollback, have your operator point the app service at the recorded previous image and recreate it without changing the data-volume mapping. Do not assume an old application can safely read a database changed by a newer release. If a database rollback is required, stop the store and use the recovery/reconciliation procedure; that decision can discard newer transactions.

9. Respond to common incidents

The homepage shows a gateway error

1. Check `docker compose ps` and the application logs before changing DNS. Caddy can be healthy while the application is stopped, restarting or failing initialization.

2. Check host disk capacity and recent changes. If the API cannot start because the data path is unwritable, confirm the correct volume and UID 1000 ownership; do not switch the app to root as a quick fix.
3. Confirm Caddy's reverse proxy target is `app:3001` on the Compose network. Avoid opening a public port 3001 to work around an internal service-name mistake.
4. If a controlled restart is appropriate, run `sudo docker compose restart app`, then inspect logs again. Repeated blind restarts do not repair corruption or restore lost keys.

Records seem to have disappeared after a deployment

1. Stop creating replacement customer/product records. Check whether the project name, volume name, `DATA_DIR` or managed-service disk mount changed.
2. Inspect the active container mounts and existing volumes privately. A fresh empty volume can coexist with the original intact volume.
3. Stop the app before reconnecting the intended data volume. Preserve both volumes until you know which records belong to which period. Do not merge SQLite files by copying one over the other.

Uploads fail or customers cannot download

1. Check free disk space, the product's current original, its publication state and the account's entitlement. A product card or public preview is not proof that the protected original exists.
2. For standard uploads, check the 100 MiB original and 10 MiB cover limits. For larger originals, use resumable publishing and review the configured size, reservation and inactivity limits. Proxy failures can happen before the API receives a request.
3. If a restored database references a file that was not backed up, restore the matching complete media backup. Do not replace a paid original with a public URL as a workaround.

Owner login or MFA fails

1. Check the correct domain, HTTPS, account email and device clock. Production secure cookies require HTTPS. An authenticator's time drift can invalidate otherwise correct-looking codes.
2. Use a previously saved recovery code or configured password-recovery email workflow as applicable. Recovery codes and account-encryption keys serve different purposes.
3. If keys were lost after a restore, locate the matching protected backup. Deleting the key file to generate a new one does not decrypt existing MFA data.
4. Bootstrap environment settings do not reset or promote an existing account. Do not delete the production database to recreate your owner login. Escalate to a qualified administrator with a backup and a controlled recovery plan.

A key or merchant credential may be exposed

1. Restrict affected access and preserve an incident timeline. Rotate the exposed external credential through its provider, then update the correct application/host secret and validate the integration.
2. Treat a committed secret as exposed even if you remove the file in a later Git commit. Follow the provider's revocation process; hiding the latest file does not revoke an earlier credential.

3. Review sessions and affected account access. Do not casually rotate the application's data-encryption keys: encrypted saved values require a planned migration or restoration, not an arbitrary replacement file.
4. Keep customers' and providers' required incident obligations under review with appropriate professional advice. This manual does not determine notification law or tax treatment for your business.

10. Your launch and recovery acceptance record

Before opening sales, record a result for each item: date, environment, person performing the check, evidence reference and any unresolved issue. Mark a test as **not performed** when that is the truth. Local automated tests, a sandbox charge and a live bank settlement prove different things.

Check	What you should observe
Production identity	Public HTTPS works, `demoMode` is false, owner login is real, shared demo access is unavailable.
Publishing	A real licensed original is uploaded, a deliberate public preview is separate, and published metadata is correct.
Customer rights	Purchase/member/team access follows the intended entitlement; an unrelated signed-out user cannot fetch the original.
Payment path	Each enabled provider's intended test/live flow has been verified in that merchant account, including authoritative completion and refunds.
Memberships	Recurring Stripe versus prepaid alternatives are correctly described; cancellation and expiry behavior have been tested.
Email and security	Verification/recovery mail reaches the controlled inbox; owner MFA and recovery codes work.
Persistence	A restart/recreation preserves accounts, products, files and settings.
Recovery	A recent complete backup has a protected off-server copy; an isolated restore passes database, media and key checks.
Commercial readiness	Correct seller details, policies, rights, support contact and professional tax/invoice decisions are recorded.

Retain this record with your private operating notebook. Re-run the affected checks after changing domains, storage, providers, membership pricing, rights rules or deployment architecture. The software delivery and this manual do not themselves establish a live merchant connection, a completed bank settlement or a successful production recovery.

CHAPTER 09

09 Reference and troubleshooting

Use this chapter when you know what you want to do but need the correct address, setting, limit or first diagnostic step. Read the relevant worked procedure in the earlier chapters before changing a live installation. A browser error, a payment-provider status and an application order status describe different parts of the same journey; record all three when investigating.

1. Address and role reference

In local development, prepend `http://localhost:5173` to a page path. On your public installation, prepend your actual HTTPS domain. Thus `/publisher` means `http://localhost:5173/publisher` while learning locally, and `https://your-actual-domain/publisher` after deployment. The text `your-actual-domain` is a placeholder, not an address you should buy or type literally.

Page path	Purpose	Who should use it
`/`	Home page, category discovery and membership introduction.	Everyone.
`/explore`	Search, category/price filters, sorting and product discovery.	Everyone.
`/collections`	Curated product discovery.	Everyone.
`/compare`	Side-by-side comparison of up to three products.	Everyone; choices are stored in that browser.
`/products/:id`	One product's details, previews and entitled content.	Visitors see public details; authorized customers see their content.
`/membership`	Explorer, Creator and Studio pricing.	Everyone.
`/library`	Owned/included products, downloads, orders and recurring membership controls.	Signed-in customer or administrator.
`/members`	Trial, membership activity and Studio workspace controls.	Signed-in account; eligibility determines available controls.
`/account`	Profile, verification, password, MFA, sessions, preferences and account data.	The account owner.
`/billing`	Customer receipts and refund requests.	The account owner.
`/admin`	Store dashboard, standard product upload, orders and support inbox.	Administrator.

Page path	Purpose	Who should use it
`/publisher`	Drafts, releases, resumable uploads, previews, publishing and keys.	Administrator.
`/operations`	Payment connections, seller details, promotions, refunds and financial records.	Administrator.
`/about`, `/support`, `/policies`	Brand information, support intake and published policies.	Everyone.

The `:id` part is a real product identifier supplied by a product link. Do not navigate to a literal colon-and-id URL. Opening an administrative address does not grant the administrator role. A normal customer registration cannot create an administrator. The Creator membership is a customer plan and does not create a seller or publisher account.

For a basic availability check, open `/api/health` on the same origin. A successful response establishes that the application process answered that request. It does not establish successful email delivery, payment capture, sufficient upload space, a usable backup or correct customer permissions. `/api/auth/me` is a useful same-browser authentication diagnostic: a missing user means that request did not carry a valid accepted session.

2. Environment-variable reference

Environment variables configure the server process. They are separate from product metadata and payment settings saved through the administrator interface. Edit the hosting environment or the intended private environment file, then restart/recreate the application as described in the hosting chapter. Refreshing a browser does not restart the server.

Node does not automatically read a file named `.env`. A direct local command such as `node --env-file=.env.local server/index.mjs` explicitly loads that file. Docker Compose `env_file` supplies variables to its container. Keep the correct file outside the public web roots and outside Git. Do not place credentials in frontend `VITE_` variables; frontend values are distributable to visitors.

Variable	Meaning and operating rule
`NODE_ENV`	Use `production` on the public installation. Production disables shared demo accounts and requires secure session cookies.
`PORT`	Backend listening port; default `3001`. Keep the reverse proxy and container configuration consistent.
`API_PORT`	Backend port override recognized by the server; takes precedence over `PORT`. Normally leave it unset in this deployment.
`DATA_DIR`	Private persistent data directory. Default is the project's `data/` directory. Use the deployment's mounted absolute path in production.
`DEMO_MODE`	Development demo behavior; `false` disables it. `NODE_ENV=production` disables demo access regardless of this value.

Variable	Meaning and operating rule
ADMIN_EMAIL	Initial administrator email. Must be a valid non-demo address that does not already exist in this database.
ADMIN_PASSWORD	Initial administrator password, at least 12 characters. Supply together with ADMIN_EMAIL, then remove the bootstrap password after account creation and restart.
PUBLIC_URL	Exact public HTTPS origin used for checkout and account links. Do not leave a localhost address or an .example domain in production.
STRIPE_SECRET_KEY	Server-only Stripe secret for the intended test/live environment. Never publish it in frontend code.
STRIPE_WEBHOOK_SECRET	Signing secret for this installation's Stripe webhook endpoint, not an API key.
STRIPE_AUTOMATIC_TAX	Default false. Set true only with the appropriate Stripe Tax configuration and business registrations.
TRIALS_ENABLED	Production no-card trial switch; default false. Original included content and verified email are also required.
RESEND_API_KEY	Server-only outbound transactional email credential. Production recovery/verification needs configured delivery.
EMAIL_FROM	Sender identity authorized by the email service, for example your actual verified domain's account address.
ACCOUNT_ENCRYPTION_KEY	Optional stable encryption input set before enrollment of authenticator accounts. Preserve it if used; do not change it casually on an existing installation.
STRIPE_PRICE_STARTER_MONTHLY, STRIPE_PRICE_STARTER_YEARLY	Known provider price IDs for the Explorer plan. Internal STARTER naming is intentional.
STRIPE_PRICE_CREATOR_MONTHLY, STRIPE_PRICE_CREATOR_YEARLY	Known Creator provider price IDs.
STRIPE_PRICE_STUDIO_MONTHLY, STRIPE_PRICE_STUDIO_YEARLY	Known Studio provider price IDs, important for team eligibility after provider plan changes.
MAX_DIGITAL_UPLOAD_BYTES	Maximum resumable original size in bytes; default 5368709120 (5 GiB).
UPLOAD_CHUNK_BYTES	Resumable chunk size; default 8388608 (8 MiB). Server caps this setting at 32 MiB.
MAX_PENDING_UPLOAD_BYTES	Aggregate pending-upload reservation allowance. Default is twice the configured maximum file size: 10 GiB with the default maximum.

PayPal credentials, Razorpay credentials, wire instructions and seller details are managed in Commerce office rather than invented environment variables. Their saved records depend on the private database and matching encryption keys. Preserve .payment-encryption-key, license-key-master, and

`.account-encryption-key` when present, or the stable account encryption value used instead. The complete data directory and private deployment configuration belong in your protected backup plan.

3. Working limits and format behavior

MiB and GiB are binary units: 1 MiB is 1,048,576 bytes and 1 GiB is 1,073,741,824 bytes. Some screens abbreviate these as MB/GB. A limit is a permitted ceiling, not a promise of available storage or upload speed. Proxy limits, free disk, quotas, connection interruptions and browser resources can impose a lower practical ceiling.

Feature	Current rule
Standard original upload	Up to 100 MiB per original; use the resumable workflow for larger files.
Standard-upload cover	Valid PNG, JPEG or WebP, up to 10 MiB. A renamed executable is not a valid image.
Resumable original	Default maximum 5 GiB; default 8 MiB chunks; configurable capacity with available private disk.
Pending resumable capacity	Default aggregate 10 GiB reservation allowance and three active sessions per administrator.
Inactive upload session	Expires after 48 hours without activity. Retaining the original locally remains your responsibility.
Public preview	Separate deliberately public asset, maximum 50 MiB. Text preview maximum is 1 MiB.
Protected original text viewing	Valid supported UTF-8 text up to 1 MiB; larger text remains download-only.
Software activation-key import	1–500 genuine keys per submitted inventory batch.
Product original	One current original file; retained releases may provide earlier versions. Use a ZIP when delivering several formats together.
Comparison	Three selected products.
Studio workspace	Five seats total, including owner and unexpired pending invitations.
Workspace invitation	Single-use, bound to the invited email, expires after seven days.
No-card trial	Seven days, once per eligible account/email; no automatic charge.
Pending wire instructions	Seven-day validity; administrative reconciliation is still required.

A PDF book can be viewed inline when its validated format and entitlement allow it. EPUB, ZIP archives, applications, executables, HTML/SVG and other unsupported original formats are delivered as downloads. Do not advertise an in-browser EPUB reader, executable preview or streaming support for every possible codec. A browser's own codec support also affects audio/video playback.

At the time of this manual, the Explore price slider has a fixed USD 150 ceiling and defaults to that ceiling. Products priced above USD 150 can therefore be absent from Explore even though the server accepts a higher product price. Resetting the filters retains that ceiling. Follow the customization chapter's source-change instructions before relying on Explore to sell higher-priced products. This is a discovery limitation, not a universal USD 150 checkout limit.

4. Diagnose a local setup problem

“node is not recognized,” an unsupported SQLite error, or an old runtime

1. Open a new terminal after installing Node. Run `node --version` and confirm version 24.2 or later; prefer the latest patched 24.x LTS version for these instructions.
2. Run `npm --version`, or `npm.cmd --version` in Windows PowerShell. If either command is missing, correct the installation/PATH and reopen the terminal.
3. Check that the terminal is in the extracted application folder containing `package.json`. A folder containing only `LICENSE` or download PDFs is not the application source.
4. Run `npm ci` from the correct folder. Preserve `package-lock.json`; do not delete it merely to suppress a dependency error.

PowerShell says npm.ps1 cannot run

Use `npm.cmd ci`, `npm.cmd run dev` and `npx.cmd` for the Windows commands. This invokes the command shim without changing your computer's script-execution policy. If PowerShell cannot find those commands either, resolve Node installation/PATH first.

The page loads but sign-in, uploads or the bag fail

1. Read both terminal outputs. The frontend may be running while the backend exited.
2. In ordinary development, use the Vite address on port 5173 and ensure Express is listening on port 3001. `npm run preview` alone does not start Express.
3. Open `http://localhost:3001/api/health` directly. If it does not answer, address the backend's printed error before troubleshooting browser styling.
4. If you changed the backend port, update the development proxy configuration consistently. Do not assume changing one environment value rewrites every tool's configuration.
5. Keep one consistent browser origin while testing. `localhost` and `127.0.0.1` are different hosts, and separate browser profiles also have independent cookies and storage. Different ports have separate local storage and origin permissions, but cookies are scoped by host/path rather than port; do not rely on changing only the port to isolate accounts.

A port is already occupied

Check whether an earlier project terminal is still running and stop your own process with Ctrl+C. On Windows, `Get-NetTCPConnection -LocalPort 3001,5173 -ErrorAction SilentlyContinue` helps identify listeners. On Ubuntu, `ss -ltnp` lists listening ports. Identify the owning process before stopping anything; do not terminate an unrelated database or another person's service to make a port available.

Environment changes appear to do nothing

Check the exact filename, including hidden extensions, and the command that actually loads it. A file called `.env.local.txt` is different from `.env.local`. Restart the process after changes. Container environment changes require recreating the affected service, as documented in the hosting chapter. Do not use `node --env-file=... --run dev` as a substitute for the documented two-terminal setup: the script subprocess may not receive the values as expected.

5. Diagnose access, content and administration

No administrator navigation or “administrator required”

Confirm the signed-in email and account role. In development use the explicitly labeled Admin workspace demonstration button for training. In production use your real bootstrapped administrator. Setting bootstrap variables to an existing customer email does not promote that account. Repeated ordinary registration will not solve this. Follow the administrator creation/recovery procedure and preserve the database before administrative changes.

Production sign-in seems to succeed, then immediately disappears

Production sessions use secure cookies. Open the public HTTPS address through the configured reverse proxy, check the certificate and keep the same hostname. A plain HTTP localhost test is not equivalent to a correctly deployed production HTTPS session. Check `PUBLIC_URL`, proxy configuration and `/api/auth/me` in that same browser. Never solve this by publishing an insecure production cookie configuration.

A product is missing, unpublished or unavailable to members

Check the selected product's status, original release, publication schedule and member-inclusion setting in Publishing studio. Production hides sample products. A draft without an original cannot be published as a real sale. Check the Explore USD 150 filter limitation above and try the actual product link. For member access, check plan status and dates; team access follows the workspace owner's active Studio eligibility. An expired team grant does not remove an independent purchase.

The uploaded book cannot be read in the browser

Confirm that the file is a supported validated original, the account has entitlement and the intended current release completed successfully. Test its download. EPUB/ZIP files being download-only is expected. A public preview is a separate asset; uploading a full original does not automatically designate the entire book as a free preview. Keep the original local source until you have verified exact delivery.

Resume upload rejects a file, stalls or says capacity exceeded

Reselect the exact original bytes; matching only the filename and size is insufficient. Check session expiry, the three-active-session allowance, global reserved capacity, free disk and proxy request limit. Finish or cancel an abandoned session through the provided workflow rather than deleting private staging files manually. A large final checksum operation can take time; read the displayed state before starting duplicate uploads.

Recovery email, MFA or an invitation does not work

Production recovery/verification needs a valid HTTPS origin and configured sender delivery. Check spam, provider delivery records and the exact recipient. Development private mailbox previews do not prove production delivery. Authenticator codes depend on correct device time; use an unused saved recovery code when appropriate. Workspace invitations are not automatically emailed: the owner copies the private link. Confirm invited email, expiry, single use and available seats. Never put MFA codes or recovery links in a public support screenshot.

6. Diagnose payments and hosting

The gateway returned the customer, but nothing was unlocked

The return page does not grant access. Check the order/intent reference, correct test/live environment, recipient, exact USD amount, provider capture status and server verification. For Stripe, confirm signed deliveries to `/api/stripe/webhook`, the matching endpoint signing secret and the event's processing result. Read provider errors and application logs together. Do not manually create an entitlement to conceal an unresolved payment mismatch.

A wire customer submitted a reference, but access remains locked

This is expected until the administrator verifies cleared funds against the preserved receiving instructions, exact USD amount, unique bank reference and current order state. A screenshot, “sent” status or customer report is not cleared bank money. Reconcile an expired/conflicting payment before attempting fulfillment. Editing bank details changes future instructions; it does not silently rewrite an existing pending order's snapshot.

MRR looks lower than total sales

MRR describes eligible recurring subscriptions. Product purchases and PayPal/Razorpay/wire prepaid membership terms are not recurring MRR. Demo records are separated from live financial records. Compare the same time interval, currency and payment status rather than adding unrelated figures together.

The public site returns 502, 413, a certificate error or an empty catalog

For 502, check application/container status and the proxy's upstream service/port. For 413, inspect the configured request-body limit; resumable delivery uses chunks, but standard upload plus cover still needs adequate headroom. For certificate errors, confirm real DNS records, firewall ports 80/443 and Caddy's certificate logs. For an empty production catalog, publish original products: production intentionally hides sample entries. Avoid replacing the data volume with a fresh one during troubleshooting.

Records disappear after redeployment, or settings cannot decrypt

Stop repeated deployments and identify the actual mounted data directory. An ephemeral or changed path can create a fresh database while the previous data still exists elsewhere. Missing encryption keys can make matching database records unreadable. Restore the coherent private backup into a new recovery volume using the recovery chapter, including all matching keys; do not generate replacement keys and expect old encrypted values to recover.

7. Boundaries and support evidence

Four payment routes are implemented: Stripe, PayPal, Razorpay and reconciled wire transfer. Provider approval, India/global availability, USD acceptance and payout arrangements remain account-specific. This application does not create merchant accounts, automatically link a real bank, supply unlimited infrastructure or integrate every international payment company.

Activation-key inventory is not a vendor DRM/activation server. Additional integrations are required for malware quarantine, shared object storage/CDN, multi-instance database architecture and jurisdiction-specific statutory tax invoices. Payment receipts are not statutory GST/export invoices. Confirm your own rights, accounting, legal policies and retention arrangements before launch.

When asking a developer or host for help, supply the time with timezone, affected page, nonsecret order/product reference, exact error, steps to reproduce, software version and redacted relevant logs. State whether the problem is local, staging or production, and test or live payment mode. Never attach the private database, environment file, encryption keys, customer originals or full payment credentials to a public issue. Keep an untouched backup before a repair.

Short glossary

Frontend: the pages and interactions seen in the browser. **Backend/API:** the running server that validates requests and manages access. **Database:** stored business records; SQLite is the database used here. **Original:** your actual protected digital product file. **Preview:** a separately approved public sample. **Release:** a version of an original. **Build:** compiling the frontend into `dist/`. **Deployment:** putting the application and configuration on a host. **Persistent volume:** storage that survives container replacement. **Origin:** protocol, hostname and port together. **Webhook:** a provider's server-to-server event delivery. **Entitlement:** a verified right to access a product. **MRR:** recurring monthly revenue measure, not all receipts. **Rollback:** returning application code to a prior compatible release. **Restore:** recovering private operating data from backup.